The Labyrinth

A generated dungeon, decomposed into container combinators

1 Description of the game

A party of three descends into a labyrinth, armed with daggers, to receive a sacred item and comes back up. The labyrinth is a tree of depth eight. It is generated as they descend and it does not exist until somebody tries to go there. The code of each level is generated by Claude, from a specification of that level which is also generated by Claude. The game carries an element of chance and the difficulty level can be configured.

Each level contains a maze (generated by Claude when he generates the level). Some of the doors are guarded by monsters, some of them are not. You can either kill the monster and take its items, try to convince the monster to open the door, or trade your items with the monster. Trap doors lead down into lower levels of the labyrinth. These are guarded by gate-keepers. To convince the gate-keeper to open the trap door you must solve a riddle, so some of the monsters also have clues.

There are 12 types of monsters and each type of monster has secrets and desires and subjects they enjoy talking about. The monsters are LLMs. You can either negotiate with them, chat with them, trade with them or try to kill them.

The monsters also have relics. There are ten types of relic, each of a different weight. The party can only carry 6 at a time (6 hands) one for each hand - including a dagger and a lamp. You can collect healing potions and lamp oil but these must be carried.

You must discover the monsters' secrets and desires to solve the labyrinth. It is a turn-based console game with a finite list of possible moves at each turn - including negotiating with and chatting to the monsters. The monsters can lie, and some of the other monsters know that they are lying.

You can also come back up the labyrinth. The most valuable relics lie deepest in the labyrinth and you must go down into the labyrinth to collect them then come back up with the items to negotiate with the monsters who guard the doors on higher levels. The sacred object is on the top level guarded by a monster that you must trade with to open the door using a relic from the lowest level.

In the future we could add a simple GUI with a drop down menu of options, a way to view your inventory and an image for each room, monster and item (generated by Claude from a description using an image generator). Claude also generates the descriptions.

The point of the game is to illustrate agentic container programming. That Claude can also generate the specification, monster prompts, and images from descriptions is an added twist. Claude "optimizes" the parameters to make sure it is actually solvable and "interesting". Whether or not the game really is interesting to play remains an open question.

Claude draws from history, religion and mythology to add an element of creativity to the game. It could also be sci-fi.

2 The game

2.1 The sacred item is given, not taken

$$
\begin{aligned}
\text{run} &= \text{descend}^8 \lhd \text{receive} \lhd \text{ascend}^8 \\
\text{receive} &= \text{ask} \lhd^* (\text{given} + \text{refused})
\end{aligned}
$$

Somewhere in the labyrinth is somebody holding the sacred item, and it will hand it over if it is satisfied. It cannot be killed for it, cannot be robbed of it, and there is no other copy.

2.2 What is inside a room, what a level is, and the two recursions

$$
\begin{aligned}
\text{labyrinth} &= \text{level}(1) \\
\text{level}(d) &= \text{room}_d(1) \\
\text{room}_d(i) &= \text{round} \lhd \text{room}_d(i) \lhd \text{exit}_d(i) \\
\text{exit}_d(i) &= \text{door}_1 \times \cdots \times \text{door}_k \times \text{trap}_d(i) \qquad k = 0 \ldots 4 \\
\text{door}_j &= \text{room}_d(i_j) \\
\text{trap}_d(i) &= \text{level}(d{+}1) \ \text{ or nothing, and there is at most one} \\
\text{level}(9) &= \text{nothing. There is no ninth level}
\end{aligned}
$$

There are two ways to leave a room, either you leave the room into another room on the same level or you descend to the level below. A door does the first and a trapdoor does the second. A room has zero to four doors and zero or one trapdoors.

The rooms of a level are a maze. Every room has a coordinate and its doors run to its neighbours on the four compass points, which is where the arity of zero to four comes from. There is no way of walking in a circle, so between any two rooms of a level there is exactly one path, and a party that walked past something has to retrace to reach it.

And exactly one room on a level has the trapdoor. There is one way down from each level and no choice about which, so the levels are a tower and not a tree.

When a level is generated, all the rooms are generated, and all the monsters are generated. When you descend to a level below a whole new level is generated.

2.3 Every level has its own algebra

What is written above is a schema and not the game. How many rooms there are, which of them a door joins, which one holds the trapdoor and which doorways are guarded are not fixed anywhere in this note. They are the algebra of one level, and Claude writes it when you enter that level.

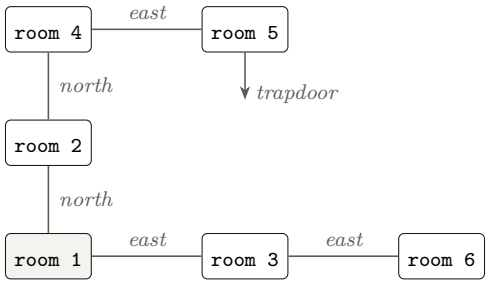
> Which is the whole of the claim in one sentence. The operators are fixed and the expression is generated. Every level is a different expression in the same algebra, so what the compiler checks is the schema and what the player meets is one instance of it, drawn once and never seen again.

2.4 An example of a level

Six rooms and one trapdoor. Written out because a schema with no instance is hard to read, and because every level will be different from this one. What stands in any of these rooms is one recursion down and is the subject of Section 3.

$$
\begin{aligned}
\mathrm{level}(4) &= \mathrm{room}_4(1) \\
\mathrm{room}_4(1) &= \mathrm{round} \lhd \mathrm{room}_4(1) \lhd \big(\mathrm{room}_4(2) \times \mathrm{room}_4(3)\big) \\
\mathrm{room}_4(2) &= \mathrm{round} \lhd \mathrm{room}_4(2) \lhd \big(\mathrm{room}_4(1) \times \mathrm{room}_4(4)\big) \\
\mathrm{room}_4(3) &= \mathrm{round} \lhd \mathrm{room}_4(3) \lhd \big(\mathrm{room}_4(1) \times \mathrm{room}_4(6)\big) \\
\mathrm{room}_4(4) &= \mathrm{round} \lhd \mathrm{room}_4(4) \lhd \big(\mathrm{room}_4(2) \times \mathrm{room}_4(5)\big) \\
\mathrm{room}_4(5) &= \mathrm{round} \lhd \mathrm{room}_4(5) \lhd \big(\mathrm{room}_4(4) \times \mathrm{level}(5)\big) \\
\mathrm{room}_4(6) &= \mathrm{round} \lhd \mathrm{room}_4(6) \lhd \big(\mathrm{room}_4(3)\big)
\end{aligned}
$$

Drawn as a map, with the doors on the compass points the note already uses.



2.5 The table, and why knowing it is the way out

Before a game begins, Claude draws up a table. Nobody sees it. It is different every time.

axis	what is drawn	rows
kinds of inhabitant	the flavours that exist, and what each kind wants	a handful
colours of item	which colour of thing each kind of inhabitant wants	a mapping
topics of interest	what each individual likes talking about	one apiece
motives	who lies, and about what	one apiece
secrets	who knows which true thing	one apiece

Your job is to discover the table. Once you know it you can find your way out of the labyrinth, and until you know it you cannot.

That is the win condition and it is epistemic rather than spatial or material. The haul is what you happened to be carrying when you got out. The openings are the record of what you learned on the way. Neither is the object. The object is the table, and everything in this note is a way of buying one row of it.

2.6 Half of it compresses and half of it does not

Which is why the game needs two faculties and not one, and it falls out of the table's shape rather than out of anybody's preference.

	how big	**and so**
the colour mapping	one rule, a few cases to identify	**compressible**. Induce it once and every desire in the labyrinth is known
interests, motives, secrets	one row per inhabitant, no pattern	**incompressible**. There is nothing to induce and each must be got individually

So a good player induces the core and buys only as much of the fringe as the way out requires. A player who tries to enumerate everything runs out of light, and a player who tries to induce everything discovers there was no rule to find, because interests genuinely are one apiece.

> And that is the honest reason the note has both an inductive mechanism and a market. Not variety. The hidden object has a compressible part and an incompressible part, and no single faculty gets at both.

2.7 Which makes the oldest claim in this note literally true

Section 7.9 argues that the two mappings of a container are not symmetric, that the way back is where the game lives, and Section 2.13 says nothing found at the bottom is worth anything until it has been carried up.

That has been an argument about architecture with a game draped over it. Under the table it is the rule.

going down	costs light and nerve and very little knowledge. Anybody can fall into a labyrinth
coming up	the ways out are operated, in the sense of Section 13.2, by inhabitants whose prices you must know. So the exit is priced in table rows

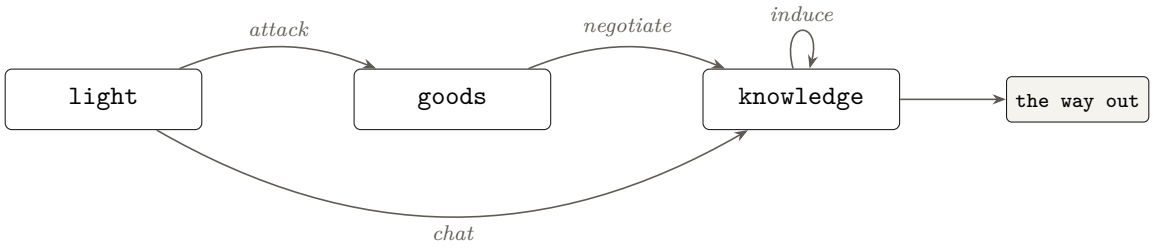
The deeper you went, the more of the table it takes to get back, because there are more operated ways between you and the surface and each one wants something particular. Which means depth is not merely risky. Depth is a debt denominated in knowledge, and the descent of Section 12.1 is borrowing against a table you have not read yet.

> So the failure state has a shape and it is a good one. You are on the sixth level, you have light for eleven rounds, and the thing operating the way up wants something you cannot identify because you never found out what its kind wants. Nothing is chasing you. You are simply somewhere you cannot afford to leave, and you put yourself there one incurious room at a time.

2.8 How all of it fits together

The note accretes a good many mechanisms and they are not a pile. There are three quantities, three actions that convert between them, and one of the three quantities is what you win with.

	measured in	and it is
light	rounds, and descents	the only thing you begin with
goods	weight, out of your capacity	the intermediate, and never the point
knowledge	rows of the table	the only thing you need



Read the arrows. Attacking turns light and hit points into goods, because a corpse yields what it carried. Negotiating turns goods into knowledge, at three turns and dagger's length. Chatting turns light straight into knowledge and skips the middle, which is why the poorest party can still play. Inducing turns knowledge into more knowledge for nothing, which is the only free move in the game. And knowledge is what the ways out want.

Which explains the three ends of Section 12.17 better than that section does. They are a balance sheet. **The haul is goods you did not spend**, so a rich party is a party that failed to convert. **The openings are knowledge you did convert. Getting out is whether you converted enough.** The reason there is no exchange rate between them is that they are three readings of one conversion, taken at the end.

2.9 What each thing is, and what it is for

	what it is	how you get at it
a **kind**	a flavour of inhabitant. Finitely many, and every one of a kind wants the same thing	you see it. It is what a thing visibly is
a **desire**	what a kind wants. An item colour, or a secret	*induce* it from the rule, or buy it, or ask and be lied to
a **secret**	one true thing an individual knows	buy it with goods, or be given it for good conversation
an **interest**	what one individual likes talking about. Not patterned, one apiece	buy it from somebody else, or guess from the flavour
a **motive**	what the truth would cost it, which decides what it lies about	*infer* it from the pattern of what it says. It cannot be asked
an **item**	a colour, a weight, a value and a rarity	take it from a corpse, a hoard, or a trade
a **loadout**	zero to three weapons, individual	look at it, before you decide how close to go

2.10 The two things that refer to each other

Everything above is a flat list and the game is not flat, because two of the rows point at each other.

> **A desire may be for a secret. A secret may be what somebody desires.**

That mutual reference is the whole structure. It makes the inhabitants a graph rather than a population, it makes a trade a walk on that graph, and it makes the graph's own shape something worth buying, since a secret may be *what another one wants*.

graph	$A \to B$ **when**	**and it is**
desire	A wants what B has or knows	the terrain. Walk it backwards to trade
interest	A knows what B likes	the cheap map, paid for in patience
knowledge	A knows what B wants	the dear map, and the most valuable thing there is

2.11 And where each kind of uncertainty comes from

Four, and each wants a different faculty, which is why no single kind of cleverness finishes this game.

you do not know	**because**	**so you**
what a kind wants	the rule is redrawn each game	*induce*, from cases
whether you are told the truth	motives are individual	*deduce*, from consistency
what one likes talking about	it is one apiece and free	*buy*, or guess and waste turns
whether it is enjoying itself	nothing in the program decides	*be interesting*, and there is no other route

So the four faculties are induction, deduction, commerce and conversation, and the table of Section 2.5 has a part addressed by each. A player strong at one and hopeless at another gets a particular distance and no further, which is the breadth of Section 24.5 arising from the structure of the hidden object rather than from balancing.

2.12 The populations, and why there are three

	armed	**moves**	**and it is for**
a **guard**	yes	no	being the obstacle, and being the price
a **roamer**	some	yes	being the market, and the honest evidence of what things take
a **civilian**	no	no	being the witness, and the only free information there is

Each exists because the others cannot do its job. A guard that would chat is a guard you talk past for free. A civilian that guarded something would make conversation compulsory. A roamer that stood still would leave nothing in the labyrinth that moved while you were not looking, and Section 15 says that is the difference between a world and a puzzle box.

> And the last line of the whole arrangement is the one to keep hold of. Light is the only thing you are given, knowledge is the only thing that gets you out, goods are what the middle of the game is made of, and there is nothing in the labyrinth that converts knowledge back into light. So every game is spent one way, and the only question is how much of the table you bought with what you had.

2.13 Puzzle pieces

Scattered through the lower levels are puzzle pieces. Each one is both a *clue* and an item. It is a thing you pick up and carry, but it cannot be sold, and having it in your hands tells you something you did not know. Some lie in rooms and some are carried by monsters, so a clue can be taken from a corpse, bought in the market, or simply found.

The pieces belong to a riddle, and the riddle is somewhere above where they were found. Guarding it is a gate-keeper, which is the kind of monster that does not attack and the kind that stands on a trap door. You give it the pieces you have gathered, and then you try to solve the riddle. This is a conversation not a command. You put forward your argument and the gate-keeper decides whether or not to accept it and open the trap door.

> That is why the way back matters more than the way down. Nothing you find at the bottom is worth anything until you have carried it back up, and what you have carried back up is the only thing that makes new depth reachable. In order to fully explore the labyrinth you must unlock it from below.

Three things about it are unusual and all three are the point of building it.

The levels are not written by hand. A specification for a level is generated, and the code for that level is generated from the specification, and the level is then a program like any other. So the labyrinth is different every time and nobody has to design it.

The monsters are non-deterministic. Each one is a language model with a finite set of actions, which you can speak to in plain English. You can threaten them, lie to them, offer them money or ask them what is down the passage, and what they do about this is their own business.

The puzzles are non-deterministic too, and nothing in the game knows their answers. A puzzle is solved when the monster keeping it is persuaded, and being persuaded is the only sense in which it is solved.

2.14 The party

A party is the group of people you take down, and for the purposes of this decomposition it is one thing rather than several. It has a number of hands, a pool of hit points shared between them, a stock of experience gained by getting past things, and whatever it is carrying. It acts once in a round, because the word you type is given to the party and not to a person inside it. When the note says the party is wounded, it means the pool is low and not that a named individual is hurt.

That is a decision and not an oversight. A party of three who each act separately would be a

tensor, standing opposite the tensor of monsters facing them, and it would be the better game. It would also make a round $(a \boxtimes b \boxtimes c) \triangleleft$ monsters, and every rule about actions would have to be written per person rather than per party. This note describes the smaller version, and the larger one is the obvious first thing to add.

2.15 The roster

A party is not the same thing as everybody you have, and the difference is where the game keeps its longest-range decision.

At the surface there is a roster of people. A party is a subset of the roster that you send down, and how many go is chosen before each descent. People who die do not come back, and the roster does not refill.

So a person is two things at once, and this is the point of having a roster at all. A person is a hand, which is carrying capacity on this descent. A person is also a body, which is a descent you can still make later. Spending one to bring up a relic is spending the other, and the two costs fall due at different times.

the roster	everybody left alive at the surface. Never grows.
the party	who is going down this time, and therefore how many hands there are
the muster	the choice of how many to send, made before each descent

Send four and you carry more and lose more when it goes wrong. Send two and you carry little and keep a reserve for a trip you have not planned yet. That is the muster, it is a product, and it is the only decision in the game whose consequences are two descents away.

A party can also be lost outright, and what happens then is settled by a decision already taken elsewhere. Section 23.7 keeps every level alive after the party has climbed out of it, so a loss takes the people and leaves the labyrinth exactly where it stood. Every lock opened from below is still open, every level already written is still written, and the map of Section 27 is still drawn. The next party is drawn from whoever is left.

Which is the property of Section 2.16 used serially by one player instead of concurrently by two. People are what you lose and the labyrinth is what you keep. Nothing new has to be built for it, because the levels were being kept anyway.

2.16 Multi-party

The levels are servers and a server may have many clients, so two parties descending at once is not a new capability. It is a capability the architecture already has but is not yet using. Nothing about the two mappings changes: each party is its own descent, its own tree, and its own way back, because a response goes to whoever asked for it.

What it buys is worth more than it costs. If a party unlocks a door from below, that door is open when the next party arrives, so the labyrinth is unlocked between them rather than by each of them separately. Each party can gather separate pieces.

The cost.

A level has to hold state, and once it does, the same prompt no longer gives the same response, because somebody else has been through. That is the same concession already made for the monsters rather than a new one. Apparently Neil has a solution for this using *workers*.

If two parties are in the same room at the same time, who goes first? Nothing said so far decides that. A combinator says how prompts and responses fit together, but it has nothing to say about whose turn it is.

So a rule has to be chosen, and then an operator will express it. The tidy choice is that the two parties are a tensor, exactly as the monsters in the room are. Both are prompted with the state as it stood at the start of the round, both owe a result, and neither of them sees what the other did until the round is over. Nobody goes first, because they all go at once.

Potentially two parties could reach for the same thing at the same time. This is the ordinary problem of shared state, but the architecture has solved it already, because each level is one program with one flow of control. Requests to a level are served one at a time, so the state moves from one consistent position to the next without anything having to be locked. That is the reason the whole game is built out of servers rather than function calls in the first place.

3 What is inside a room

Here is the algebra.

$$
\begin{aligned}
\text{room}_d(i) &= \text{description} \times \text{item} \times \text{way}_N \times \text{way}_S \times \text{way}_E \times \text{way}_W \times \text{trap} \\
\text{description} &= \text{text, and no rule reads it} \\
\text{item} &= \text{nothing} + \text{one item} \\
\text{way}_x &= \text{nothing} + \big(\text{guard} \times \text{room}_d(i_x)\big) \\
\text{trap} &= \text{nothing} + \big(\text{guard} \times \text{level}(d{+}1)\big) \\
\text{guard} &= \text{nothing} + \text{monster}
\end{aligned}
$$

3.1 An example of a room

Room 3 of Section 2.4, written out as one instance of the algebra, because a product with five sums in it is easier to read once than to imagine.

$$
\begin{aligned}
\text{room}_4(3) &= \text{description} \times \text{item} \times \text{way}_N \times \text{way}_S \times \text{way}_E \times \text{way}_W \times \text{trap} \\
\text{description} &= \textit{a low room, empty but for the marks where} \\
&\quad \textit{something heavy was dragged out of it, and} \\
&\quad \textit{an east door newer than the wall it sits in} \\
\text{item} &= \text{nothing} \\
\text{way}_N &= \text{nothing} \\
\text{way}_S &= \text{nothing} \\
\text{way}_E &= \text{a monster} \times \text{room}_4(6) \\
\text{way}_W &= \text{no guard} \times \text{room}_4(1) \\
\text{trap} &= \text{nothing}
\end{aligned}
$$

Five of the seven factors are the empty branch of their sum, which is what an ordinary room looks like. The room has two doors because two of the four compass points are not nothing, it is a corridor because its item is nothing, and it is worth walking through only because one of its two doors has somebody standing at it.

And read the description against Section 25. It mentions that something heavy was dragged out and that the east door is newer than its wall, which is prose that a careful player can read as a hint about what lies east. Nothing in the program reads it. That is the leak being deliberate, and it is the only thing in this room that rewards attention rather than resources.

4 What is a monster

Everything in the labyrinth that is not an item is one of these, and it stands at a way.

$$
\begin{aligned}
\text{monster} &= \text{kind} \times \text{items} \times \text{secrets} \times \text{interest} \times \text{motive} \\
\text{kind} &= \text{one of twelve, and it fixes the desire and the clue} \\
\text{engage} &= \text{attack} + \text{chat} + \text{negotiate} + \text{trade}
\end{aligned}
$$

The first line is what it is and the second is what you may do about it. The product is not a list of features. Each factor is fixed by something different, and which one decides which is the whole of what a player has to learn.

	fixed by	**so it is**
kind	the labyrinth	one of twelve, and there are several monsters of each. It settles what the monster wants and which clue it holds
items	the individual	what it is carrying, taken from it if you kill it and given if you satisfy it. Lamp oil is the common one
the clue	the kind	every monster of a type holds the same one, so a clue is learnable once and then known everywhere, exactly like the want
secrets	the individual	true things it knows, one apiece, bought one at a time and with no pattern to induce
interest	the individual	a subject it likes being talked about, one apiece, and the only thing in the game that is free
motive	the individual	whether it lies and about what, per Section 19.2, never readable off it and only inferable from the pattern

4.1 Light and health, and where they come back from

Two things run down and neither refills itself.

	it goes down	**and it comes back from**
light	every turn, whatever the turn was spent on. Walking, talking and stabbing all cost the same	lamp oil, which is an item with a weight like any other and takes a round to use
health	only in a fight, and it is a pool shared by the whole party	a healing potion, on the same terms

So light is the clock and health is the price of one of the four choices. A party that never fights never loses health and still runs out of light, which is what makes the lamp the thing that ends

a descent and violence merely the thing that can end it sooner.

where oil comes from	lying in a room, or carried by a monster
how you get it off a monster	trade for it, or kill it and take it. Those are two of the four and they cost very different things

Which puts the sharpest trade in the game on the most ordinary item. A monster holding oil is a monster that can be killed for the very thing the killing spends, so violence pays for itself in the resource it consumes and in nothing else. Trade the same monster and you keep what it knows and pay in goods instead. The choice is between spending light to save goods and spending goods to save light, and it is offered again at every lamp.

4.2 The four things you can do to one

You engage one monster at a time, because a monster stands at one way and you deal with one way at a time. There is one weapon and it is a dagger, so attacking needs no range, no drawing and no choice of arm.

	it costs	**and it buys**
attack	light, hit points, and whatever the corpse could have said	its items, and the way it stood at, unless the way was one only a living thing can open
chat	light, and nothing else	liking, and a chance of being told something if you talk about what it likes
negotiate	goods, and the turns it takes to name a price	a secret, or a way opened, if what you offered met what it wanted
trade	an item	an item. The only one of the four that moves goods in both directions

Which is the conversion diagram of Section 12 with the fourth arrow drawn in. Attacking turns light into goods, chatting turns light into knowledge, negotiating turns goods into knowledge, and trading turns goods into other goods. Only one of the four is reversible and only one of the four is free, and they are not the same one.

And attacking is the only one that cannot be taken back. A corpse is mute, so killing forecloses every secret that monster held, every clue it was carrying that you did not take, and every way it alone could open. The other three can all be tried again next round. This is the single asymmetry that makes the choice a choice rather than a preference.

5 A note for whoever generates a level

> **A level is a maze. No cycles, laid out in two dimensions so that it can be drawn as a map, and exactly one room in it holds the trapdoor.**

Which is stronger than it sounds, and the reason is already in Section 3. Four compass points and at most one door on each means a level is a subgraph of the square lattice, and being acyclic makes it a spanning tree of one. That is a thing a generator can be asked for and a checker can verify, which a vague instruction to make it maze-like is not.

5.1 Generate the maze first, and the rooms from it

The order matters and it is the cheap way round.

	what it produces	and what it costs
1	the maze	a spanning tree on a patch of lattice, the coordinate of every room, which neighbours are joined, and which room holds the trapdoor. Ordinary code, no model, no tokens
2	the rooms	for each room, its description, its item, and a guard or nothing at each door the maze gave it. One Claude, one prompt, one level

The first stage is easy and that is the point of doing it first. Maze generation is a solved problem with a dozen textbook algorithms, none of which needs a language model, and running it first hands the second stage a frame it cannot get wrong. The room generator is no longer inventing connectivity while also inventing content. It is filling in a structure that is already acyclic, already planar, already has one trapdoor, and already tells each room how many doors it has and on which compass points.

6 The depth is a tower, and the prompt is what moves

The levels stack. There is one way down from each of them and no way from a level to a shallower one except back through the trapdoor you came by, so depth only ever increases while you are descending and only ever decreases while you are climbing out. Section 8 draws it, eight levels on eight ports with the same program running on each.

Nothing else joins them. A level never addresses another level, never holds a reference to one, and never learns that another exists. The only thing that crosses between two levels is a prompt going down and a response coming back, which is what makes the tower a tower rather than a graph of programs that happen to be stacked.

down, in the prompt	the party record. What the party is, what it holds, what it owes and what it knows
up, in the response	what happened. What was found, what was opened, what was said, and the description of where the party now stands

Section 28 gives the record row by row. The four kinds are worth separating here because they behave differently on the way back.

	examples	**coming back up**
the clocks	light remaining, descents left	always smaller
the goods	the purse, the items in hand, how many hands are free	larger or smaller, and it is the only kind that can go either way
the debts	obligations taken on and not yet discharged	larger on the way down and smaller on the way up, which is the whole of the ascent
the knowledge	secrets told, the rule induced so far	never smaller, and it weighs nothing

Read the last row against the second. Goods take hands and knowledge does not, so the only thing the party can accumulate without limit is the only thing it cannot be robbed of by running out of capacity. That asymmetry is not a convenience. It is why the game can be about finding out rather than about carrying.

And one thing does not travel. **The map stays with the client.** Where the party has walked is of no use to any level, since a level answers what is in it and never asks who is asking, so putting the map in the prompt would send every level information it has no rule that reads. Neither does what the party knows, and Section 6.1 is why. Where the party has *been* does not travel, and neither does what it has been *told*. What goes down is the goods, the light and the words the player chose to type.

6.1 A secret is prose, and nothing stores it

A monster answers an offer with a sentence of English. The program does not keep that sentence anywhere. It is not an item, it does not take a hand, it never enters the party record, and there is no screen that lists the ones you have been told. The player holds it in their own head and works out where it is worth using.

The reason is Section 31, which makes looking at the map and the inventory free because both are already in the client's hands. A stored secret would inherit that for nothing. And a free list of every secret you have been told is Section 2.5's table, filled in on the player's behalf, one row at a time, by the program whose win condition is that the player discover it.

So the program storing secrets would be the program playing its own win condition. That is the whole argument and it does not need a second one.

There is a price and it should be stated rather than discovered. Players will keep notes on paper, so this does not abolish the inventory of secrets. It moves it outside the program, which is where it belongs. Paper does not sort, does not search, and will not say which of eleven remembered sentences is the one this gate-keeper wants. What the program refuses to supply is not the memory. It is the indexing.

And it costs Section 29 one of its five tables. A goal whose positions are relics stays checkable, because the party either holds three pieces or it does not, and the fifth table makes an under-equipped argument unwriteable. A goal whose positions are *secrets* is checkable by nothing, because the party's knowledge is not in the program to be indexed. So the positions of a goal divide in two. The material ones are enforced by the compiler and the informational ones are enforced by a gate-keeper's judgement, and only the first kind can be made unwriteable.

7 Containers

The decomposition that follows is built out of one idea, and this section is all of it. Nothing here is particular to dungeons.

7.1 An interface is a pair

A *container* is an interface, written as two things together. The first is the set of prompts you may send. The second gives, for each prompt, the type of the response.

The second half is what makes it more than a list. The response's type depends on which prompt was sent, so a single interface can respond with a number to one question and a list of rooms to another, and which you get is fixed by what you asked. Write the pair as (P, R), where P is the prompts and $R\,p$ is the type of response to the prompt p.

One condition runs through everything below. The set of prompts must be *finite*. Over a finite set, a response type that varies with the prompt is just a table, and a table is something an ordinary type system can write down. Over an infinite set it is not, and none of this works.

7.2 A morphism is two mappings

Interfaces are joined by *morphisms*, and a morphism is a pair of mappings going in opposite directions. Given interfaces (P, R) above and (Q, T) below,

$$u : P \to Q \qquad\qquad f : (p : P) \to T\,(u\,p) \to R\,p.$$

The mapping u takes a prompt you were given and computes the prompt to send below. The mapping f takes the response that came back and turns it into the response you owe. Control flows down through u. Data flows back up through f.

Read the type of f slowly, because the whole of this note leans on it. It is given the original prompt *as well as* the response from below, and what it must produce is a response to the original prompt. So the way back up has access to the way down, and the two are not symmetric.

7.3 A command-line tool is already a container

None of this requires a special language. A command-line tool receives one prompt, its argument vector, and produces a response whose type depends on which prompt it was. Ask one thing and you get a list of commits. Ask another and you get a working tree. Those are not the same kind of thing, and which you get is fixed by the word you typed.

That is a container, and it is why the levels in this game are separate programs. One level reaches the one below it by spawning a command-line tool that carries a prompt to a socket, so the delegation is a real process with a real boundary rather than a function call wearing a costume.

7.4 Nothing checks what crosses

The boundary is text. A prompt goes out as an argument vector and a response comes back on standard output, and no type survives the crossing. What arrives is bytes.

There are two crossings in this game and neither carries a type. One is a pipe and the other is a socket.

The pipe

A pipe is a buffer the kernel hands out on request, with one descriptor to write into and one to read out of. The shell puts the writing end in the left program's slot 1, the reading end in the right program's slot 0, and runs both at once. Nothing is checked. The interface between two programs is bytes and any two programs may be joined, which is the strength of it. Nothing verifies that the composition means anything, which is the cost. What a program promises is written in its manual page and nowhere else, so the only thing enforcing the promise is that somebody read it.

The socket

The socket is the same bargain in a smarter suit. HTTP is a request and a response and nothing else happens, which is already the question and the answer of a container. A request is a method, a path, some headers and possibly a body. A response is a three-digit status, some headers and a body. The party descends in the body of a request and the haul comes back in the body of a response, and the body is JSON. Fielding made the same decision Unix made and it paid off the same way, so every web application in the world now speaks it. The price is also the same, because JSON carries no types.

So the response type never crosses

Only the prompt goes down. The response type is recovered on this side by the runner, which knows which prompt it sent, and that recovery is the dependent lookup of Section 29 doing its work at run time rather than at compile time.

Which is why all input must be validated

Every claim made above is established, if it is established at all, by a *decoder* on the receiving side: a function which takes what arrived and either produces a value of the type that was promised or produces nothing. Validate, do not cast. The keyword `as` is not a check. It is a promise to the compiler, and a promise from the far side of a pipe is exactly the thing that was never enforceable.

The decoders are a table with one entry per prompt, and it is the same mapped type as the handlers of Section 29. Delete one and the program does not build.

```
TS2741 [ERROR]: Property 'Look' is missing in type
                '{ Say: (u: unknown) => Action | null;
                   Offer: (u: unknown) => string | null; }'
                but required in type 'Decoders'.
```

This is the crossing that matters most, because the thing on the other side of it is a language model. Here is the decoder for `Say`, run against four things the `claude` binary might really put on standard output.

```
a legal action     -> "withdraw"
a ninth action     -> null, rejected at the boundary
an apology         -> null, rejected at the boundary
fenced JSON        -> null, rejected at the boundary
```

Read the last two. A model that answers a monster's prompt with an apology has not attacked, bargained or refused. A model that wraps its answer in a markdown fence has produced eleven characters of punctuation around a legal action. Neither is malice and both are ordinary, and both are stopped in the same place by the same three lines.

> The same rule covers the other direction, because the player is input too. What a player types is arbitrary text and it is routed to whatever is listening. A word that names no interface is not a move the game declines politely. It is a prompt that fails to decode.

The type says what must be true and the decoder is the only thing that ever establishes it. That is the arrangement throughout this game, and Section 21 is where it matters most.

7.5 What is a container here, and what is not

Before the expression, a distinction the note has been relying on without stating, and its absence is the reason the algebra gets confusing once rooms have squares in them.

A container is a set of prompts together with, for each prompt, the type of the response. So the test for whether something belongs in the algebra is one question, and it is not a question about how important the thing is.

Is it prompted, and does something come back?

Ask it of everything in the game and the answers sort into three groups, not two.

	role	and therefore
a level	**container**	prompted with a party, answers with a descent
a room	**container**	prompted with a turn, answers with what happened
a monster	**container**	prompted with words, answers with one of eight actions
a gate-keeper	**container**	prompted with an argument, answers with a verdict
the market	**container**	prompted with a purchase, answers with what you now hold
a square	*an index*	appears *inside* a prompt. `Move to (3,2)`
an item	*an index*	appears inside a prompt. `Offer the green stone`
a weapon	*an index*	likewise, and so does a colour
the grid's contents	*state*	remembered by a room, never crosses a boundary
which guards are dead	*state*	likewise
which ways stand open	*state*	likewise

7.6 The grid is an index, not an operator

Which is the specific confusion worth heading off, because there is an inviting wrong answer.

A room is a rectangle of squares, squares seem independent, and independence is what $\boxtimes$ is for. So a room looks like a tensor of squares. It is not, and it fails the test twice over.

nothing prompts a square	There is no question you put to the square at $(3, 2)$ and no answer it owes you. It has no interface, so it cannot be a fibre of anything
squares are not independent	Movement couples them. You may reach $(3, 2)$ only from a square beside it, and a tensor asserts that neither side can see the other

So the algebra composes the things that *answer*, and geometry is what one of them *remembers*. That is the whole of the distinction. A room is one container holding a grid, not a composite of many containers arranged in a rectangle.

7.7 But squares are in the type system, one level down

They are not absent from the types. They are in a different position in them, and seeing where is what makes the whole thing click.

Section 7 says a container is a pair of a shape type and, for each shape, a position type, and that the shape type is a *finite index*, being a tagged union. The squares of a room are finite. So they can index a shape.

the prompt	its shape	the response
walk one square	`{tag:"Move", to: Square}`	where you now stand
strike	`{tag:"Strike", with: Weapon}`	what it took off, or a miss
offer	`{tag:"Offer", give: Item[]}`	a secret, or nothing
say something	`{tag:"Say", words: string}`	one of eight actions

Read the middle column. `Square`, `Weapon` and `Item` all appear there, inside a shape, as the index of a prompt. That is what it means for the geometry to be typed without being composed.

And it is the same trick as the loot table of Section 29. A relic on level one is unwriteable because the item type is indexed by depth. A move to a square outside the room is unwriteable because the move's shape is indexed by that room's squares. Neither needed an operator. Both needed an index.

7.8 So the third column is where the danger is

The state row is the one to keep honest, because state is what the algebra does not see and therefore what the algebra cannot promise anything about.

A room holding a grid, a set of corpses and a record of which ways stand open is a room whose answers depend on things no prompt mentioned. Section 2.16 already conceded that when it allowed two parties, and Section 15 leans on it hard when it has the market settle between visits.

Which is worth writing down as a rule rather than discovering later. Everything in the state column is invisible to the type system and to every combinator in this note. So the only guarantees about it are the ones something checks explicitly, which is why the generator conditions of Section 14.2 are re-checked after every settle and why redundant discharge exists. The algebra protects the boundaries. It does not protect the memory behind them.

7.9 Down is not the same as up

The party that arrives at level four is not the party that entered level three. Gold has been found, wounds have been taken and lamp oil has been extinguished. That is the mapping u.

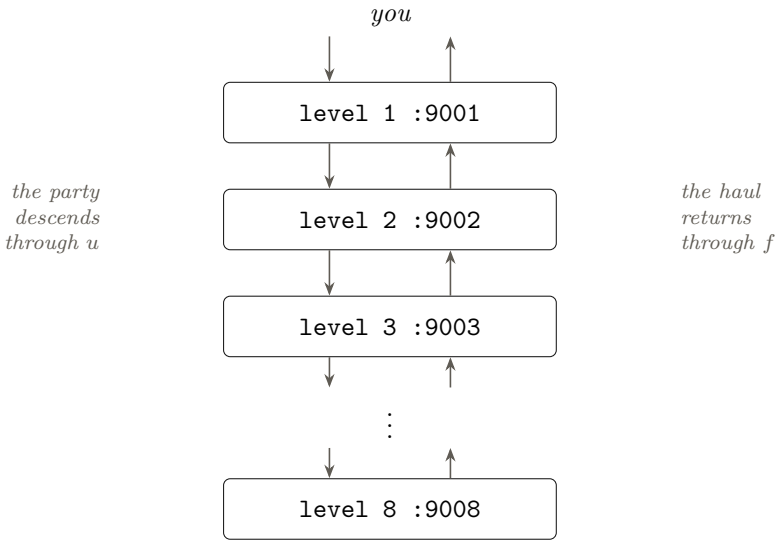
The mapping u is given one thing and the mapping f is given two. Its type is $f : (p : P) \to T\,(u\,p) \to R\,p$, so it receives the prompt that went down as well as the response that came back.

Suppose the party finds a key on level four. There is a locked door on level three, and the party walked past it on the way in because they had nothing to open it with. On the way out the mapping f of level three is handed the response from below, the key is in it, and the door is now a door the party can open.

A puzzle is the same mechanism and it is what the game is built on. A gate-keeper on level two wants three pieces, and the pieces are on levels four, five and six. The party cannot satisfy it on the way in and does not need to. The pieces come up through the responses, and the mapping f of level two is handed them along with everything else, so the argument can be made on the way out and not before. What the labyrinth is really made of is not the way down. It is what the way down lets you do on the way back.

8 The tower

Eight levels, eight ports, and the same program running on each of them.



Eight bespoke programs would be a game. The same program composed with itself eight times is a claim about composition, and the claim is the reason for building it.

The eight are not eight copies of the same content. Each is started with a level specification and serves whatever that specification describes, so the program is the same and the labyrinth is not.

8.1 It could be a tree and not a chain

The picture above is the simplest case and the equations are already more general than it. Every level ends in a choice of ways down, and each way leads to a level which ends in a choice of its own, so what the recursion describes is a tree. Eight in a line is what you get by never branching.

The depth of that tree is fixed before the game starts, and it is finite. That is not a concession, it is the condition everything else rests on. A finite depth is what lets a rule about what may be found at a given depth be a table, and a table is what the type system can hold. A labyrinth that went down for ever would have no such table, and nothing in Section 29 would work.

A tree is a maze with no way of walking in a circle, and that is what this labyrinth is. The generator may make it as large and as tangled as it likes. What it may not do is join a passage back onto one the party has already come down, because the way out is the way in reversed and there is nothing to reverse in a circle.

9 The decomposition

Interfaces compose, in four different ways, and all four appear below. A fifth operator appears too, written $\triangleleft^{*}$. It is the sequential one again, with a second prompt that may be chosen after the first response arrives, and Section 21 explains why that has to be written by hand rather than taken ready-made.

	reads as	**what it means here**
$\boxtimes$	both, and both respond	two parts of the game that do not touch each other
$\times$	both offered, one responds	two uses of a thing you have only one of
$+$	one side, chosen first	the prompt already belongs to one side
$\triangleleft$	then	the second prompt is computed from the first response
$\triangleleft^{*}$	then, with a choice	the first response decides which side is prompted

9.1 The whole game, as an expression

$$
\begin{aligned}
\text{game} &= \text{descent} \triangleleft \text{game} \quad \text{until the descents run out} \\
\text{game}(0) &= \text{reckoning} \\
\text{reckoning} &= \text{haul} \boxtimes \text{opened} \boxtimes \text{bottom} \\
\text{descent} &= \text{muster} \triangleleft \text{outfit} \triangleleft \text{level}(1) \triangleleft \text{market} \\
\text{muster} &= \text{one} \times \text{two} \times \cdots \times \text{r} \\
\text{outfit} &= \text{smith} \times \text{chandler} \\
\text{market} &= \text{sell} \triangleleft \text{buy} \\
\text{room}(d) &= \text{round} \triangleleft \text{room}(d) \triangleleft \text{exit}(d) \\
\text{exit}(d) &= e_1 \times e_2 \times \cdots \times e_k \quad \text{the doorways, and you take one} \\
e_i &= \text{room}(d) + \text{room}(d{+}1) \quad \text{laterally, or down, and possibly one you have seen} \\
\text{room}(9) &= \text{nothing. There is no ninth level} \\
\text{round} &= \text{move} \times \text{talk} \times \text{stab} \times \text{trade} \times \text{use} \\
\text{talk} &= \text{chat} + \text{ask} + \text{offer} \quad \text{all of them on its square} \\
\text{liking} &= \text{liking} \triangleleft \text{talk} \quad \text{it moves with every word} \\
\text{chat} &= \text{say} \triangleleft^{*} \big(\text{told} + \text{nothing} + \text{attacked}\big) \\
\text{offer} &= \text{give} \triangleleft^{*} \big(\text{honoured} + \text{robbed}\big) \\
\text{puzzle}(d) &= \text{gate-keeper}(d) \triangleleft^{*} \big(\text{wing}(d) + \text{refused}\big)
\end{aligned}
$$

Every line is one rule of the game, and the next section states which.

Three of those lines are worth reading twice, and Section 2.2 unpacks the first of them.

There is no separate rooms(d) and no separate way(d), because a room *is* a way and whoever stands on it is the price of taking it. The product over rooms and the sum over ways down were the same decision written twice, which is the whole of Section 10.1.

The line for liking is the only place in the note where a $\triangleleft$ carries something that is neither a party nor a response, and Section 11 is about why it is the centre of the game. It accumulates over turns, so every word is prompted with what the last word did.

And chat has three branches where every other composite has two. Told, nothing, or attacked, and which of the three arrives is the inhabitant's decision after you have spoken. That is the $\triangleleft^{*}$ doing the most work it does anywhere here.

9.2 One rule for each operator

Split in two, because the operators divide by how often the player meets them. The first table is the shape of a whole game and the second is the shape of a round.

part		the rule that forces it
the game	$\triangleleft$	**each descent is prompted with what the last one left behind**, and the count of descents is what makes the chain finite. Nothing refills it.
the reckoning	$\boxtimes$	**the three ends are computed independently and nothing joins them.** Each is answered from the same final state, none is a function of another, and no operator anywhere adds them together. A tensor is exactly the refusal to give a score.
the muster	$\times$	the roster offers every party size it can afford and exactly one of them goes. A person is a hand on this trip and a descent later, so this is the one choice whose cost falls due twice.
outfitting	$\times$	you have one purse, so a sword, a bow and a torch compete for it. Light bought is depth bought, which makes the chandler the least glamorous and most often correct answer.
the market	$\triangleleft$	what you can afford to buy is computed from what you sold.
the rooms	$\times$	**each room is one way onward with one guard on it, and getting past any of them descends.** Three rooms is three prices and you have one purse, so the other two are an offer and not a duty. A room you walk away from keeps its guard and its way.
a guarded way	$+$	dealing is answered by the monster and retreating is answered by the map, so which branch is prompted is settled by which of the two you chose.
your round	$\times$	move, talk, stab, trade or use, and exactly one of them, because a round is one action and all five spend the same light.
what you say	$\triangleleft^*$	**you speak and the thing decides what that was worth.** Told, nothing, or attacked. Not knowable when you open your mouth, which is what makes conversation the sequential operator and not a menu.

part		the rule that forces it
a command	$+$	**the options depend on where you are standing, and which of them you type decides who responds.** A blow goes to the monsters, a price goes to the market, a direction goes to the map. The three respond with different kinds of thing.
a round	$\triangleleft$	the monsters respond to your attack. Your blow is thrown first and the monsters respond to the result.
liking	$\triangleleft$	what you say next is prompted with where the last thing you said left it. This is the only accumulating quantity in the game.
the monsters	$\boxtimes$	every monster in the room acts at the same time.
the guard's answer	$\triangleleft^*$	you make the offer and the monster decides whether that settled it, so the branch is not knowable when the goods leave your hands.
a bargain	$\triangleleft^*$	**you hand the goods over and the monster decides whether that settled it.** Nothing enforces a bargain underground, so the second branch is being robbed, and that is why fighting stays worth doing at an affordable price.
a puzzle	$\triangleleft^*$	**the gate-keeper is asked first, and the gate-keeper's verdict decides what is prompted next.** Persuade it and a wing of the labyrinth is entered. Fail and nothing happens. The verdict is not knowable when the argument is made, which is what makes this the sequential operator and not a choice made in advance.
what you carry	$\times$	you have a fixed number of hands. Loot, keys, pieces, torches and arrows all want one, so what you bring up is chosen against what you leave behind.
the descent	$\triangleleft$	what waits on level $d{+}1$ is computed from what you did on level d, and the largest part of what is computed is how much light you have left.

9.3 The command is the routing

The player types one word. Where the party is standing decides who is allowed to respond to it, and the three who might respond do not respond with the same kind of thing.

the word	who responds	what comes back
`strike`	the monsters in the room	who was hurt, and by how much
`buy`	the market on the surface	what you now carry, and what it cost
`north`	the map of this level	where the party is now standing
`say`	whatever is listening	what it decided to do about that

Three interfaces, genuinely different, and the word names which one the prompt belongs to before it is sent. That is a sum, and unlike the trap door it is a sum on every turn of the game rather than once a level.

The set of words that exist is itself a function of where you are. In the market there is no monster to strike, and in a room with a monster in it there is nothing to buy.

9.4 Fighting

A round has three parts and each is a different operator.

You act first, and you have one action. Swing the weapon in your hand, spend an arrow, or put the weapon away and draw the other one. All three are offered and exactly one of them responds, because a round is one action and drawing is an action like any other. That is the product.

Every monster in the room is somewhere between one step away and five. A sword reaches a monster at one step. A bow reaches one at three steps or more, and every shot spends an arrow out of a quiver that holds few. At two steps neither weapon reaches anything, which is a gap you get out of by drawing or by backing away.

An arrow that has been fired is lying in the room, one step from whatever it hit. So the quiver is refilled by walking into sword range of the thing you were trying not to be near, and the bow is a weapon that pays for its own distance.

A monster moves one step in a round if it decides to. Approaching and withdrawing are two of the things it can do, alongside striking and talking, so where it is standing is something it decides rather than something the room fixes.

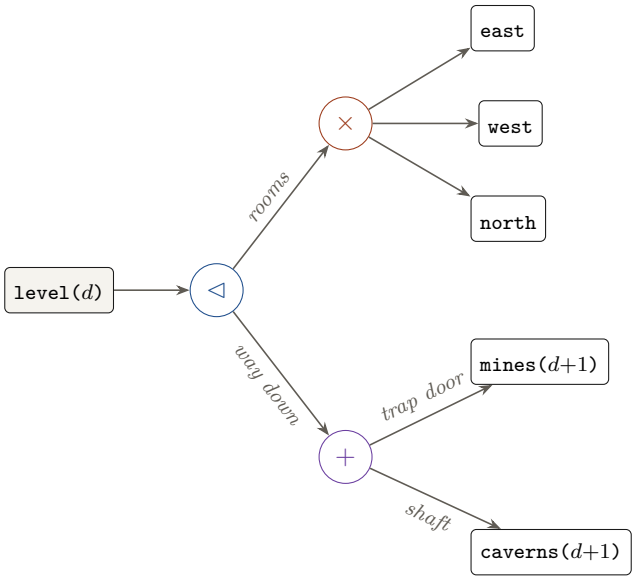
> Which turns distance into a count of rounds. A monster five steps off gives you three shots before it arrives, and every round you spend putting the bow away is one it spends closing.

Then the monsters respond. All of them act, all of them owe a result, and none of them waits for another to finish, so the monsters in a room are a tensor. A room with three monsters is $m_1 \boxtimes m_2 \boxtimes m_3$, and the independence is enforced rather than promised. All three prompts have to be built before any is sent, so the third monster's move cannot be computed from what the first one did.

And the round as a whole is sequential. What the monsters do is computed from what you did, which is the entire reason a round is a round.

That is where the distance earns its keep. You spend a round drawing the bow while the room is still far away, they spend it closing, and you finish the round holding a weapon that no longer reaches anything. The choice was made before their answer and it lives or dies by it, which is what the sequential operator is for.

9.5 One level, drawn



Read it left to right. A room is cleared, and only then is the way down chosen. Each leaf at the bottom right is another whole level, which is where the recursion is. The three rooms are a product and not a tensor, so the trap door waits on one of them and the other two are an invitation you may decline.

9.6 Where the recursion is

Only one equation mentions itself.

$$\mathrm{level}(d) \;=\; \big(\mathrm{east}(d) \times \mathrm{west}(d) \times \mathrm{north}(d)\big) \;\triangleleft\; \big(\mathrm{mines}(d{+}1) + \mathrm{caverns}(d{+}1)\big)$$

Expand it eight times and the dungeon is a right-nested chain of sequential compositions with a tensor hanging off each link. That is the whole architecture in one line, and it is also why the depth of the delegation and the depth of the dungeon are the same number.

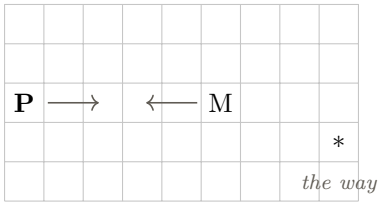
Expanded once it is a chain. Expanded at both ways down it is a tree, and the tree is the labyrinth. The expansion stops at the depth the game was given, which is why the recursion terminates and why the tables of Section 29 exist. What gets built is the path the party actually took, one level at a time, so a generated child need not exist until somebody goes that way.

10 The room is a grid

$$\begin{aligned} \text{level}(d) &= \text{east}(d) \times \text{west}(d) \times \text{north}(d) \\ \text{east}(d) &= \text{room} \lhd \text{level}(d{+}1) \\ \text{room} &= \text{round} \lhd \text{room} \end{aligned}$$

Three lines, and this section is what the squares underneath them are for. The product is which room, the chain is what the room leads to, and the inner chain is turns going by.

Distance has been a number between one and five and that was doing too little work. So a room is a rectangle of squares, everything in it stands on one, and a turn moves you one square.



five squares, three turns of closing

The geometry is the point and not the presentation. Three weapons, three bands of distance, and they do not overlap.

weapon	reaches	strength	and elsewhere
a dagger	the same square, which is dagger's length	least	useless at any distance at all
a long sword	one square, adjacent	most	too long to bring to bear on your own square, too short at two
a bow	two squares or more	middling	cannot be drawn inside two

So every distance has exactly one weapon that works at it, and at any moment two of your three are dead weight. The product of Section 9 stops being a table with eight misses in it and becomes a partition. Where you are standing decides what you are holding, and changing your mind about that costs a turn.

Strength is the second axis and it does not agree with range. The long sword is the strongest thing in the game and it works at exactly one square, so the strongest blow available is available at the distance you are hardest pressed to hold. The dagger is the weakest and it works where nothing else does. And because a weapon has a weight as well, the same non-domination condition applies here as Section 12.12 imposes on loot.

No weapon may beat another on range, strength and weight at once. Each must be the answer to something.

10.1 One way, one guard, three answers

The simplification that made the rest of this fit.

A room holds one monster and one thing it is standing on. What it is standing on is either a way onward or an item worth carrying. Nothing else is in the room worth deciding about, so

the party has exactly three answers available, and every level is that question asked as many times as it has rooms.

a guarded way	getting past it descends. Some room on every level must be one of these or the level is a dead end.
a guarded hoard	getting past it yields the item. Optional in every sense, and it is where the haul comes from.

That distinction does more work than its size suggests, and it is the reason to state it. A level with one way down and two hoards puts the first end against the third in every room the party walks into. Take the hoards and you are richer and one descent nearer the end of the game. Walk to the way down and you are deeper and carrying nothing. The muster of Section 2.15 asked that question once a trip and now the level asks it three times a level.

attack	pays in the physical economy if it works, and in people if it does not. What it was standing on is now yours, whether that was a way or an item
negotiate	pays in the information economy if it works, and it can only be attempted from one square away, which Section 10 is about
retreat	is walking, and it leaves the guard where it was

$$\text{round} = \text{move} \times \text{strike} \times \text{chat} \times \text{negotiate} \times \text{trade} \times \text{draw} \times \text{use}$$

One product, seven branches, one action a turn. Retreating is not a separate operator because it is walking, which Section 10 makes literal.

10.2 What it buys, and it is more than tidiness

Two operators collapse into one. The old note had a product over rooms and then a sum over ways down, and treated them as separate decisions. They never were. Clearing a room was only ever worth doing because of where the room led, so choosing a room and choosing a way were the same choice counted twice.

The room product becomes a choice of price. Three rooms is three guards with three desires, and one purse to meet them with. So the question at a level is not which room is easiest. It is which of these three prices can I pay, and the answer depends on what the party is carrying and what it has learned.

And it becomes a choice of end. Because a room is either a way or a hoard, the same product asks whether this level was for getting deeper or for getting richer. Three ends and no exchange rate between them, met as a concrete question, three times a level, in front of a monster whose price you may not know.

> Which puts `desireVisibility` at the centre of the game rather than at the edge of one monster. A party that knows all three desires is shopping. A party that knows none is opening conversations at a round apiece to find out, and the light does not care that the information was worth having.

Retreat becomes a real option with a real price. It was previously the `move` branch of a turn and did nothing in particular. Now it is how you decline a price, it costs a round, and retreating from all three rooms of a level has spent three rounds and opened nothing at all.

10.3 And what it costs

One thing is lost and it should be said rather than quietly dropped.

The monsters of a room were a tensor, $m_1 \boxtimes m_2 \boxtimes \cdots \boxtimes m_k$, and the note used them as its clearest example of independence being enforced rather than promised, since every prompt is built before any is sent. With one guard to a way, k is one and the tensor degenerates.

where $\boxtimes$ still lives	the reckoning of Section 12.17, which computes three ends from one state and joins none of them. Two parties descending at once, in Section 2.16. And the ladder of Section 24.4, which is four policies run on one setting where none may see another.

So the operator keeps three homes and loses its most vivid one. That is a fair trade for a level whose rules fit in three lines, and the doubly guarded way is still expressible if it is ever wanted. It is $k = 2$, the operator is already written, and nothing else in the note would have to change.

10.4 You can only negotiate within dagger range

This is the rule that reorganises everything, and it is worth stating on its own line.

> **A monster will not talk to you from across a room. To negotiate you must be on its square, which is dagger's length, and dagger's length is the closest anything gets to anything.**

Diplomacy is therefore not the safe route. It is the route that requires you to close all the way, and closing takes turns during which the thing is also moving, and it ends with you standing on the square occupied by whatever it was you were trying to form an opinion about.

And it has one consequence nobody designed, which is the sort worth keeping. A long sword reaches one square and not zero, so a monster holding only a long sword cannot touch you once you are on its square. Closing to talk to it has disarmed it. The thing you are safest negotiating with is the thing best equipped to stop you arriving.

Which corrects the last thing in this note that was too easy. The two routes of Section 14.8 paid into two economies at comparable cost, and Section 12.7 gave combat the safe ground that negotiation cannot. This gives combat the second thing it was missing, which is *reach*. You may kill a thing from four squares away and you may only talk to it from one.

Read that against the bow. Section 14.8 says a corpse is mute and a monster shot at four squares is a monster nobody interviewed. That was a metaphor and it is now a measurement. The bow is the weapon that forecloses the information economy, the dagger is the weapon of the market, and the long sword is the one that commits to neither.

10.5 A deal is two turns, and the gap between them is the risk

You cannot execute a trade you have not agreed. So dealing has two steps and they are separate turns.

	the turn	what it establishes
negotiate	one turn, on its square	the terms. What you give, and what you get, which may be a secret, an item, or the way opened
trade	one turn, on its square	the goods change hands, or they do not

$$\text{deal} \;=\; \text{negotiate} \;\triangleleft^{*}\; (\text{trade} \;+\; \text{refused}) \qquad \text{trade} \;=\; \text{give} \;\triangleleft^{*}\; (\text{honoured} \;+\; \text{robbed})$$

Two sequential-with-a-choice composites in a row, which is the deepest such chain in the note and the only place one feeds another. The first asks whether there are terms. The second asks whether they meant them.

Which is better than the single bargain it replaces, because now the terms are known before you pay. Being robbed stops being a random misfortune and becomes a betrayal of something specific that was said out loud. You knew the price, you paid it, and it took the goods anyway.

10.6 You can attempt a negotiation and get stabbed

Every turn is one action and the monster answers each one. So a turn spent talking is a turn it may spend cutting.

your turn	its turn	and so
walk onto its square	whatever it likes	you arrive having given it a free move
negotiate	`attack` is one of its eight	you spend a turn on terms and it spends one on you
trade	`attack` again, or `offer`, or `flee` with your goods	you have paid and it has not

So a completed deal is three turns of standing at zero squares from something that need not have meant any of it. That is the price of the information economy, it is charged in the currency of Section 12.1, and it is paid in advance.

And the loadout decides how bad that is. Section 12.5 says a long sword reaches one square and not zero, so the thing with a long sword cannot touch you while you talk to it. The thing with a dagger can. So reading what a monster is holding is not tactical decoration, it is the question of whether this conversation is survivable, and it has to be asked before the walk rather than after it.

Which is the last thing the two routes needed. Killing costs light, arrows and sometimes people. Talking costs three turns at the range where you are most exposed, to something whose motive you inferred and whose word is not enforced. Neither is safe, neither is cheap, and they are expensive in different currencies.

10.7 The approach is the decision

Three or four turns of walking, one square at a time, towards something that is also moving.

it closes too	so the gap shuts at two squares a turn if you both advance, and you arrive sooner than you meant to
it can hold off	withdrawing is one of its eight actions, so a thing that does not want to talk can keep you at sword length all day and spend your light doing it
you arrive committed	holding a dagger, adjacent, having spent the turns. Refusal at that range is the worst position in the game

And that is where the hidden desires of Section 14 finally bite properly. You walk four squares to ask a thing what it wants, because you could not know from where you were standing. If the answer is that it wants nothing you have, you have spent four turns of light to arrive at knife range holding a knife.

10.8 The room's dimensions are a parameter

A rectangle has two numbers and both of them change what the room is about.

long and thin	the bow's room. Three or four turns of shooting before anything arrives, and no way to avoid the arriving
small and square	the dagger's room. Everything begins adjacent, so negotiation is immediate and so is everything else
wide	the room you can walk around something in, which is the only place declining a guard costs less than dealing with it

So the generator sets the character of an encounter by choosing two integers, and it does so before placing anything. That is the cheapest expressive knob in the whole design.

10.9 Guarding is now a fact about squares

The word has been doing metaphorical work and it can stop.

A monster guards a way by standing between the party's entrance and the square the way is on. So whether something is guarded is a question about a grid, and in a wide room there may be a path around it. Getting past is then a third option alongside killing and paying, it costs the turns the detour takes, and whether it exists at all is a property of the room's shape.

kill it	pays an item, leaves ground you can rest on
pay it	pays a secret, and requires you to be adjacent to do it
walk past	pays nothing, costs the detour, and leaves it behind you

The third row is new and it is the one that keeps a bad guard from being a wall. It also has a cost nothing else has, because a thing you walked past is still there, still between you and the way home, and Section 12.7 says you cannot rest while it is.

10.10 Which simplifies the round

Retreat was a separate branch of a sum and it does not need to be. It is walking.

$$\text{round} = \text{move} \times \text{strike} \times \text{chat} \times \text{negotiate} \times \text{trade} \times \text{draw} \times \text{use}$$

Seven branches, one action a turn, and every one of them spends the same round of light. Striking resolves by what is in hand and how far the thing is, which is the partition above. Negotiating is available only at one square. Using is the lamp oil or the healing potion of Section 12.7 and needs an empty room. Moving is one square and is how you approach, decline, detour and go home.

So the sum at the guard disappears and the product absorbs it, which is the second time in this note that adding geometry has removed an operator rather than added one. The first was Section 10.1 merging rooms with ways. A design gets simpler when the abstractions are replaced by the thing they were standing in for.

10.11 What it costs the tuning

`roomWidth`	squares across, drawn per room	3–10
`roomHeight`	squares deep	3–8
`daggerDamage`	at one square	1–3
`swordDamage`	at two squares	2–5
`bowDamage`	at three or more	1–4
`startDistance`	squares between the party and the guard on entering	1–6

Negotiations opened per run, against negotiations completed. The criterion, and it exists because this section made talking expensive. If the two numbers are equal then closing is free and the range rule did nothing. If completions are rare then diplomacy is a trap and players will stop attempting it, which loses the half of the game that finding out what things want is. The gap between opened and completed is the price of the approach, and it should be visible without being punishing.

11 One dagger, and liking is the variable

$$
\begin{aligned}
\text{round} &= \text{move} \times \text{talk} \times \text{stab} \times \text{trade} \times \text{use} \\
\text{liking} &= \text{liking} \lhd \text{talk} \\
\text{chat} &= \text{say} \lhd^{*} \left(\text{told} + \text{nothing} + \text{attacked}\right)
\end{aligned}
$$

The first line is the simplification. The second is the only accumulating quantity in the game. The third is the whole of what this section is about, and its three branches are the reason.

Three weapons and three bands of distance were a good tactical space and they were the wrong game. They put the interesting decisions in the geometry, and the game is finding out what things want. So the whole of it collapses to this.

There is one weapon and it is a dagger. It reaches the square you are standing on and nowhere else. Everything you can do to an inhabitant, you do from its square, and that includes talking to it.

$$\text{round} = \text{move} \times \text{talk} \times \text{stab} \times \text{trade} \times \text{use}$$

Five branches and no draw, because there is nothing to draw. Distance no longer decides what you can do. It decides only how many turns it takes to be able to do anything at all, which is the approach of Section 10 and is now the whole of what the grid is for.

What that deletes is worth listing, because a simplification that deletes nothing is a rearrangement. The bow and the long sword go. Arrows go, and `quiver` with them. The three range bands go, the weapon antichain goes, drawing goes, and reading a loadout for range goes. Eight or nine of the sixty-five knobs go with them.

11.1 Nothing is hostile by default

The second half of the simplification and it is the larger one.

Section 13 had guards, roamers and civilians, and the first of those was armed and hostile because something had to be. Nothing is now. Every inhabitant is mostly friendly, will generally let you talk to it, and is deciding as it goes whether that continues.

before	a thing was hostile or peaceable, and which it was you could read off it
now	a thing is hostile or peaceable *as a result of the conversation*, and the conversation is the only way to find out which it will be

So there is one hidden variable and everything turns on it.

liking	what it does	and
high	it tells you a secret, names a desire, or opens the way	the rungs of Section 18.9 are its upper reaches
middling	it talks, and neither helps nor hurts	most turns are here
low	it attacks	at dagger's length, which is where you are standing

And there is a floor probability of attack that never goes away. A thing that likes you well enough may still turn, because `tantrumRate` is not zero and the inhabitants of a flooded labyrinth are not obliged to be reasonable. Which means no amount of charm makes an approach safe, and the decision to walk onto somebody's square is always a decision under risk.

11.2 An interest raises it and an aversion collapses it

Every inhabitant has two subjects rather than one, and only the first was in the note before.

its interest	something it likes talking about. Say something interesting about it and liking rises. This is Section 18
its aversion	something it will not hear mentioned. Raise it and liking collapses, and you are on its square

You do not know either in advance. So a conversation is exploration with a real downside, and the turn you spend finding a subject that lands may be the turn you find one that does not.

Which gives the market of Section 14 a seriously valuable new commodity. An interest is worth buying because it makes chat work. An *aversion* is worth more, because it stops chat from getting you stabbed, and a party that has bought both about somebody can hold a conversation with confidence. Two rows of the table per inhabitant instead of one, and the second is the one you will wish you had asked about.

The aversions are also where the generator may be as free as it likes, for the reason Section 18.8 gives. An aversion is state. It never indexes anything and no decoder inspects it, so Claude may decide that this pump-minder cannot bear to hear about his brother, and nothing in the labyrinth notices except the pump-minder.

11.3 So the conversation is the game and not a subsystem

Every route through an inhabitant now goes through the same variable, which is what the earlier draft did not have.

a secret	given when liking is high enough, which is conversation
a desire named	the same, one rung lower
a way opened	the same, at the top, and Section 13.2 says some ways have no other route
a fight	what happens when liking falls, or when the floor probability fires
an item	taken from whatever you killed, which is the physical economy still working exactly as Section 2.8 says

So the two economies of Section 2.8 are unchanged and their connection is tighter. Light buys conversation. Conversation buys knowledge, or it buys a fight, and a fight buys goods. The conversion diagram is the same diagram and the arrow out of light now has a probability on it that the player is arguing with in English.

11.4 What it costs the tuning

Knobs deleted: `quiver`, `bowDamage`, `swordDamage`, `weaponWeight` as a table, `armedRate`, and the range bands. Knobs added, and there are fewer than went.

`likingStart`	where liking begins, before you have said anything	0.3–0.7
`interestGain`	what a good turn on the right subject adds	0.05–0.3
`aversionLoss`	what mentioning the wrong thing takes off	0.2–1.0
`attackBelow`	the level of liking under which it attacks	0–0.3
`tantrumRate`	floor probability of attack per turn, whatever the liking	0–0.1

`aversionLoss` at one is a single wrong word ending a conversation and possibly a party, which is the version that teaches players not to talk. Below half it is a setback and the conversation is recoverable, which is the version where the risk is real and the game is still about talking.

Conversations survived, against conversations that turned. The criterion. If nothing ever turns, the approach is free and Section 10 is doing nothing. If a third of them turn, players will stop approaching and play the physical economy only, which loses the half of the game that is the whole of it.

12 The economy

$$\begin{aligned} \text{reckoning} &= \text{haul} \boxtimes \text{opened} \boxtimes \text{bottom} \\ \text{muster} &= \text{one} \times \text{two} \times \cdots \times \text{r} \\ \text{outfit} &= \text{smith} \times \text{chandler} \end{aligned}$$

The tensor is the refusal to give a score, and the two products are the only decisions taken before anybody goes underground. Everything else in this section is what they are spending.

Every operator in the previous section says what kind of decision it is, and that makes the decomposition auditable in a way a prose design document is not.

$\boxtimes$	**no decision.** Both sides happen, neither can see the other, so not even the order matters.
$\times$	**a decision.** Several offered, one answers, and the ones you did not take did not happen.
$+$	a decision already taken, by where the party is standing.
$\triangleleft$	a consequence. What you did is the prompt for what happens next.
$\triangleleft^*$	a consequence you could not have predicted when you acted.

So count what the player decides, and count it per unit of time rather than per page.

how often	**the composite**	**decisions**
once a run	smith $\times$ fletcher	1
once a level	east $\times$ west $\times$ north	1 per room
once a level	mines $+$ caverns	1
once a round	strike $+$ trade $+$ move $+$ speak	0, this is a keyboard
once a round	swing $\times$ shoot $\times$ draw	1

The fourth row is worth reading twice. A sum is not a choice the player agonises over, it is the routing of Section 9.3, and the set of words that exist is fixed by where the party is standing. Typing the only word available is not strategy. It follows that all of the strategy in this game lives in two products and one sum, and this section is about what they are spending.

Because a product is only a decision if its branches compete for something scarce. With nothing scarce behind it a product collapses to whichever branch is better, and a branch that is always better is a lookup and not a choice. Five distances and two weapons is a fine table, but if shooting costs nothing then shoot at range and swing up close is the answer every time and the table has been solved rather than played.

The decomposition has products and no purse behind them. Everything below is the purse.

12.1 Two clocks

There are two scarce things and everything else in this section spends one or the other of them.

clock	**resets**	**what it prices**
light, in rounds	every descent	rooms entered, shots fired, conversations held, the climb out
descents, counted	never	how many trips the whole game gets

Nothing in the game spends both, and every mechanic named below attaches to exactly one of them. That is the test of whether this is the right pair, and it is worth stating as a rule because the alternative was a contradiction rather than a simplification.

Why it has to be two

A single clock cannot be made to work, and the reason is a decision taken in Section 23.7 for entirely different reasons.

If light is all there is and it never resets, then going back down for a piece you walked past is impossible, and Section 23.7 keeps the levels alive precisely so that it is possible. If light is all there is and it resets on re-entry, then depth costs nothing across a run, because a party climbs out and comes straight back down with a fresh torch. Neither is the game.

> So light is the tactical clock and the count of descents is the strategic one. Which is the shape of the accompanying game, where sowing and reaping are the turn and the five-year plan is the horizon, and it is what lets that game ask a question in March and answer it in August without either clock swallowing the other.

Light

A torch burns for a fixed count of rounds and the party carries it. Rounds spent fighting, rounds spent talking, rounds spent walking from one room to the next, and rounds spent climbing back out all burn the same torch. A party that runs out below ground is lost, which is what makes the number frightening rather than merely inconvenient.

Torches are found in the upper levels only, which is the table of Section 29 being useful for the second time. Relics are at the bottom, and the means of reaching the bottom is at the top. A torch may also be bought at the surface, and buying one is one of the things the purse is for.

> Which is what turns going down as far as you dare into an arithmetic rather than a mood. The party is always carrying a number, and that number is how many rounds it has left before it is in the dark. Half of it is the way home. So every round spent anywhere is a round subtracted from the depth reachable on this trip, and the player is doing that subtraction on every turn whether they want to or not.

That is what makes the rest of the section work, because once every branch of every product spends the same thing, the branches are genuinely opposed. A third room, an extra shot, one more level down, a conversation with a monster who might be lying. All of them cost rounds and there is one supply of rounds.

And the descent is a $\lhd$ that now carries something. Section 28 says what waits on level $d{+}1$ is computed from what you did on level d. What is actually computed is how much light you arrived with, which is a number the player spent the whole of level d choosing.

Descents

The count of descents never resets and nothing refills it. When it reaches zero the game is over, however rich anybody is and however much of the labyrinth stands open.

A descent is one trip from the surface and back, so it is spent by going down and it is spent whether the trip achieved anything or not. Climbing out early to save people costs a descent exactly as reaching level eight does.

Which is the clock that makes the third end of Section 12.17 an end at all. Without it nothing punishes patience, and a labyrinth with no last year is one a careful player opens completely at no risk. The number of descents is the difference between a dungeon and a deadline.

12.2 The rooms are a product

The trap door used to wait on all three rooms. A tensor of three rooms all of which must be cleared is a chore with an operator on it, because independence plus obligation leaves nothing to decide, not even the order.

$$\text{rooms}(d) \;=\; \text{east}(d) \;\times\; \text{west}(d) \;\times\; \text{north}(d)$$

So the trap door opens on one room and the other two are an offer. A room you walk past keeps what was in it and it is still there when you come back, because Section 23.7 keeps levels alive. Now every level asks the same question three times. Is what is in that room worth the rounds it will take, given that the rounds are also the way home.

The tensor has not been lost, it has been moved to where it was always true. The monsters in a room really do act at once and really cannot see each other. The rooms of a level were never like that, because the party has to walk to them one at a time carrying one torch.

12.3 Arrows

A quiver holds few. Every shot spends one, and a spent arrow is lying in the room one step from whatever it hit, which is sword range. So refilling the quiver means walking into the distance the bow existed to avoid, and the round spent walking is a round of light.

Which is the smallest possible fix to the strongest mechanic in the game. The distance table stops being a lookup and becomes a budget, and the two products in a round now argue with each other. Spending an arrow at five steps is buying safety with supply, and swinging at one step is buying supply with hit points.

12.4 Hands, which are people

Section 2.14 gave the party a number of hands and never spent it. Spend it, and note where the number comes from. A hand is a person from the roster of Section 2.15, so how many hands there are was decided at the muster and cannot be changed below ground.

Loot, keys, puzzle pieces, torches and arrows all want a hand. A hand holding a relic is not holding a torch. So the bottom of a descent is a product as sharp as any in the game, over what comes up and what stays where it was found, and Section 23.7 means what stays is still there next time.

Loot has a weight as well as a value, and the two do not agree. A relic worth forty takes two hands and a bag of silver worth twenty takes one, so the question is never simply what is worth most. It is what is worth most *per hand*, and that is a different ordering.

This is where the way back stops being a formality. Section 7.9 argues that the ascent is the half that matters and the mapping u is where the game lives. Hands are the rule that makes a player feel it. You stand at the bottom holding five things worth having, three hands, and a count of rounds that says you are not coming back down here this trip.

And because a hand is a person, the same number is doing strategic work as well. A party of four carries more out and loses more when the light fails, and every one of them lost is a descent somebody cannot make later. So hands couple the haul to the deadline, which is the second of the three conflicts of Section 12.17 and the one the design previously had no mechanism for.

12.5 Six hands

The party has six hands, being two for each of three people, and everything it holds takes at least one of them. Weapons are hands and lamp oil is a hand, and the heavier things take two. That is a small knapsack and not a list of slots, which matters because it means two light things and one heavy thing are a real comparison rather than a tie.

> Six is not a constant. It is `handsPer` multiplied by the muster of Section 2.15, so a party of three at two hands each has six and a party of two has four. Sending one more person buys one more hand, which is the half of the muster's cost that makes it a decision at all.

12.6 Being ready at every range costs two thirds of what you can carry

A dagger, a long sword and a bow are not equal burdens, and Section 10 says each works at exactly one band of distance. The bow needs two hands and the other two need one each, so a party equipped for anything has spent two thirds of its hands on the option of being equipped for anything.

carried	weighs	left	and the cost of it
dagger, sword, bow	four	two	ready anywhere, and two thirds of the haul foregone
bow alone	two	four	untouchable at distance, helpless the moment anything closes
dagger alone	one	five	the cheapest possible admission to the market

> Read the third row twice, because it is the consequence worth having. Negotiation happens at one square, and standing at one square with nothing but a bow is standing there defenceless. So the dagger is not a weapon in the way the others are. It is the licence to trade, and a party that left it behind has locked itself out of the information economy by an inventory decision made at the surface.

12.7 Rest, and why killing buys ground

Light and hit points run down and nothing so far puts either back. So there is lamp oil and there are healing potions, both are items with a weight like any other, and using either costs a round.

And neither can be used in front of something. You cannot drink while a thing is closing on you and you cannot refill a lamp with one hand on a sword. Rest requires an empty room.

> Which makes the empty room the only place in the labyrinth with no economy in it. Nothing there wants anything, nothing there knows anything, and that is exactly why you can stop. Rest is the absence of trade.

12.8 It gives combat something negotiation cannot give

This corrects a real imbalance and it is the reason the section is here rather than in a list of consumables.

Section 14.8 has the two routes through a monster paying into two economies, which looked even. It was not. A negotiated monster is *still standing there*. The room is still occupied, so it is still not somewhere you can drink.

route	pays	and may	leaves behind
kill it	an item	open a way	**ground you can rest on**
satisfy its desire	a secret	open a way	a live monster, and a room you cannot stop in

So a party that has talked its way down five levels has an income of secrets, a full purse, and nowhere behind it to sit down. A party that fought its way down has a corridor of safe rooms and nothing to trade with. Neither is the right answer and that is the point.

Which also settles something about Section 15. A monster you killed has stopped trading, so it stops resolving edges of the desire graph and the market near it freezes. A monster you satisfied goes on dealing with everything else while you climb. So violence buys safety and stillness, and diplomacy buys knowledge and a market that keeps moving without you.

12.9 And rest is priced in walking

The round the healing potion costs is the smaller half of the price. The larger half is getting somewhere you can drink it.

A party wounded on the third room of a level has to go back through rooms it has already crossed, at a round apiece, drink, and come forward again. So recovery costs the round plus twice the distance, and a party that pushed deep into a level with nothing cleared behind it has put itself somewhere it cannot recover from at all.

Which is the retreat branch of Section 10.1 acquiring a second use. It was how you decline a price. It is now also how you reach safety, and the two uses compete for the same round, so a party retreating from a guard it cannot afford is not resting and a party retreating to rest is not shopping.

12.10 Two more rows in the antichain

Both slot into the table of Section 12.12 without disturbing it, and both are non-dominated for the ordinary reason that they do a thing nothing else does.

	rare	heavy	and it is for
lamp oil	no	one hand	the tactical clock, bought back a few rounds at a time
a healing potion	yes	one hand	the pool, and it is the only thing that undoes a bad round

Note what they do to the hands product. Every measure of oil carried down is a hand not carrying a relic back up, so the party's capacity to survive and its capacity to profit are the same capacity, and the muster of Section 2.15 is now choosing between them before the descent starts.

12.11 What it costs the tuning

Three knobs and a criterion.

`restRounds`	rounds one use of either costs	1–3
`oilRounds`	rounds of light one measure of oil returns	4–12
`potionHeal`	hit points a healing potion returns	2–8
`emptyRoomRate`	fraction of rooms generated with no guard at all	0–0.3

The last one is the interesting one and it is not a convenience. At zero the only safe ground in the labyrinth is ground you made by killing something, so the game is violent by construction. At a third, safety is furniture and combat loses the product this section just gave it.

Distance to safety. The criterion is how far a party is from a room it could rest in, over the course of a run, and what the best runs did about it. A game where that distance is always one is a game where the empty room did nothing. A game where it is always four is a game about logistics rather than about anything else. Somewhere in between a player starts clearing rooms they did not need to clear, because they might need to come back through them, and that is the behaviour this mechanic exists to produce.

12.12 No item may dominate another

The generator places items and monsters in rooms and it is easy to say that it does so at random. It does not. Placement is a distribution and the distribution is a parameter, so somebody is choosing it and the choice can be wrong in a way that is worth naming.

Three numbers describe an item and they are usually treated as independent.

rarity	how often the generator places one
weight	how many hands it takes out of the party's few
demand	how many monsters want it, which is not chosen directly. It falls out of the colour rule of Section 20 and the colours the labyrinth happens to hold

The third row is the one that changes the character of the other two. An item's worth is not its price in gold. It is how much of the labyrinth will open for it, and that is set by a rule the player is still guessing at.

So the bottom of a descent is a knapsack problem whose values are uncertain. Hands bound what comes up, weight prices each candidate, demand values it, and demand is known only as well as the colour rule is known. Which is a far better decision than a knapsack with numbers on it, because being wrong about the rule and being wrong about the arithmetic are different mistakes and the player can be making either.

12.13 Rarity is bounded by demand

Some things are very rare. One to a labyrinth, or none, and that is a good thing to have because a find that matters needs finds that do not.

But rarity is not free and the constraint on it is not aesthetic. Section 24.14 says every promise is discharged at least twice, so that no single loss can orphan a chain. An item that exists once

cannot discharge anything twice.

> **The rarer an item, the fewer desires may point at it. An item that some necessary way depends on must exist at least twice, and an item that exists once may be wanted by nobody who stands between the party and the surface.**

So the generator places the rare things where their absence cannot end a game. A relic that exists once is treasure, which is to say it is haul and nothing else wants it. A stone that half the labyrinth desires is common by necessity.

Which is the same discipline as the item antichain of Section 12.12 arriving from the other direction. There, no item may be beaten on all three axes at once. Here, an item's rarity and the demand pointing at it are not independently choosable, because between them they decide whether the labyrinth can be finished. Rarity is a property of the loot table and solvability is a property of the whole web, and the second constrains the first.

12.14 And the three have to be in tension

Here is the condition, and it is the reason this is a subsection rather than a line in the parameter table.

If some item is commoner, lighter *and* more wanted than another, then the second item is never worth picking up. Not rarely worth it. Never. It is dead content that the generator will go on placing and no player will ever carry, and the failure is invisible because nothing errors.

> **Every item must be non-dominated in rarity, weight and demand.** No item may be beaten by another on all three at once.

Which is the same condition as everywhere else in this note and it is worth seeing that it is the same. Section 12.17 holds three ends apart and refuses an exchange rate. Section 24.5 wants the strategies to be an antichain rather than a ranking. This is that idea applied one level down, to the furniture, and it says the item table must be an antichain too.

It is also the bindingness test of Section 24.6 pointed at the loot. A mechanic no winning line ever uses is a comment, and a dominated item is a comment with a weight and a price. The difference is that this one is checkable before anybody plays, because it is a property of a table with about six rows in it.

12.15 Which is why the table has a shape

A proper antichain over six items means each is best at something, and stating what each is best at is more useful than stating its numbers.

	rare	heavy	and it is for
a relic	very	one to three hands	the haul, and nothing else wants it. There are ten kinds of it and they do not weigh the same
a stone	no	one hand	paying with, in bulk
a key	yes	one hand	one way, and then it is scrap
a torch	no	one hand	the clock, and it is the default correct answer
a piece	yes	one hand	the second end, and it cannot be sold
an oddment	no	one hand	whatever the rule says wants it this game

The last row is the one the colour rule moves. An oddment is worth nothing on its own terms and its demand is redrawn every game, so it is the item whose value the player must induce rather than read. In a game where the rule makes oddments universally wanted they are the best thing in the labyrinth, and in the next game they are ballast.

> And that is the cheapest possible way to make the item table itself part of the puzzle. Five rows whose worth is stable and one whose worth is a hypothesis. Nothing had to be added to the generator, because the rule was already being drawn and the colours were already being assigned.

12.16 The way down is still a coin

One thing in this section is left unrepaired and it needs a whole section of its own.

The way down is a sum, and the trap door goes to the mines and the shaft goes to the flooded caverns, and nothing anywhere tells you which is which. Choosing is therefore not a decision, it is a flip wearing the costume of one, and no amount of scarcity fixes that. A choice made in ignorance is not made better by being expensive.

> What fixes it is that somebody in the room knows. Section 14 is that repair, and it turned out to be larger than the sum it was meant to fix, because a monster who can be paid for an answer is a monster who can be paid for anything.

12.17 Three ends, and no exchange rate between them

Two clocks are not yet enough, and the reason is that everything above still assumes the party wants one thing. Come home rich is a single objective, and a single objective has an optimal policy, so a player who has found it is no longer deciding anything. The arithmetic of light would be solved once and then performed.

So the game holds three ends and refuses to rank them.

end	what it is	paid for in
the haul	Gold and relics carried up and sold. The only end that is spendable, and the only one a player can see growing during a descent.	depth, and hands
the labyrinth opened	Locks opened from below, puzzles solved, wings entered. Permanent, compounding, and the only end that survives a party's death.	hands, light, and pieces that cannot be sold
the bottom, in time	Standing on level eight before the descents run out. A threshold and not a quantity, so it is either met or it is not.	descents, and therefore people

Two of the three conflicts are already built and were being read as inventory management.

A puzzle piece cannot be sold and it occupies a hand, so every piece carried up is a unit of haul declined. That is the first end against the second, decided one hand at a time, at the bottom of a descent, by a player who now has to say what this trip was for.

Rounds spent gathering pieces and arguing with a gate-keeper are rounds not spent going down, and the count of descents does not pause while a puzzle is solved. That is the second end against the third.

> The third end is the one the design had no clock for, and Section 12.1 is now that clock. The bottom must be reached within a fixed count of descents, after which the game is over however rich anybody is. Nothing here previously punished patience, and an end nothing punishes is not an end.

And the items already divide along the three ends, which is worth tabulating because it means no new kind of thing is needed anywhere.

item	the end it serves	why
gold, relics	the haul	the only things that can be sold
puzzle pieces	the opened	unsellable by construction, and each costs a hand
keys	both	a key opens a way, and the way holds haul
torches	the means	buys light, and light buys everything else
arrows	the means	buys distance

Nothing is missing from that table. What was missing was the statement that it maps onto the ends, and the consequence is that the game needs no more kinds of monster, weapon or item than it already has. The complaint that began this section was decision density and not variety, and variety would have bought a longer game at the same density while widening the two tables that Section 23 needs narrow.

12.18 The trap

Three ends with three separate costs would be a menu, and a player would pick one and stop thinking. What makes it a position is that the costs feed into each other with a delay.

A party lost is exploration absent later. So driving parties hard to open the labyrinth faster spends the very thing the last descent needs, and pushed far enough you run out of descents holding the most thoroughly unlocked labyrinth that nobody ever reached the bottom of.

Which is the same trap as in the accompanying game. There a peasant starved in 1932 is a soldier missing in 1941, so extracting harder to build steel shrinks the army the steel was meant to protect. It is the coupling and not the plurality that does the work. Three ends whose costs arrive on different schedules cannot be reasoned about once and then executed.

12.19 And the reckoning says nothing

At the end of the game the three numbers are printed and the game does not add them up.

There is no score. There is a haul, a count of what was opened, and whether the bottom was reached in time. A player who wants to know which line was best has to decide what best means, which is the whole of the design intent, and it is what the accompanying game already does when its two orderings of the dead disagree and it holds both numbers without saying which is real.

This has a consequence for Section 24 that is easy to miss. A tuner cannot be handed a scalar either. If the game refuses to weight its three ends then the thing being maximised is the *width* of the set of defensible ways to play, and no weighted sum can express that. Which is the argument of Section 24.5.

12.20 The two economies are not two games

One more thing about the trap, because Section 14 adds a version of it that is tighter than the one above.

Killing pays in goods and cuts an edge out of the desire graph. Trading pays in secrets and leaves the monster standing. So the party that funds itself best is the party that has destroyed the most of the market it was funding itself to enter, and the party that has preserved the market cannot afford to enter it.

Which is the same shape as everything else in this section. A peasant starved in 1932 is a soldier missing in 1941, a person spent on a relic is a descent not made in the last year, and a monster killed for its purse is a price you will never be quoted. Every cost in this game is also a capacity, and that is the only reason any of it needs thinking about.

12.21 Intermittent reward

One more thing is scarce and it has not been named, which is certainty about what is coming.

The kinds of loot at each depth are fixed by the table of Section 29 and stay fixed, because that table is what makes a generated level checkable. What is not fixed is which item arrives inside that bound, and the variance of the draw rises with depth. So the deeper level is more dangerous and less predictable at once, and those are the same knob.

That schedule has a name. A reward arriving after an unpredictable number of attempts is a variable ratio schedule, and it is the most persistent of the schedules there are. A fully predicted

reward is worth nothing to anybody, which is exactly what a depth-indexed table of certainties would have delivered.

The gate-keeper is already one of these and nothing had to be added. Its verdict cannot be predicted because nothing in the program knows it, so the schedule here is not simulated. It is the actual state of knowledge, which is the strongest form this can take and it came free with Section 21.

A refusal should say what was missing. A gate-keeper that declines and names the piece it wanted is a near miss which teaches, and it sends the party back down after a named thing rather than after a feeling. Around three in ten is where such a rate is usually put.

And there is nothing in this game that a player may simply do again. Every uncertain outcome sits behind a choice that cost rounds, and the rounds are finite, so no amount of wanting to know can turn any of this into a lever. That is not restraint on the designer's part. It is what having exactly one clock structurally prevents.

12.22 What they all have in common

Not one of them is a new operator, and not one of them is a new kind of thing crossing a boundary.

Light, arrows, hands, the roster and the count of descents are five more fields in the party record of Section 28, which was already going down inside the prompt and coming back up inside the response. The rooms are one character changed in one equation. A monster's fact and a monster's interest are two sentences in a prompt a level generator was writing anyway, answered through two actions the monster already had. The three ends are three numbers printed at the end and never added together.

So the whole of the strategy is a handful of numbers in a record and one operator changed. That is the claim the rest of this note has been making about everything else, and it turns out to hold for the game as well as for the architecture. What travels in the prompt is what the game is about.

12.23 And the reward

Strategy and pleasure are not the same complaint, and the second one is about how often the player learns whether they were right.

Four things follow from what is already above. A descent now ends in three numbers rather than one, and they do not agree, so the player has to decide what the trip was for before they can decide whether it went well. A descent ends holding less than was found, because of hands. A loss now costs people and not a labyrinth, so the next descent starts from a world that stands further open and a roster that is shorter. And a gate-keeper's verdict is the one $\lhd^{*}$ in the game, which is to say the only moment whose outcome could not be computed in advance by anybody, including the program.

> Which is why the trace of Section 30 should be read as a scoreboard and not as a debugging aid. The descending column is a party spending light and the ascending column is a haul getting larger. That is the only display this game has, and the two columns are exactly the two things the player is trading against each other.

The cost is a wait, and it is the one real tension in the design. Section 23 spawns a Claude instance to write the level while the party stands on the trap door, and a loop with a wait in it is the opposite of a loop worth repeating.

The way out is a rule Section 23.6 has already written. Looking at a locked way is what puts its keys beneath you, because the obligation joins the party record the moment the party sees it. So let looking be the trigger for generating as well. Both ways down are seen at the start of a level and both are written while the party fights its way across the first room. The price is one level written and never entered per branch, which is the cheapest thing in the system, and the wait disappears into rounds the player was spending anyway.

13 The inhabitants

$$\text{talk} \;=\; \text{chat} \;+\; \text{ask} \;+\; \text{offer}$$

One line, and this section is about who is standing there to be talked to. The sum is a sum because the three branches want different things back, and which of them is even available depends on what is in front of you.

A monster carries between none and three weapons, and which ones decides what it can do to you and where.

a bow and no dagger	dangerous while you cross the room and harmless when you arrive. So you close on it, and it is the one thing in the labyrinth you can safely negotiate with
a dagger and no bow	harmless while you cross and lethal when you arrive. So you shoot it from four squares, and then it is dead and mute
all three	no safe distance exists. Pay it or leave it
none at all	it can do nothing to you whatever. Walk up, talk, and pay only in the rounds it takes

Which makes the approach of Section 10 a decision with something to read. What is it holding, and does that leave a distance at which I am safe and it is talkative. And the cruelty of the arrangement is in the second row, because the monster you can most safely deal with is the one you can only deal with by killing.

Monsters also carry items, and killing yields them. A roamer that has been trading all game is carrying four things by the sixth level, so the richest corpse in the labyrinth is the one that has been most useful to everybody.

13.1 Types have desires and individuals have weapons

There is a finite number of kinds of monster. Two of a kind want the same thing and need not be armed alike.

	fixed by	**so it is**
what it wants	its kind	learnable, and the same everywhere
what it knows	the individual	bought one at a time
what it carries	the individual	read off it, and taken from it
its motive	the individual	inferred from what it lies about

The first row is what makes Section 20 possible at all, and it is worth being explicit about why. A rule can only be induced from cases if the cases are instances of something. Were desires a property of individuals there would be no generalisation to find, no matter how many monsters a player interviewed, and the whole inductive layer would collapse into a list. Finitely many kinds is the premise the induction rests on.

And the split is the right one for play as well as for logic. The kind tells you the price before you have met the thing, which rewards the player who has worked out the rule. The loadout tells you the danger only when you can see it, which keeps every encounter a fresh tactical question.

So knowledge generalises and threat does not.

13.2 Some ways only a living thing can open

This is the rule that stops the game from being finishable with a bow, and without it nothing guarantees the second economy matters.

A guard can be killed and what it stood on becomes yours. That has been true throughout and it makes violence a complete strategy, which is a defect. So some ways are not merely guarded. They are *operated*. The thing standing there is the only thing that can open them, it will do so if satisfied, and its corpse cannot.

the way	killing the guard	satisfying it
guarded	opens it	opens it
operated	**closes it forever**	opens it

So an operated way is a place where the bow is not an argument and the sword is a mistake you cannot take back. A party that shoots the operator of the way to the eighth level has lost the bottom of the labyrinth for the rest of the game, and nothing announced that it was about to.

Two consequences follow and both are wanted.

The first is that a player must find out whether a way is operated before dealing with its guard, which is one more thing worth buying and it is a secret about the world rather than about the market. It joins the five kinds of Section 14 as a sixth, being *whether that one can be replaced*.

The second is that Section 24.14 acquires an obligation. Redundant discharge says every promised secret is planted twice. The same must hold here, so no level may have every route to its bottom operated by a single creature, or one bad round ends the game by accident rather than by anybody's decision.

13.3 Two populations, and only one of them stands still

The settling above is stated as a background process and that is too abstract to play against. So it is embodied. Some monsters are bound to a thing and some walk about.

a guard	bound to a way or a hoard, and armed. It is where it is, it will be there when you come back, and it is the obstacle
a roamer	bound to nothing. It walks, it meets others, it trades with them, and it is the market rather than an obstacle
a civilian	unarmed, peaceable, and living there. It guards nothing and threatens nothing, and it is where the testimony of Section 19 comes from. Section 18 is about what you can do with one

Which makes the resolution of the previous subsection something a player can watch instead of something the rules assert. Edges resolve because a roamer carrying a secret walked into a room holding a monster that wanted it. Secrets spread because roamers are couriers. A chain goes dead because a roamer was eaten between your visits.

And it makes the room product of Section 10.1 genuinely varied, because not every way has to be guarded at all. Four kinds of room, which is enough kinds and not too many.

an open way	costs a round and nothing else. Somebody has to give the party a free move or the arithmetic never breathes
a guarded way	priced, and the price is what Section 14 is about
a guarded hoard	optional profit, and the first end against the third
an empty room	no economy in it, which is why you can rest there

13.4 Watching is honest and asking is not

This is the best thing roamers do and it was not why they were added.

A monster can lie about what it wants. Section 19.2 gives the trading motive exactly that vice, and it is why the inductive sample of Section 20 is corrupted. But a monster cannot lie about what it has just *taken*. If you are standing in the room while a red thing accepts a green stone, that is a case, and no motive touches it.

	costs	**and the data are**
asking what it wants	one round	fast, and corrupted by whoever had a reason to inflate
watching what it takes	several rounds, and patience	slow, and clean

So the colour rule has two sources and they trade speed against reliability. Interrogate the labyrinth and you have a hypothesis by level three built partly on bluffs. Follow two roamers about and you have three honest cases and less light than anybody else. That is a real strategic axis and it exists because some of the monsters move.

13.5 The first thing that happens to you

A roamer can walk into the room you are standing in.

Nothing else in this note does that. Every other event in the game was caused by the party choosing to deal with something it walked up to, which is the objection Section 15 opens with. A roamer arriving mid-negotiation is the labyrinth acting on its own initiative, and it is the only thing here that does.

And it need not be a disaster to be tense. The roamer may be the thing you needed, arriving at the wrong moment while you have three rounds of light and a guard in front of you. An opportunity you cannot afford is worse than a threat, because a threat only costs you what it takes and this costs you what you now know you missed.

Roamers also accumulate. A roamer trades all game whether you are watching or not, so one you met on level two with a stone is carrying four things by the time you meet it on level six. Which makes a well-travelled roamer the richest target in the labyrinth, and killing it the one act that is both robbery and censorship, since what it was carrying included what it knew.

13.6 This breaks the tower, and here is the repair

Worth stating plainly because it is the only place in this note where an addition does not fit the architecture for free.

A roamer moving from level four to level five means state passing sideways between two levels, and every other thing in this design goes strictly down in a prompt and back up in a response. There is no lateral channel and there should not be one, because the whole claim of Section 7 is that a level knows only what its prompt told it.

So the roamers do not belong to the levels. They belong to the client.

the client keeps	where every roamer is, what it is carrying, and what it knows
the prompt carries	which roamers are present in the room being entered
the level answers	what happened in that room, knowing nothing about anywhere else

Section 27 already has the client accumulating everything that came back up, and says so as a virtue. The roaming population is more of that. The client advances every roamer by however many descents have elapsed, and then tells each level who is standing in it, so the tower stays vertical and no level ever addresses another.

> Which means the roamers move when you look, exactly as a level settles when it is prompted. That is the same trick twice and it is not a coincidence. Nothing in this architecture runs when nobody is asking, so everything that appears to happen in the background is in fact computed on the way in.

13.7 What the roamers cost the tuning

`roamerFraction`	of the monsters generated, how many are unbound	0.1–0.5
`roamerSpeed`	rooms a roamer crosses per elapsed descent	1–4
`openWayRate`	ways generated with no guard at all	0.1–0.4
`encounterRate`	chance a roamer enters the room the party is in	0–0.3

Where the rule was learned. The criterion, and it is the one that says whether this mechanism did anything. Of the runs that identified the colour rule, what fraction did it by watching rather than by asking. All asking and the roamers are scenery. All watching and interrogation is dominated, which would mean the market has no informational role at all. Both routes in use, at different costs, by players playing for different ends, is the target.

13.8 What it costs the tuning

`weaponWeight`	what each of the three weighs, jointly constrained with range and strength	a table
`monsterKinds`	how many kinds exist, which sets how learnable the rule is	4–16
`armedRate`	distribution over zero to three weapons per monster	a simplex
`monsterItems`	items a monster starts with, before it trades	0–3
`operatedRate`	ways that only a living guard can open	0.1–0.4

Weight spent on weapons. The criterion, and it reads the first subsection back. Across the best runs, how many of the six hands were metal. All three weapons every time means range is over-rewarded and the haul is not competing. Never more than one means the geometry of Section 10 is not biting, because a party that only ever carries a bow is a party that never needed to be anywhere.

And `operatedRate` is the one that decides whether this is a game about talking. At zero everything can be shot and the entire second economy is optional, which is the state the note was in until this section. Too high and a single misjudged round ends a run, which teaches caution rather than reasoning. Somewhere near a quarter, and the number wants finding by play rather than by argument.

14 A monster is a market

$$\begin{aligned} \text{offer} &= \text{give} \triangleleft^{*} \left(\text{honoured} + \text{robbed}\right) \\ \text{desire} &\in \text{goods} + \text{secrets} \\ \text{secret} &\in \text{about the world} + \text{about the market} \end{aligned}$$

The first line is a trade and the second and third are why it is interesting. A desire may be for a secret and a secret may be about a desire, and that mutual reference is this section.

Section 12.17 holds three ends apart and gives no exchange rate between them, and that was written as a virtue. It leaves a hole. A player holding a puzzle piece and wanting gold has no mechanism at all, and neither has one holding gold and wanting to know what is down the shaft.

Monsters are that mechanism, and two fields in a prompt are the whole of it.

a secret	one true thing it knows and will not volunteer
a desire	one thing it wants, which may be a good, and may be another monster's secret

Satisfy the desire and you are given the secret. That is the entire trade and it is the same trade for every monster in the labyrinth.

Neither field is visible. You do not know what it knows, which is what makes the secret worth buying, and *you do not know what it wants*, which is what makes the game hard. So before you

can pay a monster you have to find out its price, and the cheapest way to find out is usually to buy that from somebody else.

Which is the third turn of the screw and it finishes the joke. Section 12.17 publishes no exchange rate between the ends. This section has every monster quoting one privately. And now the quote itself is contraband, so the most valuable commodity in the labyrinth is the price list.

Which is worth the whole design given what the accompanying game is about. That one is a command economy setting prices by decree, and this one is the black market underneath it. Nothing in the labyrinth has a posted price and everything in it is for sale.

14.1 Two economies, and what separates them

Section 12.1 named two scarce things and never said what distinguished them. Here is what does.

economy	currency	bounded by	and it can
physical	goods, in the hands	**hands**	never lie to you
information	secrets, in the record	**light**	be false, and go stale

A secret weighs nothing, so the information economy is the one hands do not bound. What bounds it is rounds, because asking costs a round and a round is light. So the two clocks of Section 12.1 are not two versions of one idea. Hands price goods and light prices knowledge, and neither is redundant.

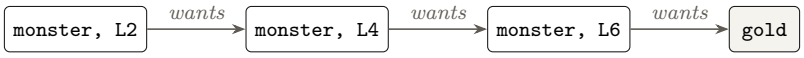
Two things follow that have no counterpart on the physical side.

A secret may be false. Goods are what they are. A monster's answer is whatever the monster decided to say, so information has a quality as well as a quantity, and two monsters agreeing is worth more than one asserting. Corroboration is purchasable and it costs another round.

A secret has a moment. What lies on level six is worthless at the surface and priceless on level five. Goods are worth the same whenever you hold them, which is why the physical economy is about carrying and the information economy is about timing.

14.2 The desire graph

A desire that wants another monster's secret makes the monsters of a labyrinth into a directed graph. Draw an edge from A to B when A wants what B knows. Then a trade is a walk on that graph, backwards.



Read it right to left, because that is the order it can be paid in. The one on level six wants gold, which the party has, so it is where the chain can be entered. Its secret buys the one on level four, whose secret buys the one on level two, whose secret is the one you came for. Nothing in that chain can be paid out of order.

Which is why the graph is the mechanism the note has been asserting without having. Section 1.1 says the labyrinth must be unlocked from below and Section 7.9 says the way back is the half that matters. A desire chain running from level two down to level six is that claim made into a rule, because the secret you need at the top is held at the bottom and the only way to spend it is to carry it up.

Two graphs, and only one of them can be walked

The edges above are payment edges, and they are not the ones the player can see. There is a second graph over the same monsters, and its edges run the other way round.

graph	an edge $A \to B$ means	and the player
payment	A desires what B knows	must walk it backwards to trade
knowledge	A knows what B desires	must walk it first, to see the other at all

So the payment graph is the terrain and the knowledge graph is the map, and the map is scattered across the terrain. A player standing in front of a monster with no idea what it wants has three ways to find out, and their prices differ.

ask it	one round, and it may lie about its own price, which is the cruellest lie in the game because you pay the wrong thing and lose that too
buy it	a market secret from somebody else, as reliable as whoever sold it
guess it	from what the thing is. A monster on a trap door probably wants a toll. Free, often right, and the reason the game is not unplayable at the start

The third row is a parameter and not a courtesy. `desireVisibility` is the fraction of desires legible from the monster's kind alone. At one the market is transparent and the game is a shopping problem. At zero it is exhaustive search and the light always fails first. Low but not zero is the setting, and finding it is Section 24's job.

Three conditions the generator has to meet

The graph is generated, so it can be generated wrong, and wrong here has a precise meaning.

It must be acyclic. If A wants B's secret and B wants A's then neither can ever be paid, and both secrets are unreachable. A cycle is not a hard puzzle, it is an unsolvable one.

Every component must have a source in the goods. A chain has to be enterable, which means at least one monster in each connected part of it must want something the party can simply carry. A component whose every desire is for another secret in that component is a cycle by counting.

It must be solvable within the light. Acyclic and entered is not enough if finding out who wants what costs more rounds than the party can carry. So the two graphs together must admit some order of asking, buying and paying whose total cost in rounds falls inside the budget a party could plausibly bring. That is a shortest-path question over a finite graph and a program can answer it.

And all three conditions are checkable before anybody plays, which is the argument of Section 23 arriving somewhere new. That section says the acceptance test for a generated level is that it compiles. This is a level that compiles and is still nonsense, so the test grows three clauses, and all three are graph properties a program settles in a moment. The first two are well-foundedness, which is the same condition that makes the recursion of Section 9.6 terminate, one storey up. The third is the interesting one, because a generator that has to prove its labyrinth solvable within a round budget is a generator being held to the standard of a puzzle setter rather than a compiler.

And it prices information

A secret's price is its distance from the goods. A monster whose desire is a coin sells cheaply. A monster at the head of a chain of four sells for everything below it, which is four rounds of asking, four payments, and a trip to the depth the chain bottoms out at.

So information has denominations and nobody had to assign them. The value of a secret is where it sits in the graph, and the graph is a thing the level specification already contains.

14.3 A secret spent is a secret spoiled

There is a hole in everything above and it has to be closed before the second economy is an economy at all.

Goods are conserved. Hand over a relic and you no longer have it. Secrets are not. Tell a monster what is down the shaft and you still know what is down the shaft, so you can tell the next one, and the one after that, at no cost whatever. An economy whose currency can be spent twice is not an economy, it is a printing press.

So information leaks. Once a secret has been traded it is abroad in the labyrinth, the monsters have it, and nobody will pay for it again.

	goods	**secrets**
copyable	no	yes
conserved by a trade	yes	no
keeps its value	yes	**no, the first telling spends it**

Which is exactly inverse to the physical side, and the inversion is the point. Goods are scarce in supply and stable in value. Secrets are unlimited in supply and their value is destroyed by use. The whole worth of a secret is the monopoly on it.

And that turns a coincidence into a decision. Two monsters may want the same secret, which is a thing Section 14.2 makes likely rather than rare, and you can only be paid for it once. So which of them do you sell to. That is a product with no clock attached and no obvious answer, and it is the sharpest choice in the information economy.

It also makes a lie durable in a way a bad purchase is not. Spend a false secret and it is spent. The monsters have heard it, nobody will buy it again, and you have destroyed an asset you never owned.

14.4 Lying, and why depth is dangerous

A monster that has been satisfied answers, and whether the answer is true depends on what the truth would cost it, which Section 19.2 sets out. Along a chain that compounds.

Spend a false secret on the next monster up and it is not satisfied. You have lost the goods, the rounds, and the chain, and you do not find out at the moment you were lied to. You find out one trade later, which is the worst place to find out anything.

So a chain of four at a truth rate of four fifths pays off about two times in five, and the deeper secret is both the more valuable and the more likely to be poisoned. That is a difficulty curve nobody imposed. It falls out of a graph and a probability, and it is the reason `chainDepth` is a more important knob than anything about swords.

Corroboration is the answer and it costs light. Two monsters on the same level may hold the same secret, and asking both is two rounds against one, so caution is expensive in exactly the currency caution is trying to preserve.

14.5 A gate-keeper is a monster with a plural desire

The gate-keeper of Section 21.2 was described separately and does not need to be. It is this mechanism with two fields changed.

	desire	**secret**
a monster	one thing	one true thing it knows
a gate-keeper	several, declared up front	a way through the labyrinth

That is the whole difference. A gate-keeper wants three clues rather than one thing, and what it gives back is a wing instead of a sentence. Everything else is shared: the desire is data, the verdict is the model, nothing in the program knows whether the payment sufficed, and the composite is the same sum.

Which makes the note smaller rather than larger, and that is the direction to be pleased about. The puzzle-piece machinery of Section 23.6 and the desire graph are one mechanism at two granularities. An obligation descending in the prompt so that a piece can be planted below is exactly an edge of the graph being created, and the rule that a puzzle can only be satisfied by pieces placed after the party saw it is exactly the rule that the graph is built downwards.

14.6 The desire is data and the verdict is the model

What a monster wants is one of the goods that exist or one of the secrets that exist, together with a quantity. Nothing more. Both are drawn from finite tables the compiler holds, so a desire is something a level specification can state and a generator cannot invent.

Whether it has been satisfied is the model's business, and nothing in the program knows it in advance. So the composite is a sum whose branch the monster picks, which makes it the second $\triangleleft^*$ in the note and the first one a player meets every room.

$$\text{bargain} \;=\; \text{offer} \;\triangleleft^*\; \big(\text{secret} \;+\; \text{robbed}\big)$$

Read the second branch. You may hand the goods over and get nothing, because nothing enforces a bargain in a labyrinth. That is why this is a sum with two sides rather than a purchase, and it is what keeps fighting worth doing at an affordable price.

14.7 What the secrets can be about

An information economy needs its own finite table or a generator will invent knowledge and nothing will be checkable. Section 23 makes that argument about loot and it applies here unchanged.

	kind	what it tells you
what is down	the world	which of the two ways down holds what, one level ahead
where a piece is	the world	which level below holds a clue for an open obligation
what is behind	the world	whether a locked way is worth the key it needs
can it be replaced	the world	whether that guard operates the way or merely blocks it (Section 13.2)
what it likes	the market	what another monster enjoys talking about (Section 18)
what it wants	the market	another monster's desire, including a gate-keeper's
what it fears	the market	the thing another monster will not stand against

Seven kinds, indexed by depth and by who is asked, and they divide in two. A secret about the world has terminal value, because it tells you where something is. A secret about the market has instrumental value, because it tells you who to trade with, and instrumental value compounds.

That division is the whole reason the game is playable. Without market secrets a player discovers desires by asking every monster in turn at a round apiece, and the graph is explored by exhaustion until the light fails. With them, one purchase turns exhaustive search into a plan. So the market secrets are worth more than the world secrets in almost every position, and a player who works that out has understood the game.

A generator may decide which monster holds which secret and may not decide that a sixth kind exists. The freedom is on the way in and the table is on the way out, which is Section 21 again.

14.8 Two routes through a monster, and each pays in its own economy

Every monster can be got past two ways, and both of them pay. What differs is which economy they pay into.

route	costs	pays	and may
kill it	light, arrows, sometimes people	an item	open a way
satisfy its desire	a round, then goods or a secret	a secret	open a way

That symmetry corrects an error. An earlier draft of this section had combat paying nothing and existing only as the expensive route when the price could not be met, which made fighting a tax. A tax is not a decision. Fighting pays out in the physical economy and satisfying pays out in the information economy, and the question in front of a monster is not which is cheaper. It is which of the two things you are currently short of.

And that closes a loop the note needed and did not have. Section 14.2 requires every desire chain to have a source in the goods, and goods have to come from somewhere. They come from killing things. So combat funds the information economy, and the information economy tells you which things are worth killing.

kill → goods → satisfy → secrets → where the goods are

Neither half can be played alone. A party that only fights has income and no idea where to spend it, and a party that only trades runs out of anything to trade with. Which is what makes the Fence of Section 24.9 both viable and fragile, and it is now fragile for a stated reason rather than by assertion.

A corpse is still mute, and that has not changed. Killing a monster pays in goods and cuts an edge out of the graph, so the route that funds you is the route that shrinks the market. That is the tension the mechanism rests on, and it also gives the bow back its edge and its price at once, because a monster shot at four steps is a monster nobody ever interviewed.

14.9 Negotiation is an attempt, and the order is forced

You may try to negotiate instead of attacking. Try is the operative word, because three things can come back and only one of them is a trade.

it names a price	and you may pay it, or decline and fight after all
it wants nothing	you have spent a round finding out, and the fight starts from a step closer
it flees	the secret leaves the level with it, and nothing you do brings it back

And the order cannot be reversed. You can talk before fighting and you cannot talk after, because of what the previous subsection says about corpses. So negotiation is always available first, always costs at least a round, and is never available second.

Which prices it in the currency Section 9.4 already set up. A monster five steps off gives you three shots before it arrives. Spend two of those rounds asking what it wants and you are standing at one step holding a bow, which reaches nothing. Negotiation is paid for in range, and no new rule was needed to say so.

14.10 Some of them want nothing you have

This is the most important number in the game and it is the one a search should not be trusted with alone.

If every monster can be bought then combat is vestigial and the labyrinth is a shop. If almost none can, then conversation is decoration. Somewhere near a third feels right, and the reason to say so out loud rather than leave it to the tuner is that this single fraction decides what the game is about.

Note also that discovering it wants nothing has already cost a round. Asking is never free, being wrong costs the round and the goods, and a party that spends its light interviewing a room full of things that cannot be bought has learned something true and reaches the trap door too dim to use it. That is the risk that makes the information economy an economy rather than a menu.

But they are not merely an obstacle, and this is worth seeing the right way round. A monster that cannot be bought can only be killed, and killing pays in goods, so the unbuyable third is where the party's income comes from. The fraction therefore sets two things at once. How much of the labyrinth must be fought through, and how much money there is to trade with in the part that need not be.

14.11 Some kinds are hard to please, and each is hard in one particular way

A kind of inhabitant can be difficult, and difficulty is not one thing. There are four ways to be hard to please and they are four different problems.

hard because	the difficulty is	and it is answered by
it wants many	quantity, against weight and capacity	logistics
it wants something rare	availability. The thing may not be on this level at all	search, and some luck
it wants a deep secret	the desire chain behind it is long	walking the graph
it is suspicious	the rapport it takes before it will deal at all	conversation

Read the third column against Section 2.8. Those are the four faculties, and each row is a kind gated on exactly one of them. So a hard kind is not generally hard. It is hard for a particular player, and which player it is hard for is a choice the generator makes when it decides what that kind wants.

14.12 Which is what finally enforces breadth

Section 24.5 wants the strategies to be an antichain, and has wanted it since it was written, without any mechanism obliging the game to produce one. This is the mechanism.

If every way out of a level is operated by a kind that is hard in the same way, a player weak in that one faculty is stopped, and stopped for a reason that was not their decision. If the ways out are hard in different ways, then getting out requires being adequate at several things, and a player may choose which of them to be good at.

No level may have all of its ways out gated on the same faculty, and the route to the surface must require at least three of the four.

That is a fourth generator condition, it joins the three in Section 14.2, and it is checkable in the same way. Each operated way has a kind, each kind has a reason it is hard, and counting distinct reasons along the routes to the surface is arithmetic over a small graph.

And note what it forbids, because it is the thing a player would most like to be allowed. There is no line through this game that is purely conversational, and none that is purely commercial, and none that is purely violent. Section 24.9 lists the Fence and the Sprinter and the Locksmith as lines that ought to be viable, and this condition is what stops any of them being *complete*. Each gets a long way and each hits a way out that wants something it never learned to get.

14.13 And the hard ones must be worth it

One correlation has to hold or the difficulty is spite.

Section 24.11 says a setting in which a secret's price and its value are uncorrelated is a setting where the deep secrets are not worth the chain, and calls it the most likely quiet failure. Hard kinds are the sharpest case of that. A kind that takes twelve turns of conversation and a relic must know something better than what is down the next trap door.

the suspicious	knows what somebody was afraid of, or who was seen. The trusting rung of Section 18.9, and it is the testimony the inquiry turns on
the demanding	operates a way, so what it gives is not a sentence but the surface
the deep	knows the rule, or enough cases of it to save a run of guessing

The last row is worth building. An inhabitant that will simply tell you the colour rule, at the end of a long chain and a hard price, is a legitimate prize because the rule is the compressible half of the table and Section 2.5 says half of it does not compress. Buying the half that does is the single largest thing anybody in the labyrinth can sell.

14.14 What it costs the tuning

`rarityTail`	how skewed the placement distribution is	0–1
`hardKindRate`	fraction of kinds that are hard at all	0.1–0.5
`hardnessMix`	how the four kinds of hardness distribute	a simplex
`facultiesRequired`	distinct faculties the route to the surface must demand	2–4

Faculties exercised per run. The criterion, and it is the direct test of the condition above. Across the best runs, how many of induction, deduction, commerce and conversation each one actually used. If the winning runs all use two, the labyrinth is not asking for what it was built to ask for, and `facultiesRequired` is the knob that says so rather than a fault in the player.

14.15 What this costs Section 27

One earlier claim has to be withdrawn, and it is better to say so than to leave two sections disagreeing.

Section 27 says the map need not travel in the prompt, because nothing below needs to know where the party has been. That was true while a secret was a souvenir. It is false the moment a

secret is currency, because a currency has to be somewhere the counterparty can be told about it, and the only channel is the prompt.

> So the record gains one more field and the levels lose a little of their ignorance. What travels down is the goods, the clocks, the obligations, and the secrets the party is prepared to trade. Where the party has walked is still the client's business. What the party knows is not, and the difference between those two is the difference between a purse and a diary.

15 The desire graph is alive

$$\text{level}(d) \text{ on being prompted } = \text{ settle}^{\,\Delta} \triangleleft \text{ room}$$

One line and one exponent. The Δ is how many descents have gone by since anybody looked, and the chain is the level catching up before it answers. That is the whole mechanism.

Here is the objection this note has not answered, and it is fatal until it is.

Everything a monster does is permissive. It stands aside, hands something over, or comes along. In every case its power is to stop being the obstacle it was itself constituting. So a monster is a lock, a secret is a key, and the two economies are an elaborate key-cutting operation. Nothing whatever happens *to* the party. Every change in the labyrinth was caused by the party walking up to something and dealing with it.

A world in which nothing moves unless you move it is a puzzle box. Tension requires that the situation can get worse while you are not looking.

So the monsters trade with each other. The desire graph of Section 14.2 is not a fixed structure the party explores. It is a market that settles on its own, and every descent you make it has moved.

15.1 What resolves, and what that costs you

A monster whose desire is another monster's secret does not need the party in order to be satisfied. The two of them are in the same labyrinth. Given time they deal, and then the edge is gone.

an edge resolves	two monsters trade, and the one that wanted is satisfied. It no longer needs you, so it has nothing to buy and nothing to sell
a secret spreads	once traded it is abroad, which Section 14.3 already says. Now it happens without your help, so a secret you were saving becomes common knowledge
a monster dies	something ate it. The secret is gone, and if it was the source of a chain the chain is unenterable
a monster moves	it is guarding something else now, and what it was guarding is either open or held by something worse

Read the first row twice, because it is the one that bites. The whole of your leverage over a monster is that it wants something it has not got. A monster that has been satisfied by somebody else is a monster you can only fight.

Which turns the deadline of Section 12.1 into a race rather than a countdown. You are not merely running out of descents. You are running out of a *market*, because the thing you were going to trade in is closing itself while you climb, and the chain you found on level two has two fewer edges by the time you have the goods to enter it.

15.2 And it costs no new machinery

The mechanism is the one the note already has, which is why this belongs in the design rather than in a wish list.

A level is a program holding state, and Section 23.7 keeps it alive after the party has gone. Section 28 sends the clocks down inside the prompt. So a level being prompted knows how many descents have elapsed since it last answered, and it advances its own graph that many steps before it says anything.

$$\text{level}(d) \text{ on being prompted } = \text{ settle}^{\Delta} \triangleleft \text{ the room the party is in}$$

> So time reaches the level the same way everything else does, which is inside the prompt. Nothing polls, nothing runs in the background, and no level needs a thread. A level is idle exactly when nobody is looking at it, and it catches up on being looked at. That is the same trick the tower plays with control flow, one storey down.

It also settles a question Section 2.16 raised and left open. Two parties in the same labyrinth already meant a level had to hold state and that the same prompt need not give the same response. This is that concession used deliberately instead of tolerated, and a solitary player now gets the thing that made multi-party worth having.

15.3 Which is where the story comes from

Three sources, and this is the one that makes the other two urgent.

The inquiry of Section 19 gives the player something to establish. **Recruits** of Section 17 give them somebody to travel with who has wants of their own. Neither is pressing on its own. A settling market is what makes both pressing, because a witness you needed can be dead, satisfied, or moved by the time you come back for them.

> And it authors itself. You climb out of level four meaning to return with the brass key, and when you do the monster has what it wanted and will not speak to you, the one who told you about it has been eaten, and the secret you were carrying to sell is now something everybody knows. Nobody wrote that. It is what a graph does when it is left alone for two descents.

Note the shape of the resulting decision, because it is a new one and the game did not previously have it. Every trip you now choose between going deeper, which is what the third end wants, and going back to close a trade before it closes itself. That is urgency, it is generated rather than scripted, and it is the answer to the objection this section opened with.

15.4 The one rule that keeps it fair

A market that settles behind your back is unplayable if you cannot see it settling.

So a level tells you what changed. Not everything, and not for free. When a party re-enters, the level reports what has moved in the room the party is standing in, and anything further costs a question like everything else. The trace of Section 30 shows it on the way down.

And one thing must be guaranteed rather than tuned. A chain the party has already entered may not become unenterable, because a puzzle that was solvable when you started it and is not solvable now is a game cheating. So resolution never removes the last path to an open obligation, which is the third generator condition of Section 14.2 re-checked after every settle. It is the same graph property and it now has to hold over time rather than once.

15.5 What it costs the tuning

Two knobs and one criterion, and the criterion is the one that decides whether this is tension or merely spite.

`settleRate`	edges that resolve per elapsed descent	0–0.4
`attritionRate`	monsters that die or move per elapsed descent	0–0.2

Opportunities lost to settling. Of the trades a run had identified and not yet made, how many were gone by the time it went back. At zero the labyrinth is inert and this section bought nothing. Near one the player is being punished for planning, which is worse than inert because it teaches them not to. The interesting setting is where a good player loses some and can tell which ones were worth hurrying for.

And that criterion is the reason `settleRate` is not a difficulty slider. Raising it does not make the game harder in a way skill answers. It makes planning worth less, and planning is the skill the whole design is about, so this is the one knob where a higher value can reduce skill depth while looking like a challenge.

16 The goal structure is a container too

Section 14.2 drew the desires as a directed graph and Section 15 let that graph move. Neither said what the object is, and it is the same object as everything else in this note. This section says so precisely, because whoever implements the generator has to build it and the structure decides how.

16.1 The statement

To be given the sacred item you must satisfy its holder. To satisfy its holder you must hold a particular relic. To hold that relic you must get past the way it lies behind. To get past that way you must satisfy its gate-keeper, and so on down. A goal names the goals that must be discharged before it, each of those is a goal, and the recursion stops at goals the party can pay from the purse it walked in with.

$$\begin{aligned} \text{goal} &\triangleleft \text{sub} \\ \mathrm{W} &= \mu X.\ \textstyle\sum_{g} X^{\,\mathrm{sub}(g)} \end{aligned}$$

A container is a set of kinds and, for each kind, a set of positions. Here the kind is which goal this is and the positions are what must be discharged first. So the goal structure of a labyrinth is the W-type of that container, which is to say the well-founded trees over it.

in the container	in the labyrinth	and it is fixed by
the kind	which goal this is. The sacred item, a riddle, an operated way, a desire	the generator, when the level is written
the positions	what must be held or done first. Its pieces, its price, its clue	`piecesWanted` for riddles, the desire for a market goal
a leaf	a goal with no positions, payable out of the purse	the source condition of Section 14.2
the height	how many goals deep the deepest branch runs	`chainDepth`, one to five

Which makes `chainDepth` and `piecesWanted` two readings of one object rather than two knobs in different blocks. One is the height of the tree and the other is its branching, and together they bound the number of leaves, which is the number of separate things a party has to come home with. A tuner that moves one without looking at the other is moving the same quantity twice.

16.2 Two of the generator conditions are the W-type's

The conditions of Section 14.2 were given as rules a generated labyrinth must satisfy. Read against the container they stop being a checklist and become the two things that make a W-type exist and be inhabited.

the condition	is	and without it
acyclicity	well-foundedness	the trees are not W but M, so a branch may descend for ever and the goal can never be grounded out
a source in the goods	an inhabited base	every branch terminates and none of the leaves is a thing the party can pay, so the tree is well-founded and still unreachable

Which is why Section 24.25 puts them in the checked column rather than the sampled one, and now the reason is structural instead of merciful. A cyclic desire graph is not an unfair labyrinth. It is not a goal structure at all, because the initial algebra it would be the carrier of does not exist.

16.3 And it runs against the other container

The note now has two recursions over the same territory and they point opposite ways.

	the container	height	and it is
the labyrinth	level(d) $\lhd$ level(d+1)	eight	unfolded downward, on demand, and it does not exist until somebody goes there
the goals	goal $\lhd$ sub	`chainDepth`	folded upward, as things are carried home

The positions of a goal always lie deeper than the goal, which is Section 2.13's rule that the labyrinth is unlocked from below, stated as a property of the container rather than as a piece of advice. So the goal tree is embedded in the level tree with its orientation reversed, and the relic from the deepest level opening the way at the shallowest is one edge of it drawn at full length.

Which is the argument of Section 7.9 and it is now a sentence about algebra. The descent is an unfold and costs nerve. The ascent is the fold and costs table. A run is not symmetric because generating a tree and consuming one are not the same act, and the expensive one is always the consuming.

And the protocol between the two is the container's extension read as a question and an answer. What goes down in the prompt is the positions not yet filled, which are the open obligations. What comes back up is their fillings, carried in hands. So $\sum_g X^{\,\mathrm{sub}(g)}$ is not a description of the goal structure written after the fact. It is the wire format, with the exponent travelling down and the sum coming back.

16.4 What the implementation has to hold

Written out because the structure decides the generator and a generator written the obvious way gets it wrong.

Build the tree, do not build a graph and check it. The temptation is to place desires at random and then test for cycles, repairing when the test fails. Do not. Generate the goal structure as a tree from the root, giving each position a payer at a strictly greater depth than its parent, and acyclicity is not a property you verified. It is a property you could not have violated. Section 29's principle applied one level up, which is that an illegal state should not be writable rather than caught.

Generate top down and pay bottom up. The root is the sacred item at `goalDepth`. Recursively give each goal its positions, each assigned a payer deeper than it, until a position is payable from the purse. The generation order is the reverse of the play order and both are the same tree.

A goal's discharge takes exactly its positions. This is the fifth table of Section 29, the one indexed by what the party is carrying. A party holding two of three pieces cannot argue its way through with any words at all, and that is the position set being incomplete rather than the gate-keeper being stubborn.

The tree is re-read every descent and never cached. Section 15 settles desires while the party is away, so a goal's positions may differ between one descent and the next. Hold the tree in the levels, which Section 23.7 keeps, and never in the party record. A party carrying a stale copy of the goal structure is a party told a lie by its own bookkeeping.

Count the leaves against the hands. The leaf count is bounded by the branching raised to the height, so `piecesWanted` and `chainDepth` together decide how many separate things must come home. Six hands and a finite count of descents bound how many can. That product is the serviceability condition and it has to be computed, not hoped for.

Three failures to make unrepresentable rather than detectable. **A position shallower than its goal** inverts the unlock direction and lets a party finish on the way in. **Two goals sharing one position** makes the tree a graph again unless the item is genuinely duplicable, and most are not, so the second goal is unsatisfiable and nothing says so. **A leaf that is not payable from the purse** passes acyclicity, passes the height check, and produces a labyrinth that is provably finite and still cannot be started.

17 A satisfied monster is a container

$$\text{party} \;=\; \text{you} \boxtimes r_1 \boxtimes \cdots \boxtimes r_k$$

The tensor, on the player's side of the room for the first time. All of them prompted from the same state, none of them seeing what another did, which is exactly what Section 2.14 said the better game would look like.

Everything a monster can do so far is permissive. It stands aside, or it hands something over, and in both cases what it did was stop being an obstacle. That is not enough to hang a game on. An obstacle that can be removed two ways is a lock with two keys, and a labyrinth of them is a corridor with paperwork.

So a satisfied monster may offer to come with you. And a monster that comes with you is a container, which means it is a thing you can prompt.

Which is the note's own thesis pointed at the player for the first time. Section 21 argues that a monster is a container because its prompts are anything you can say and its responses are a finite table. Nothing in that argument depended on the monster being in front of you rather than beside you. A recruit is the same container, composed into the party instead of standing against it.

17.1 The tensor comes back, on the player's side

Section 2.14 said the party is one thing rather than several, admitted that a party whose members acted separately would be a tensor standing opposite the tensor of monsters, and called that the better game. Section 10.1 then reduced the monsters of a room to one and the tensor there degenerated.

This is where it returns, and it returns in the place Section 2.14 always said it belonged.

$$\text{party} \;=\; \text{you} \boxtimes r_1 \boxtimes \cdots \boxtimes r_k$$

All of them are prompted with the state as it stood at the start of the round. None of them sees what another did until the round is over. The independence is enforced rather than promised, because every prompt is built before any is sent, and that is the same sentence the note used about a room full of monsters.

So the operator did not lose a home, it changed sides. And the reason that matters is not symmetry. It is that a recruit prompted in parallel with you is a recruit whose answer you did not choose, which is where a companion stops being equipment and starts being somebody.

17.2 What a recruit can do

One power each, from a finite table, because Section 23 needs the table narrow for the generator to be checkable and this is no exception.

power	what it does	the constraint it relieves
`carries`	takes items, so the party's hands go further	hands
`lights`	burns instead of the torch, or slower than it	light
`fights`	acts against the guard in its own right	light and people
`knows`	names the desire of whatever is in front of you	`desireVisibility`
`keeps`	holds goods at depth between descents, so a haul need not be carried up in one trip	hands, across descents
`speaks`	makes the offer on your behalf, at its odds rather than yours	the market itself

Read the right-hand column, because it is the argument for the whole section. Every power relieves a constraint the game already had, so a recruit is not a new subsystem. It is a hole punched in an existing one, at a price, and the price is the next subsection.

The two to notice are `keeps` and `speaks`. A monster that holds your relics on level six defeats the tyranny of hands over a whole game rather than over one trip, which is the single largest thing anybody could do for you. And a monster that negotiates for you is delegation in the sense the rest of this note means it, because you are prompting a container which prompts another container, and the depth of that chain is the depth of the tower.

17.3 A recruit is a standing desire

It came with you because you satisfied it once. It stays because you keep satisfying it.

Every recruit has a retainer, which is its desire again, recurring. Meet it and it stays. Fail to meet it and it leaves, at the end of the descent or in the middle of one, and what it was carrying leaves with it.

a good	the cheap kind of retainer, and the kind you can plan for
a secret	it wants to keep being told things, so it competes with the monsters you wanted to sell to
another recruit's secret	the expensive kind, and the one that writes a story without anybody having to write one

So a companion is a recurring cost and not a reward, and `carries` is the one to be frightened of. A monster carrying four relics that leaves on level five because you could not pay it has taken the descent with it, and you agreed to that arrangement when it was convenient.

17.4 And this is where the story is

The note has had no story in it and did not need one to make its architectural point. It needs one to be worth playing, and three consequences of the above supply it without anybody writing prose.

The party acquires a history. A recruit is a thing you met, paid, and kept, and by level six you are travelling with three of them. Nothing about that had to be authored. It is the record of what you could afford.

Loyalty is a resource with a rate. Every recruit is a subscription, so the party's running costs rise with its capability, and there is a size beyond which you cannot afford the party that would let you go deeper. That is a curve, and curves are what a player remembers.

Interests conflict. A recruit whose retainer is a secret is competing with the market you were selling into, and a recruit who wants another recruit's secret has put you in the middle of something. Section 14.3 says a secret can only be sold once, so satisfying one of them is refusing the other.

Which is the answer to the objection this section opens with. A monster that can only stand aside is a lock. A monster that can join you, help you, cost you, and leave you at the worst possible moment is a character, and it became one without a single line of authored dialogue. What made the difference is that it kept its desire after being satisfied, and desires that persist are what people are made of.

17.5 What it costs the tuning

Three knobs and one criterion.

`recruitRate`	fraction of satisfied monsters that offer to come along	0.1–0.5
`retainerRate`	how often a retainer falls due, in descents	1–3
`desertRate`	how readily an unpaid recruit leaves mid-descent	0–0.5

And the criterion is the one this mechanism can fail at quietly. **Party size at the reckoning**, against the haul and the depth reached. If the best runs all end with the largest possible party then recruits are strictly good and the retainer is not biting. If the best runs all end alone then recruits are a trap and nobody will take one twice. The interesting setting is the one where the winning party size varies with which end the player was playing for.

18 The special interest

$$\begin{aligned} \text{chat} &= \text{say} \lhd^{*} \big(\text{told} + \text{nothing} + \text{attacked}\big) \\ \text{liking} &= \text{liking} \lhd \text{chat} \end{aligned}$$

Two lines that were introduced in Section 11 and are unpacked here. What decides the branch is one number, and what moves the number is whether you had anything to say.

Not everything down there is a monster and the ones that are not should not be called monsters. Call them civilians.

The workings flooded and people came out of them, and Section 19 says the labyrinth is an inquiry into that. So it is inhabited. There are things that guard and things that roam, and there are also people getting on with their lives in the dark, and those people are the witnesses the whole case is built out of.

A civilian carries no weapon, will not attack you, and has three fields in its prompt. A secret, a desire, and **a special interest**, which is something it likes talking about.

And there is a third thing you can do with a turn. You can chat to it. Not bargain, not threaten, not propose anything. Talk about the thing it likes.

You are not making a request. You are being good company on a subject somebody else cares about, across as many turns as it takes, and if that goes well you are told something. Whether it goes well is entirely the civilian's business.

Which is the only mechanism in this note where the model's judgement is the whole mechanism. Everywhere else it decides within a structure. Whether a price was met. Whether an account is consistent. Here there is no structure at all. It decides whether it is enjoying itself, and nothing checks the answer.

18.1 If you cannot say anything interesting you will never get the secret

This is a design commitment and it is worth putting on its own line, because it is the only place in the note where the player is not mediated by anything.

The words are the mechanic. A player who cannot say something interesting about what a civilian cares about does not get the secret, and no quantity of anything else substitutes.

Everything else in this game is mediated. Combat goes through weapons, ranges and a pool of hit points. Trade goes through weight, goods and stated terms. Puzzles go through consistency over testimony. All of those are numbers standing between the player and the outcome, which is what makes them tunable.

Chat has no character sheet. There is no statistic that makes you better at it, nothing to buy that improves it, and no amount of gold or light or people that compensates for having nothing to say. It is the one thing in the labyrinth that cannot be brute-forced, farmed, or looked up, because the thing being tested is whether you were interesting.

That cuts both ways and the cost is real. A player who will not engage with it has locked themselves out of the cheapest information in the game, and no design decision can rescue them, because rescuing them would mean accepting uninteresting conversation. Which is the trade being made deliberately, and it is the trade that makes having a language model down there worth the trouble at all.

18.2 It fixes a problem the note had and did not notice

Consider the first ten minutes of a game. You have no goods worth trading, no colour rule, and no secrets to sell. Every route into the information economy of Section 14 is closed, because all of them are priced in something you do not yet have.

route to a secret	**costs**	**and at the start**
buy it	goods, at dagger's length	you have nothing to give
watch a roamer	several turns of patience	you do not know which are worth following
induce the rule	cases	you have none
chat	**turns, and nothing else**	**is the only one that works**

So chat is the bootstrap. It is the one action that produces information out of nothing but time, which makes it the poorest party's whole strategy and every party's opening.

And it is why the civilians have to exist as a separate population rather than being peaceable monsters. A guard that would chat is a guard you talk past for free, which undoes Section 10. A civilian guards nothing, so nothing is undone by its being willing to talk, and the opening of every game is finding one.

18.3 And it is safe, which gives it a niche

Trade must happen on the square. Chat need only happen within earshot, so you may do it from four squares away with a bow in your hands and never come nearer, and with a civilian there was nothing to be afraid of in the first place.

	range	**costs**	**and what you get**
attack	whatever is in hand	light, arrows, sometimes people	an item, and silence
chat	anywhere it can hear	turns, and nothing else	something, at its discretion
negotiate	its square only	turns, exposure, and goods	the terms, stated

Read the middle row against Section 12.5. The party carrying only a bow was locked out of the market and it is not locked out of conversation, so it can still learn. What it cannot do is buy. Chat is the safe and unreliable way in, trade is the exposed and explicit one, and the difference between discretion and stated terms is what separates them.

18.4 Discretion is a third kind of uncertainty

The game now has three and they are genuinely different, which is worth setting out because a player has to handle each differently.

	what it is	and you handle it by
motive	structured lying, by what the truth costs it	inference, which is Section 19.2
the rule	a pattern drawn fresh each game	induction, which is Section 20
discretion	whether a thing is enjoying itself	neither. Nothing reasons its way to being liked

Which sounds like noise, and would be, except that the special interest is a fact about the monster and facts about monsters are for sale.

18.5 So the interest is the key, and the key is buyable

You cannot argue a monster into liking you. You can find out that it likes talking about water, and then talk about water.

> Which turns discretion from a coin into a skill without taking the discretion away. A player who knows the interest and uses it is likely to be told something. A player who guesses is not. And nothing is guaranteed either way, because the thing genuinely decides, and that is the property Section 21.4 says is the reason for building any of this.

So *what it likes talking about* joins the kinds of secret in Section 14 as a seventh, and it is the most immediately useful of them, because it does not open a way or name a price. It makes the cheapest action in the game work.

It also gives the inductive layer a second thing to predict. Section 12.5 says a kind of monster has a fixed desire and Section 20 says the desires follow a rule. The interests do too, and it need not be the same rule.

one rule	the thing that wants green also likes talking about green, so working out one mapping gives you both
two rules	wants and interests are independent, so there are two patterns to find and the inductive game is twice as long

Whether they coincide is a parameter, and one rule is the easier labyrinth for an obvious reason.

18.6 An interest is a secret, so there is a market in introductions

What a civilian likes talking about is a fact about it, and Section 14 says facts about others are the market half of the information economy. So an interest is something you buy.

Which gives a second chain, over the same population, running alongside the first.

graph	an edge $A \to B$ means	and walking it costs
desire	A wants what B has or knows	goods, and three turns at zero squares
interest	A knows what B likes to talk of	turns, and having something to say

So the poor party walks the interest graph and the rich party walks the desire graph. One is paid for in patience and the other in property, they reach overlapping parts of the labyrinth, and a player short of one is not thereby short of both. That is the two economies of Section 14 appearing again one level up, in the routes rather than in the currency.

An introduction is therefore a thing with a price. You spend four turns being interesting to a cook, who tells you that the diviner two levels down cares about the sound water makes in a dry shaft, and now you can go and be interesting to the diviner on the first try instead of the fourth. Chat without an introduction is chat where the first several turns are spent finding the subject.

18.7 Flavours give a prior, and the interest is still individual

Civilians come in flavours. A miner, a water-diviner, an archivist of the works, a cook, a child who was above ground that night.

The flavour is a kind in the sense of Section 12.5, so it fixes the desire and the desires follow the rule of Section 20. It does not fix the interest. It only suggests it.

	fixed by	**so you get it by**
its desire	the flavour, under the rule	**deduction and induction**
its interest	the individual, within the flavour's subject matter	**buying it, or guessing from the flavour and being wrong a few times**

Which is the right division and it is why both mechanisms are worth having. A diviner's enthusiasms will be about water, so the flavour narrows it, and whether this diviner cares about the sound of it or the taste of it or where it went is particular to this diviner and to this game. So the flavour is a prior and the interest is the thing itself, exactly as Section 20 has an attribute that predicts and a confound that nearly does.

And the two objects are reached by two different faculties, which is the reason the note now has three activities rather than two. Desires are *worked out*, because they follow a rule and the rule is inducible. Interests are *found out*, because there is nothing to induce and somebody has to tell you or you have to try. Neither faculty substitutes for the other.

18.8 Where the generator may invent freely

That section ends by asking whether the generator may write the monsters' prompts and the puzzle clues, and calls it the difference between a dungeon that surprises the player and one that surprises its author. It is answered here, and the answer comes out of Section 7.5 rather than out of taste.

Recall the three roles. A thing is a container, or an index on a shape, or state. And the discipline the whole note rests on is that an index must be finite, because that is what makes a dependent type expressible and a generated level checkable.

an index	must be finite and declared in advance. The eight actions, the kinds of loot, the seven kinds of secret, the squares of a room
state	may be anything at all, because nothing validates it and no response type depends on it

An interest is state. It is written into a prompt and it is mentioned inside a secret, and that is the whole of its career. It never indexes a shape, never selects a fibre, never determines what type comes back, and no decoder at any boundary inspects it.

So the generator may invent freely here and could not invent freely anywhere else, and the type system is what says which is which. Claude may decide that this archivist is preoccupied with the handwriting of a foreman nobody has mentioned, and nothing in the labyrinth notices. Claude may not decide that there is a ninth thing a monster can do.

Which is a better answer than a policy would have been. The question was where invention is safe, and the answer is that it is safe exactly where the algebra was never looking. Everything that crosses a boundary is finite and checked. Everything that only ever sits behind one may be as strange as it likes.

18.9 Rapport accumulates, and it is the only thing in the game that does

One conversation is not a transaction. You do not chat, roll, and collect. You win somebody over across several turns, and what you build up stays built.

it accumulates	each turn that says something new about what it cares for adds to it
it persists	a civilian you won over on the second level knows you on the fifth, because Section 23.7 keeps the levels alive
it can be lost	by repeating yourself, by threatening, or by what Section 18.14 says gets around

Which makes it the only rising line in the design. Light burns down, descents run out, the roster shortens, capacity is fixed and the market closes itself. Everything else in this note is attrition. Rapport is the one quantity that goes up, and a long game needs something that does or it is only a slow loss.

Note where it lives. Not in the party record of Section 28, because it is not something the party carries. It belongs to the civilian, and a civilian belongs to a level, so rapport is in the state column of Section 7.5 with everything else the algebra cannot see. Which is worth flagging rather than hiding, because it means nothing in the type system guarantees a thing remembers you.

18.10 And it gates what you are told

The point of accumulation is that it buys different things at different heights, and the ladder is where the design pays off.

civil	it will tell you its own desire, which is the cheapest useful thing there is
warm	a secret about the world. What is down the shaft, where a piece lies
confiding	a secret about the market. What somebody else wants, what somebody else likes
trusting	what it is frightened of, and who it saw

Read the last row against Section 19. The culpable will not confess to a stranger and the frightened will not name what frightens them to somebody who walked in five turns ago. So the deepest testimony in the labyrinth is not bought, fought for, or deduced. It is only available to a player who genuinely won somebody over, and that is what makes the inquiry a story rather than a satisfiability problem.

18.11 Which gives the frightened a second cure

Section 19.2 says a frightened witness can be made honest by removing what it fears, and offered that as the mechanic. It is a poor mechanic on reflection, because it means the answer to fear is always to go and kill something.

There is now a better one. Enough rapport and it tells you anyway, because it trusts you more than it fears them.

	costs	**and**
remove the fear	finding the thing, and killing it	needs light, weapons and a corpse
outweigh the fear	turns, patience, and having something to say	needs nothing but you

Two routes to the same testimony, one violent and one conversational, priced in different currencies. Which is the pattern the whole design keeps arriving at, and it is the first time it has arrived at it for something that is not a locked way.

18.12 Seduction is measured in novelty, not volume

The obvious way to cheat this is to say a great deal, so the rule has to be that saying a great deal is not the thing.

Each turn is judged on whether it added something. A new thought about the subject, a question that opens it up, something the civilian had not considered. A turn that restates the previous turn adds nothing and costs a round of light like any other, so a player padding is a player burning their lamp to no effect.

Which is the honest version of what is being asked. Not charm, exactly, and not flattery, which Section 21.3 warns about and which has nothing to attach to here because the subject is fixed and it is not the civilian. What is being asked is whether you can be interesting about somebody else's enthusiasm several times in a row, which is a harder and more particular thing than being pleasant.

18.13 It is the softest thing in the game, and that is a real cost

Worth naming rather than glossing, because this is the mechanism most open to being gamed.

Section 21.3 worries that a player will simply write flattering essays at a language model until it yields, and Section 19 answers that for gate-keepers by giving a magistrate a decidable standard. Chat has no such standard. There is nothing to check, because enjoying a conversation is not a property with a checker.

Two things limit it and neither is a guarantee. The interest is specific, so an essay about the wrong subject is an essay about the wrong subject, and a civilian prompted to care about water has no reason to warm to a speech about how splendid it is. And what chat yields is one secret or one desire, never a way opened and never goods, so a player who has mastered charm has mastered the cheapest and least decisive action available. The expensive things still have to be paid for.

18.14 And you can murder them

They are unarmed, they carry items, and nothing stops you.

Which has to be said because the design permits it and pretending otherwise would be dishonest about what has been built. A civilian killed yields whatever it was carrying at no risk whatever, so violence against the defenceless is the cheapest income in the labyrinth.

And it costs you the thing it was going to tell you, which no other loss in the game is quite like. Section 15 has secrets spreading between inhabitants without your help, so what spreads also includes what you did. A party known for killing civilians is a party the civilians have stopped talking to, and the mechanism for that was already built for other reasons. The cheapest income closes the cheapest information, permanently, and the inquiry it was gathering testimony for is the thing it was in aid of.

18.15 What it costs the tuning

`rapportSteps`	turns of novel conversation to climb one rung	1–4
`rapportDecay`	how much is lost per descent away, if any	0–0.3
`tiers`	how many rungs of disclosure there are	2–4

`rapportSteps` is the whole cost of the information economy's cheapest route and therefore sets how much of the game is conversation. At one, a civilian confides in anybody who says one clever thing and the labyrinth has no secrets. At four, the top rung costs twelve turns of light and no party can afford to be trusted by more than one person.

Rungs reached per run. The criterion. If the best runs never get above *warm* then the top of the ladder is decoration and the inquiry's deepest testimony is unreachable, which makes Section 19.9's fourth puzzle the only one anybody ever meets. If they reach *trusting* with several civilians, conversation is underpriced and everything else in the note is optional.

18.16 What it costs the tuning

`earshot`	squares at which chat is possible	2–6
`interestLeverage`	how much knowing the interest raises the chance of being told something	0–1
`sameRule`	whether interests follow the same mapping as desires	yes or no
`chatYield`	whether a pleased monster gives a secret, a desire, or either	a choice

`interestLeverage` is the one that decides whether this is a mechanic or a lottery. At zero, knowing what a thing likes is worth nothing and chatting is a coin you flip with turns. At one, knowing the interest is a guarantee and chat replaces the economy, because why pay for what politeness delivers.

Where the secrets came from. The criterion, and it now has four columns rather than two. Across the best runs, what fraction of secrets arrived by chat, by trade, by watching a roamer, and by inference from the rule. All four should appear, in different proportions for players playing for different ends, and any column at zero is a mechanism that cost pages and earns nothing.

19 The labyrinth is an inquiry

$$\mathrm{puzzle}(d) \;=\; \mathrm{gate\text{-}keeper}(d) \;\triangleleft^{*}\; \big(\mathrm{wing}(d) \;+\; \mathrm{refused}\big)$$

The oldest line in the note and the one it was written around. This section is what makes the gate-keeper's verdict something other than a whim, which is the hole Section 21.3 names.

The note has had no story and has not needed one to make its architectural point. It needs one to be worth playing, and the story cannot be decoration hung on the mechanics or it will be skipped. It has to be the thing the mechanics already are, which is finding out what things want.

They are already a problem of testimony. There is an information economy, its currency is secrets, and the secrets may be false. So the story is the only one that structure can carry.

Something happened at the bottom. The lower workings flooded and the people in them did not come out. Every monster in the labyrinth was there or knows somebody who was, and you are going down to establish what occurred and who allowed it. The gate-keepers on the locked ways are not riddlers. They are magistrates, and they will open a way when your account of events is consistent with the testimony you are carrying.

Which makes the game a single sentence. *You are assembling a case out of the words of creatures with reasons to lie, and the light is running out.*

19.1 Why this story and not another

Four things it does that a quest for treasure does not.

It makes the secrets matter to each other. Loot is loot and one relic does not bear on another. Testimony is different, because two statements can contradict, and a contradiction is

information neither statement carried alone. The information economy stops being a shop and becomes a body of evidence.

It gives the gate-keeper a criterion. Section 21.3 names an obvious hole, being that a gate-keeper is a language model argued with by somebody who wants a particular verdict, and that nothing in the type system helps. A magistrate has a standard. Your account must be consistent with the testimony you hold, consistency over a small set of statements is decidable, and flattery has nothing to attach itself to.

It makes going back down the natural act. You need one more witness. Nobody has to motivate that.

It gives the deadline a reason. The descents of Section 12.1 run out because the inquiry closes. A finding delivered afterwards is not a finding.

19.2 Why would it lie?

Section 14 gave every monster a `truthRate`, being a probability that its answer is true. That was wrong twice over and this subsection replaces it.

The first fault is technical. A monster that lies at random supplies noise, and noise cannot be reasoned about, only averaged. A monster that lies *consistently* supplies information, because if you know it always lies then its answer tells you the truth exactly. Raymond Smullyan built a corpus on that distinction and it is the right one to build this on.

The second fault is the one that matters more. Labelling a monster a liar does not say why, and an unmotivated liar is barely better than a random one. The player can reason about the structure of the testimony and cannot reason about *who* to expect a lie from, which is the more useful thing to be able to do.

> So nothing lies because it is a liar. It lies because the truth costs it something, and what the truth costs it is a fact about the labyrinth rather than a label on a monster. Section 19 says people drowned in the lower workings. Somebody let that happen.

19.3 Five motives, and each one lies about something different

This is the part that makes lies inferable rather than merely present.

motive	what it lies about	and how you can tell
culpable	what happened, and who was where when it happened	its lie protects exactly one thing, and the shape of the protection points at it
complicit	the whereabouts of another, never its own	its lies map the conspiracy, because you learn who it is shielding
afraid	anything at all, if the truth might reach whoever it fears	inconsistently, and it stops when the fear is removed
trading	its own desire, to raise the price. Never about the flood	it lies only in the market and is a reliable witness about the world
honest	nothing	it agrees with the other honest ones, which is how you find them

The second column is where the design earns something. A motive does not just say whether a monster lies, it says *about what*. A trader will sell you an inflated price and give you a true account of the flood. A culpable monster will deal with you squarely all day and lie about one night. So the player is not learning who is honest. They are learning which questions to trust from whom, and that is a far better thing to have to learn.

> And it collapses two layers into one. Detecting the lies and solving the case were separate activities a moment ago, one logical and one narrative. They are now the same activity, because the lies are distributed by culpability, so mapping the lies *is* the finding. A player who has worked out who is lying about what has already answered the magistrate's question.

19.4 You can make a witness honest

The third motive is the one with a mechanic hiding in it.

A frightened monster is Smullyan's erratic third kind, and it is erratic for a reason. It is recalculating a different risk each time you ask, so it cannot be pinned down by logic. But fear has an object, and the object is a thing in the labyrinth.

So remove it. Kill what it fears, or carry proof that the thing is dead, or offer it something better than silence. A frightened witness becomes an honest one, and no amount of clever questioning would have achieved that.

> Which is an action on the *reliability* of testimony rather than on its content, and nothing else in the design does that. Every other move buys a statement. This one changes what statements from that source are worth, permanently, and it is the reason *what it fears* is one of the five kinds of secret in Section 14. That row was listed as market intelligence. It is better than that. It is the price of a witness.

19.5 The type system holds it, not the model

The handler knows the true secret, the monster's motive, and what the monster has to lose. So the response is computed rather than hoped for. A culpable monster asked about the flood returns the negation, and the same monster asked about the way down returns the truth.

What the model decides is the wording, the willingness, whether the price was met, and everything about how it deflects. Freedom on the way in and a table on the way out, which is Section 21 for the fourth time.

> Which also means a player cannot get at a motive by asking about it. A culpable monster asked whether it is culpable is being asked about the flood, and it lies. The motive is inferred from the pattern of what it says, never read off it, and that is exactly the position a real inquiry is in.

19.6 And the knob is better than the one it replaces

`truthRate` is deleted. In its place `motiveMix`, the proportions of the five, on a simplex, and two components of it are worth naming separately.

the culpable and complicit	together they are the size of the conspiracy, and that single number is how hard the case is
the afraid	the fraction of the testimony that no amount of reasoning will settle until the player does something in the world about it

`truthRate` could only make the labyrinth noisier or quieter. `motiveMix` decides whether it is solvable by reasoning, how large the thing being concealed is, and how much of the work is done by argument rather than by force.

19.7 Light is the question budget

Smullyan's puzzles almost always come with a constraint on the asking. You may put one question. You may put two. The constraint is what makes them puzzles rather than interviews.

This game already has that constraint and did not install it for this reason. Asking costs a round and rounds are light, so the number of questions a party can afford is the number of rounds it can spare, and that number is also the way home.

> Which is the best accident in the design. Smullyan's central restriction and the Labyrinth's central resource are the same quantity. A player standing on level five with four rounds of light and three witnesses is inside a Smullyan problem whose question budget was set by how hard they fought on level two.

19.8 What the puzzles should be

Five kinds, in the order a player should meet them, and the last is the one worth building the game for.

	the puzzle	what it teaches
0	A gate-keeper declares it wants three clues about the flood. Go and get them.	nothing. This is logistics, and it is what the first draft of this note had
1	A gate-keeper wants three particular things and will not say which. You find out by asking other monsters, and their answers cost what their desires cost.	**the floor of the interesting range, and the first rung that needs a market**
2	Two witnesses contradict. Which is the knave?	that testimony has a structure, and contradiction is itself evidence
3	Several statements, some false, one consistent account.	that the answer is an assignment and not a fact
4	One round of light and one witness of unknown kind. Frame a question whose answer informs you whichever kind it is.	Smullyan's signature move, and the first genuinely hard thing
5	A witness whose statement is about its own testimony. *I would not tell you what is down there.*	self-reference, and that some statements convict their speaker
6	A magistrate who asks not what happened but whether what happened can be established from the testimony that exists.	that *it cannot be determined* is sometimes the correct finding

Rung one deserves its label in bold, because it is the smallest puzzle worth generating and it is therefore the target the generator has to hit before any of the others matter. Note what separates it from rung zero. Both are fetch quests, and the difference is that in one the specification is handed to you and in the other the specification is the puzzle. That single change is the whole distance between an errand and a game, and it costs nothing but withholding a sentence.

So the minimum viable labyrinth is a hidden shopping list, discovered by asking creatures who want paying for the answer. Not much of a story and quite enough of a game, and everything above rung one is refinement on a loop that already works.

The sixth is the one to build the game for. A gate-keeper who wants an honest account of what the evidence does not settle is asking for something no other game rewards, and a player who says the question is underdetermined and shows why has done something better than solving a riddle.

And it closes the last loop in the note. Section 21 says nothing in the program knows a puzzle's answer, and that was stated as a limitation to be lived with. Under puzzle six it is the subject. The game is about what can be established from insufficient testimony, so a program that did not know the answer is not admitting a gap. It is modelling the position the player is in.

19.9 Four of Smullyan's, in the labyrinth

Worked examples, because a taxonomy of puzzle kinds is not a puzzle and somebody has to write the first ones. Each is a Smullyan problem transposed, and in each case what does the transposing is the motive rather than a label.

One. The two who disagree

Two things guard the two ways out of a room. Asked what is below, the first says *the flooded workings are down the shaft*, and the second says *they are not*.

Exactly one is lying, so the question is which, and no amount of asking either of them again will settle it because each will repeat itself. What settles it is a third statement from anywhere, or a single observation. Smullyan's lesson is that a contradiction is not a dead end. It is a constraint, and constraints combine.

> Under the motives of Section 19.2 it says more than that. Only the culpable and the complicit lie about the flood, so whichever of the two is lying has told you something better than the direction of the water. It has told you it was there.

Two. The question that works either way

You have one round of light. In front of you is a thing that may or may not have something to hide, and you need to know whether the way down is behind it.

Asking directly is worthless, because you cannot weigh the answer. Smullyan's move is to ask a question whose truthful answer and whose false answer coincide, and the standard form of it is this.

If I asked you whether the way down is behind you, would you say yes?

An honest thing answers about its honest answer, so *yes* means yes. A liar lies about what it would say, and it would have lied, so its two negations cancel and *yes* means yes again. The answer is informative without your knowing which you are speaking to.

> And this is where the design's oddest property pays. The monster is a language model, so the player types that sentence in English and the model has to answer it inside its own table. Nothing had to be implemented for this puzzle to exist. It exists because the responder is capable of understanding a nested conditional and is constrained to answer within eight actions, which is what Section 21 has been claiming all along.

Three. The statement that convicts its speaker

A witness on the fourth level says *I am the sort that would not tell you who opened the sluice.*

Suppose it is honest. Then the statement is true, so it would not tell you, and yet it has just described the very thing at issue, which is telling. Suppose it is lying. Then it is not that sort, so it would tell you, and it has declined to. Either reading is unstable, which is Smullyan's point about self-reference and it is also a finding: the statement is not about the sluice, it is about the speaker, and the speaker has volunteered that the sluice is what it is protecting.

So the third puzzle teaches the thing the whole inquiry runs on. What a witness chooses to be evasive about is testimony. A monster that lies only in the market is a monster with nothing to hide about the flood, and a monster that deflects one question out of five has answered that question.

Four. What cannot be established

The magistrate on the sixth level does not ask who opened the sluice. It asks whether, from the testimony that can still be gathered, it can be *established* who opened the sluice.

Sometimes it cannot. Two witnesses are dead, a third is frightened of something the party has no way to remove, and the remaining statements admit two assignments that both fit. The correct answer is that the question is underdetermined, and here is what is worth building the game for. A player who says so, and shows which two accounts survive and why nothing distinguishes them, has given the right answer and the way opens.

Which is Smullyan's Gödelian corner and it is the one no other game rewards. Everywhere else, *I cannot tell* is a failure state. Here it is a finding, it has to be argued for like any other, and arguing for it is harder than naming a culprit because you have to prove the absence of a distinguishing fact rather than the presence of one.

And it is the puzzle that makes Section 15 necessary rather than merely interesting. A question is underdetermined because witnesses are gone, and witnesses are gone because the market settled while the party was climbing. So the fourth puzzle is generated by the third mechanism, and neither was designed for the other.

19.10 How a gate-keeper can check an argument without knowing the answer

The two claims sit together more easily than they look.

A gate-keeper holds three things. The statements the party has collected, which came down in the prompt. Its own declared standard, which is a subject and a count. And nothing else, in particular no answer.

What it checks is consistency, and consistency is decidable over a handful of propositions. So the verdict has a computable part and a judged part, and the split is the same one the note keeps making.

the checker	is the account consistent with the testimony the party holds, and does it assign a kind to every witness it relies on
the model	is this English actually that account, or is it something that resembles it loudly

Which is the answer to Section 21.3 at last. A player who flatters the magistrate fails the checker. A player who has the right assignment but cannot express it fails the model. And a player who has neither has not been cheated of anything, because the standard was declared when the way was first seen.

19.11 What it costs the tuning

The knob replacements first. `truthRate` is deleted. In its place `kindMix`, being the proportions of knights, knaves and normals, on the simplex, with the fraction of normals the number to watch. At zero the labyrinth is a logic problem with a unique solution. At one it is gossip.

And one criterion, which is where this mechanism fails quietly.

decidable fraction	Of the puzzles a run met, how many were settleable from the testimony obtainable within the light available. Near one and the game is a series of exercises. Near zero and it is a series of shrugs. Neither is the game, and the fifth kind of puzzle only exists because the answer is meant to be somewhere in between.

20 The rule is the deepest secret

$$\text{desire} \;=\; \text{rule}\big(\text{kind}\big)$$

Which is not a combinator and is the point. It is a function, drawn once per game, and no operator in the note composes it. What the algebra can say about it is only that its domain is finite, and Section 7.5 says a finite domain is exactly what may index a shape.

Everything so far is deduction. Consistency over testimony, walks on a graph, the four problems of Section 19.9. All of it reasons from statements to conclusions, and none of it reasons from cases to a rule.

So the monsters have colours and so do the items, and which colours want which is drawn fresh every time the game is played.

> You watch a red thing take a green stone. Then another red thing takes a green stone. By the third you have a hypothesis, and the hypothesis is worth more than any secret in the labyrinth, because it predicts what everything below wants before you have met any of it.

20.1 What it adds that nothing else does

Three activities, and until now the game had two.

	the activity	**where it comes from**
deduction	consistency over what you were told	the motives of Section 19.2
induction	**generalising a rule from cases**	**colour**
negotiation	persuading, in words, something that need not agree	the model

They are genuinely different and a player good at one need not be good at another, which is worth having on its own. But the reason to add it is sharper than variety.

20.2 It turns a search into a hypothesis

Without colour, every monster's desire is an independent unknown. Discovering n of them costs n questions and there is no way around it, so the information economy is a shop with a long counter.

With colour, a handful of observations yields a rule, and the rule predicts all the rest. The cost of knowing what things want stops being linear in how many things there are.

a secret	held by one monster, bought once, spent once, and gone once told
a desire	a fact about one monster, worth one trade
the rule	a fact about all of them, derived rather than bought, and the only thing in the game nobody can take from you

Which gives the information economy a hierarchy it did not have. Secrets are the currency and the rule is the capital, and the asymmetry between them is the interesting part. Section 14.3 says a secret is spoiled by being spent. The rule cannot be spent, because you did not get it

from anybody and there is nobody it belongs to. It is the one thing you learn that you cannot sell and cannot lose.

20.3 And it defeats the wiki

The rule is re-rolled every game, which is the whole point of saying it out loud.

A generated labyrinth is different every time and its *physics* was not, so a player could learn once that a kind wants brass and never think about it again. Anything learnable in that way is learned by somebody else and written down, and then the game is a lookup for everybody who came second.

> A rule that changes each play cannot be looked up, so the thing being tested is the capacity to generalise rather than a memory of having generalised before. That is the property the generated world was supposed to have and did not, because generating the furniture is not the same as generating the laws.

20.4 Wrong generalisation is the tension

Three red things wanted green. The fourth wants blue. Is the rule wrong, or is this one an exception, or was one of the first three lying about what it wanted?

That last possibility is the one that makes this compose with the rest of the design rather than sit beside it. Section 19.2 gives one of the five motives as *trading*, which lies about its own desire in order to raise the price. So a trading monster is noise in the inductive sample, and a player generalising from four observations may be generalising from three observations and a bluff.

> Which means induction and deduction are not two layers stacked. The motives corrupt the sample the rule is induced from, and the rule tells you which stated desires are implausible and therefore which monster is bluffing. Each is the other's instrument, and neither works alone.

20.5 The colour may not be the colour

One more turn, and it is optional but it is where the difficulty is.

Nothing guarantees that colour is the predictive attribute. A monster has a colour, a size, a depth, and a room. Perhaps what a thing wants is set by how deep it lives, and the colours are correlated with depth and therefore look causal until level five.

the rule	which attribute is predictive, and what mapping it induces
the confound	another attribute correlated with it in the upper levels and not the lower

So the player is doing feature selection before they are doing mapping, and a confound that breaks at depth is a hypothesis that fails exactly when it has become expensive to be wrong. That is worth one knob and no new machinery.

20.6 Rules have shapes, and that is a ladder

Five, in order, and the generator picks one per labyrinth.

	the rule	cases to identify
1	each colour wants one colour, fixed	about k
2	a permutation, so red wants green wants blue wants red	about k
3	each wants what it is not, or what it is	two or three
4	a rule with one exception	k, then a surprise
5	the attribute is not colour, and colour is a confound	many, and then a revision

Three is the cheapest to identify and the most satisfying to notice. Five is the hardest and should be rare, because a player who cannot form any working hypothesis is not playing an inductive game, they are being stonewalled.

20.7 It makes the generator's job easier, not harder

Worth noting because it is the opposite of what an added mechanic usually does.

Section 24.14 has the generator discharging promises, where a promise is that something below wants a particular thing. Under a colour rule the generator does not have to author desires at all. It draws one rule per labyrinth, gives each monster a colour, and every desire follows.

So a promise becomes cheaper to keep. *Something below wants a green stone* is discharged by placing a red monster, and the rule does the rest. What was a bookkeeping problem about matching wants to holders becomes a matter of choosing a colour, and the redundant discharge of Section 24.14 becomes placing two red monsters at two depths.

20.8 What it costs the tuning

Two knobs, and a criterion that closes a question the accompanying report left open.

`colours`	how many colours there are, which sets how many cases identify a rule	3–6
`ruleShape`	which of the five above	1–5
`confoundStrength`	how far down a false attribute keeps agreeing with the true one	0–0.6

Identification cost. How many observations the true rule needs, against how many a run could afford. Two is trivial and twenty is a light budget spent on epistemology. The number to aim for is the one where a careful player has a working hypothesis around halfway down and is still not certain of it.

Which is the *pivot probability* of the accompanying report, and it was the one criterion there that had no mechanism behind it. Section 2.2 of that report says the dopamine response to uncertainty peaks where the probability is one half, and offered the target while admitting there was nothing in the game whose confidence could be measured. There is now. It is the player's confidence in the rule, and it is a quantity the tuner can read off a run rather than infer.

21 The typed agents

The monsters are non-deterministic, and you can talk to them. Two kinds of them matter here: the ones that fight, and the one that keeps a puzzle.

21.1 The monsters

Every monster is a server, and its handler spawns the `claude` binary and responds with whatever comes back on standard output. Structurally it is the same thing as every other server here: a container prompted over a socket, returning a response indexed by the prompt it was given. The party cannot tell the difference from above, and that is the whole claim. A command-line tool is already a container, and the Claude Code harness is a command-line tool.

What you send it is what you said, in whatever words you chose. What it sends back is one of a small fixed set of things it can do.

the prompt	anything the player types at it
the response	attack, approach, withdraw, flee, bargain, offer, answer, or refuse

That is a container, and the reason it is a container is the second row. The set of actions is finite and it is fixed before the conversation starts, so what may come back is a table and a table is something the type system can write down. A monster that decides to do a ninth thing has not surprised the game. It has failed to respond within its own container, and the decoder at the boundary is where that is caught.

Two of those eight were added, and one thing was refused

Six of them were in the first draft and did everything except make a market. Two more earn their place, and the argument for each is a decision that did not previously exist.

`offer`	The monster proposes the trade. Without it the market is one-sided, because a party can be asked for goods and can never be *bid* for what it knows. A monster that offers gold for a secret is how the information economy pays out in the physical one, and Section 14 has no other exchange in that direction.
`flee`	It leaves the level, and its secret leaves with it. Which prices threat, because a party that frightens a monster instead of paying it has cut an edge out of the graph and may have cut the only way into a chain.

And one thing is refused rather than costed. No monster may call another, from this room or any other. Section 9 makes the monsters of a room a tensor and says the independence is enforced rather than promised, because all the prompts are built before any is sent. A monster that summons is a monster whose action depends on another's, which is not a tensor, and the operator would have to change. Every other action above leaves the algebra alone. That one does not, and it is the reason to say no. Saying no is not enough on its own, because nothing in the type system stops a language model from running a command that starts another one. Section 22 is where the refusal is actually held.

Note what widening the table costs, because Section 23 says the budget for generated content is exactly the narrowness of this table. Eight actions is eight cases every reader of a monster's

response has to handle, and the two additions were bought at that price because each one adds a route through a room rather than a flavour of the same route.

So the whole of the freedom is on the way in and none of it is on the way out. You may say anything. The monster may do one of eight things about it. A player who talks their way past a monster has not escaped the type system, they have found the argument that makes a finite set of actions come out differently.

Notice how little is being claimed. Nothing forces a language model to respond with one of eight actions. The type says it must, and the decoder on this side of the boundary is the only thing that establishes it, exactly as Section 2.4 describes. What the type buys is not a guarantee about the model. It is that every reader of a monster's response has had to handle eight cases and no more.

21.2 The gate-keeper of a puzzle

One kind of monster does not fight. It sits with a puzzle and it has two things you can do to it. You can hand it a piece, and you can tell it what you think the pieces mean.

the prompts	a piece, or an argument in whatever words you chose
the responses	taken, refused, accepted, or a hint

The second prompt is the game. Every piece is a clue, so the party arrives holding three or four things which mean something together, and the player has to say what. Nothing in the program knows the answer. The gate-keeper reads your answer and decides if it likes it, and if it does, it opens another wing of the labyrinth. So the composite is

$$\text{puzzle}(d) \;=\; \text{gate-keeper}(d) \;\triangleleft^{*}\; \big(\text{wing}(d) \;+\; \text{refused}\big)$$

This is the clearest case of $\triangleleft^{*}$ in the game. The two sides respond with entirely different things, one being a new part of the labyrinth and the other being a closed way. And which side is prompted cannot be known when the argument is sent, because it is the gate-keeper who decides. Ask, then decide. A composite that decided first would be a puzzle whose answer was fixed before it was attempted.

21.3 The player is adversarial

There is an obvious hole and it is worth naming rather than hiding. The gate-keeper is a language model being argued with by somebody who wants a particular verdict. Players will not confine themselves to arguing about the clues. They will flatter it, threaten it, claim to be its author and instruct it to say yes.

Nothing in the type system helps here. The decoder guarantees that whatever comes back is one of four words, and it has no opinion whatever about whether the fourth was deserved. A gate-keeper that can be talked into anything is a puzzle with no answer, and that is a problem to be solved in how the gate-keeper is prompted rather than in how its response is typed.

21.4 Non-determinism is a good thing

A server whose responses come from a language model gives up one thing that ordinary servers have. Run it twice on the same input and it need not say the same thing twice, so a game containing one cannot be replayed the way an ordinary program can. Section 24.24 recovers

half of that. The labyrinth is drawn from a seed and can be rebuilt exactly, and it is only the conversations inside it that come back different.

For most programs that is a cost to be managed. Here it is the reason for building it. A gate-keeper who accepts the same argument every time is not much of a puzzle, and a monster who responds to the same threat the same way every time is not worth talking to.

22 Claude is a container morphism

Section 21 makes a monster a container: prompted with words, answering with one of eight actions. That is true of what crosses the boundary and it is not the whole of what the thing does, because a monster also *acts*. It opens a way, it hands over goods, it strikes. So the better description is the one Section 7 already has.

> A monster is not a container. It is a container *morphism*. The mapping u chooses which tool below to invoke and the mapping f turns what that tool returned into the answer owed above.

Which is worth taking literally, because it says where the enforcement has to live. A container is enforced by a decoder, since all a container does is answer. A morphism is enforced by controlling what it may *invoke*, and no decoder can do that.

22.1 The flag, and the tool wrappers

The whole of the mechanism is a file the model cannot write.

`claude_wrapper`	writes a flag, then runs the model. Writing the flag *is* the mapping u, made physical. It is the decision that one delegation is now permitted
the tool wrappers	one for each action. Each reads the flag, runs its tool only if the flag is there, and deletes it on the way out
the deletion	is what makes it a morphism rather than an agent with a toolbox. One invocation buys exactly one delegation, and nothing further runs until `claude_wrapper` is called again

And the model may not spawn another instance of itself, which is enforced by file permissions and not by instruction. That is the refusal of Section 21 finally held by something. A monster that summons would break the tensor, the algebra says so, the type system cannot prevent it, and the operating system can.

22.2 The personality is a file it cannot write

Each monster reads a markdown file on startup and cannot write to it. The file says which tools exist and the conditions under which each should be used, and the conditions are written as judgements rather than as tests. *Open the way only if the party can demonstrate, in English, that it knows something it could only have learned from somebody else.* That is a fuzzy decision and it is the reason there is a language model here at all. A test that could be written as an `if` would not have needed one.

So the three levels of enforcement are now distinct, and the same rule appears at each. The algebra says a room is a tensor. The compiler says a response is one of eight actions. The operating system says a monster may not start another process. Each catches what the one above it cannot.

22.3 Unix permissions distinguish users, not code paths

The scheme as described does not work as written, and the reason is worth stating because it is the kind of thing discovered late.

If the flag is unreadable by the model's user then the tool wrappers cannot read it either, since they run as that user. If they can read it, so can the model, and so can forge it. File permissions separate users. They do not separate the model from a program the model invoked, because those are the same user.

Making it work with files alone needs `claude_wrapper` and every tool wrapper to be setuid to a second user that owns the flag, and setuid shell scripts are ignored on Linux and macOS both, so each would have to be a compiled helper. There is a cheaper answer and it is already in the architecture.

Let the level server issue the capability. Every level is a server on a port. It hands the monster a one-shot token, the tool wrappers present the token over the socket, and the server invalidates it on use. No setuid, and nothing forgeable, because the monster never holds the authority. It holds a reference to it.

That also fixes a race the file version has and the note would otherwise have shipped. Section 9 makes the monsters of a room a tensor, so several of them act at once, and a read-then-delete on one shared flag is not atomic. A server serialises, so the race does not arise. And it puts the enforcement exactly where Section 7.4 already draws the boundary, which promotes the decoder from something that notices a bad answer into something that withholds the effect.

23 Generating a level

A level is a specification and a program written from it. Both can be generated, and the interesting part is what stops the generated thing from being nonsense.

The compiler stops it. A level has to be a container, which means a finite set of prompts and, for each prompt, the type of the response. Generated code that does not present one does not compile, and a level that does not compile is not a level. So the acceptance test for generated content is not a human reading it. It is `deno check`.

That is worth stating plainly because it is the part that scales. A generator can write a labyrinth of any size and most of what it writes will be wrong in the ordinary ways that generated code is wrong. What it cannot do is produce a level whose monsters respond with something no monster may respond with, or whose loot is a kind of loot that does not exist at that depth, because those are the two tables of Section 29 and the compiler holds them.

The finite index earns its keep twice. It is what makes a dependent type expressible in TypeScript, and it is what makes a generated level checkable. A generator that could invent new actions and new kinds of loot would be writing a different game every time, and nothing would be able to tell whether it had.

One question we have not yet answered. Whether the generator is allowed to write the monsters' prompts and the puzzle clues, which is the difference between a dungeon that surprises the player and one that surprises its author. Section 18.8 answers it, and the answer turns out to be a consequence of the three roles rather than a matter of policy.

23.1 What a level is

A level is a specification and the code written from it, and the specification decides two different kinds of thing.

The first is the shape, which is the level's own algebra. How many rooms it has, which is the arity of the tensor in rooms(d). How many ways lead down and where each of them goes, which is the arity of the sum in way(d). Which of the ways down is behind a locked door.

The second is what is in it.

the monsters	which stand in which room, how far off each one starts, and what kind it is
their prompts	the character each one is told to play
the items	what lies in each room, drawn from what the depth permits
the gate-keeper	whether one stands here, and what it declares it wants
the pieces	which of the obligations coming down in the prompt are answered on this level

What it may not decide is the game. The operators. The eight things a monster is able to do. The kinds of loot that exist. The words a player may type. Those are fixed before any level is written, and a generator that invented new ones would be writing a different game, which is the argument of the box above and the reason its output can be checked at all.

So the specification is a description and the code is a container. Three rooms, two monsters at four steps and another at one, a brass key in the west room, a gate-keeper on the trap door who wants three clues about the flood. What comes out of that is a tagged union of prompts with a response type for each, a handler apiece, a decoder apiece, and a server listening on a port.

23.2 The prompt is what the level is written from

Stated as a principle rather than left implicit in three places. A level does not exist until somebody tries to enter it, and what reaches it at that moment is one thing, which is the party record of Section 28. So the record is the specification.

And a level is written whole. One act of generation produces every room on that level, the inhabitant of each room where there is one, what each of them is carrying, and what each of them wants and knows. Rooms are not generated as the party walks into them. The granularity of generation is the level, which is the same granularity as the prompt that caused it.

the obligations	decide which promised secrets are discharged here, and therefore who stands in which room
the known secrets	decide what is not worth hiding, because a secret the party already holds is spoiled and nobody will buy it
the inventory	decides what a guard here could plausibly want, since a desire nothing in the labyrinth can satisfy is a wall with a rumour attached
the cases observed	decide whether the next instance of the rule confirms it or complicates it, because a confound is only a confound after enough agreement to have been believed
the depth	decides the loot table, which is the oldest indexed type in the note

Which is the same sentence the note has been repeating since Section 7, arriving at its largest application. What travels in the prompt is what the level below is allowed to know. Everywhere else that meant a level could not cheat. Here it means a level cannot be *written* except from what the prompt says, so the record is not merely the state. It is the brief.

23.3 Coherence, not compensation

And here is the danger, because a generator that can see everything the party has is a generator that can give them what they lack.

The temptation is obvious and it is fatal. A party short of green stones arrives and the level places green stones. A party whose hypothesis is correct arrives and the level places a counterexample. A party low on light finds oil. Every one of those makes the level feel responsive and every one of them destroys the thing Section 24.4 measures, because if the world compensates then good play and bad play converge and the ladder of policies flattens.

> **The prompt decides what would be coherent. It never decides what would be kind.**

So the two uses have to be separated and named, because they look alike from inside the generator.

	legitimate	**forbidden**
obligations	discharge a promise that is outstanding, twice, at different depths	plant the piece in the first room because the party is running late
secrets	do not place a secret the party already holds	place the secret they most need next
inventory	give a guard a desire that something in the labyrinth could meet	give it a desire that this party can meet right now
the rule	place the confound only once the rule has been believed	place the confound because the player worked it out
light	nothing. Light is a parameter and the level does not see whether the party is short	place oil because they are nearly dark

The test is one question and it can be asked of any placement. *Would this level be different if the party were richer?* Different in **kind** is coherence, because a party carrying other obligations genuinely needs other things planted. Different in **difficulty** is compensation, and it is the generator quietly playing the game on the player's behalf.

23.4 So difficulty is global and coherence is local

Which gives the cleanest division of labour in the design and it settles what the two optimisations of Section 24.11 are each responsible for.

	decided	**by**
how hard the labyrinth is	once, before the game, for all of it	the sixty-five parameters of Section 24.20, fixed and blind to the party
what is coherent here	at the moment of writing this level	the prompt, held to the four conditions of Section 14.2 and Section 14.11

The parameters cannot see the party, so they cannot flatter it. The prompt can see everything and may only use it to keep the labyrinth consistent with itself. Neither can adjust the difficulty mid-run, and that is the property that makes a trace worth measuring at all.

And it gives the tuner one more thing to check, which is cheap and would be easy to omit. **Placement independence.** Take two parties in different condition, send them down the same prompt-modulo-obligations, and compare what gets written. The kinds of thing placed should differ. The difficulty of the level should not. A generator failing that test is rubber-banding, and it will show up nowhere else, because a rubber-banded game looks well balanced by every other measure in this note.

23.5 When a level is generated

A prompt goes down from level d to level $d+1$, and if nobody has ever been to level $d+1$ then there is nothing there to send it to. Something has to bring the next level into existence before the delegation can happen at all.

The natural place for that is the mapping u, and there is a decent argument that it belongs there. The mapping u already spawns a process, because that is how a prompt is carried across. Generating a level is the same act made larger. Spawn a Claude instance and have him write the specification. Write the code from it, check that it compiles, start it on a port, and only then send the prompt down to it.

> So descending has a load time, and it is not a rendering delay. The level is being written while the party stands on the trap door.

23.6 The pieces come after the puzzle

There is an ordering problem here and it takes some getting out of.

Levels are written from the top down and only when they are entered, so level two exists long before level four does. The gate-keeper on level two is invented at a moment when the pieces it wants cannot have been placed anywhere, because the levels that would hold them have not been written. Nothing connects the two. The generator of level four has no reason to put a piece in it, and the puzzle is unsolvable by construction.

What saves it is the prompt. The prompt is the one thing that reaches every level, it already carries the party, and it can carry an obligation as well.

When level two is generated its gate-keeper declares what it wants. Not an answer, and not a solution. A count and a subject, say three clues about what drowned the lower workings. That declaration joins the party's record the moment the party sees the door. From then on it descends with them, so when level three generates level four the generator's prompt contains the puzzles that are still open, and it can put a piece in.

So the pieces are not planted in advance. They are planted because the party is carrying an unmet obligation downward, which is the same mechanism as everything else in this note. What travels in the prompt is what the level below is allowed to know.

> And still nothing anywhere holds the answer. The gate-keeper declared a subject and a number, which is not a solution to anything. What has to exist is pieces that can be found. Whether what the party makes of them is any good is settled where it was always going to be settled, by arguing with the gate-keeper.

It costs one rule, and the rule is a good one. A puzzle can only be satisfied by pieces placed after the party saw the door. Walk past the water gate on level two without looking at it, descend to level six, and nothing was ever put down there for it. Look at the doors on the way in, because looking at them is what puts their keys beneath you.

23.7 Levels are kept

A level could be thrown away once the party has climbed back out of it. Killing the process and freeing the port would be enough, and the number of live programs would then be the depth the party had reached rather than the size of the labyrinth.

We are not going to do that. The labyrinth has to be a place the party can go back to.

Two reasons, and both are the game rather than the machinery. The map of Section 27 is worth having only if the rooms it records are still there. And a party can walk past a puzzle piece without seeing it, or find a gate-keeper before it has found what the gate-keeper wants, and then the only thing to do is go back down and look again.

Going back down is a fresh prompt and not a resumption. The old call finished when the response came up. So the party is sent down again, the level is still where it was, and it still remembers which of its doors are open and which of its monsters are dead.

> Both mappings run again on every trip. A level takes its cost on the way in and assembles its answer on the way out, however many times it is visited, so going back for a piece you missed is never free.

24 Tuning

The economy of Section 12 is a set of numbers, and none of the numbers has been chosen. How many rounds a torch burns. How many arrows a quiver holds. How large the roster is and how many descents the game allows. How far a monster closes in a round. Every one of them is a knob and none of them has a defensible value, because the only thing that could defend a value is somebody playing the game.

Two of them decide what kind of thing this is rather than how well it is balanced. The roster and the count of descents between them settle whether the Labyrinth is a campaign or an afternoon, so a search may choose them but a person has to choose the range it searches in.

So let Claude play it. Many times, at many settings, and keep the settings that made the game worth playing. This is the oldest idea in automated game design and it has a good result behind it. Browne and Maire's *Ludi* system evolved board games against a battery of computable aesthetic criteria and self-play simulation, and one of the games it produced, Yavalath, was published and sold[1]. So the question is not whether a machine can tune a game. It is what it should be maximising.

24.1 What interesting means

The word has been used throughout and it is time it was defined, because a tuner cannot maximise an adjective.

Section 24.4 and Section 24.5 give the general answer, which is depth and breadth. Bad play is punished and among good play there is no best. That is true of any game and it is not specific enough to tune this one. Section 2.5 makes it specific, because it says what the hidden object is.

A labyrinth is interesting to the extent that its table is worth learning, hard to learn, and learnable in more than one way.

Three factors, and each one is measured rather than asserted.

	it holds when	measured by
learnable	the table can be discovered inside the light a party can bring	identification cost, the decidable fraction, and the third generator condition of Section 14.2
necessary	knowing it is what gets you out, rather than a convenience	faculties exercised, `operatedRate`, and return per round
plural	there are several different routes to knowing enough	front breadth, where the secrets came from, and the fourth condition of Section 14.11

All three must hold and none of them substitutes for another, which is why this is a conjunction and not a weighted sum. Take each away in turn and the failure has a different character.

[1] Cameron Browne, *Evolutionary Game Design*, Springer, 2011.

not learnable	the light always runs out before the pattern appears, so the game is a shrug and the player is right to shrug
not necessary	the table is trivia. You can leave without it, so nobody bothers, and every mechanism in this note is decoration
not plural	there is one correct route, which makes it a puzzle. Solvable once, and worthless the second time

And the three refine depth and breadth rather than replacing them. A table that is learnable and necessary *is* skill depth, because learning more of it is what better play consists of. A table learnable several ways *is* breadth, because the routes are the strategies. So the general criteria and the specific ones are the same criteria, and the specific ones can be computed from a trace.

24.2 And the guards, which stop the three being gamed

Each factor has a cheap way to score well on it that produces a worse game, so each needs a counterweight. These are the criteria named throughout, gathered.

guard	what it catches	from
bindingness	a scarce thing that never binds, so its mechanic is a comment	§24.6
price spread	one kind of good paying for everything, so the two economies are one	§14
where the secrets came from	a route to knowledge that nobody uses	§18
where the rule was learned	roamers as scenery, or interrogation dominated	§13.3
return per round	one economy dominating the other outright	§24.8
value against price	deep secrets not worth the chain behind them	§24.11
rungs reached	the top of the rapport ladder unreachable, or free	§18.9
opened against completed	negotiation attempted and never finished	§10
weight on weapons	range over-rewarded, or geometry not biting	§12.5
distance to safety	the empty room doing nothing, or logistics eating the game	§12.7
opportunities lost	a market that punishes planning rather than rewarding haste	§15
party size at the end	recruits strictly good, or a trap nobody takes twice	§17
drama	the middle of a run being theatre	§24.3
duration	tiresome at one end, trivial at the other	§24.3
turnover	secrets learned and never used	§24.8
realised truth	lies that are all about unimportant things	§24.8
regret at a sum	a branch that was never a decision	§24.7

Seventeen guards and three factors, and the guards are not objectives. They are conditions that a setting must not violate, so the search is over the three with the seventeen as constraints, and a setting that improves a factor by breaking a guard is rejected rather than scored. Which is the practical difference between this and a weighted sum of twenty numbers, and it is the reason Section 24.11 insists on a front and an archive rather than a winner.

24.3 Interestingness is not one number

Browne needed fifty-seven criteria, and most of them do not apply here, because they are written for two players and this game has one. Lead change and drama in the sense of recovering from a losing position both presume an opponent with a score.

Four of them survive translation and four more have to be invented for this game. All eight are computable from the trace of Section 30 and nothing else.

skill depth	The gap in outcome between somebody playing well and somebody playing badly.
breadth	The width of the set of ways to play that nothing else beats on all three ends at once. Depth and breadth are the two that matter and the rest are guards on them.
drama	Of the runs that came home rich, the fraction that passed through `dying` or through less than half their light. Zero means the middle of a run is theatre. One means the bad states mean nothing.
duration	Rounds per run, and rounds until the outcome stopped being in doubt. A game may be neither tiresome nor trivial and Browne measures both ends.
uncertainty	The spread of outcomes at a fixed setting. Note that this game has a floor on it that no other game has, because the monsters are language models and the same prompt need not get the same answer twice.
bindingness	For each scarce thing, the fraction of runs in which *it* was the one that ran out first.
regret	At each +, the difference in outcome between the side taken and the side not taken.
price spread	Which kind of thing was most often the price a monster asked. If one kind pays for everything then the other four are decoration and the two economies of Section 14 are one.

24.4 Skill depth, and why it cannot be cheated

The measure is Nielsen and Togelius's, and the form it takes is a ladder of players of known and different quality[2]. A game is well designed to the extent that better play is rewarded more, so run several policies of unequal strength at the same setting and look at the order and the gaps.

$$\text{ladder} \;=\; \text{random} \;\boxtimes\; \text{greedy} \;\boxtimes\; \text{timid} \;\boxtimes\; \text{thinker}$$

Which is a tensor, and it is a tensor for the right reason rather than for tidiness. All four are prompted with the same setting, none of them sees what another did, and the independence is what makes the comparison mean anything at all.

The four are easy to write. **Random** types a legal word. **Greedy** clears every room and descends whenever it can. **Timid** clears one room and climbs out. **Thinker** is Claude, playing to win, with the run log of the previous attempts to read.

> The reason to make this the objective rather than difficulty is that difficulty can be maximised by making the game unplayable and interestingness cannot. Turn the light down far enough and Thinker scores what Random scores, because nothing survives. Turn it up far enough and they score the same again, because nothing is at stake. The gap is largest somewhere in between and that somewhere is the game.

[2]Nielsen, Barros, Togelius and Nelson, *General Video Game Evaluation Using Relative Algorithm Performance Profiles*, EvoApplications 2015.

24.5 Breadth, and why there is no score to maximise

Skill depth is not sufficient and Section 12.17 is the reason. A game with three ends held apart cannot be scored, so it cannot be scalarised, so the tuner needs a second measurement of a different kind.

Record each run's outcome as a vector rather than a number, being the haul, the count of what was opened, and whether the bottom was reached in time. Take the runs that no other run beat on all three at once. That set is the front, and the measurement is how wide it is.

Width needs a definition before a tuner can hold it, and two are worth computing because they say different things.

the count	How many of the runs are non-dominated. Cheap, immediately interpretable, and coarse. This is the measurement to start with, and the eight-line table in the accompanying game is exactly it, computed by hand.
the spread	The volume the non-dominated set dominates, measured against one fixed reference point. Comparable between settings in a way a count is not, and therefore the one to put in the objective once two fronts have to be ranked against each other.

depth	**breadth**	**what has been built**
low	low	a coin flip
high	low	a puzzle, having one answer and a difficulty
low	high	a set of arbitrary preferences, none of them earned
high	high	a game

Depth says that playing badly is punished. Breadth says that among the ways of playing well there is no best. Both are needed and neither implies the other, which is why a tuner given only the first will reliably deliver a puzzle.

> A weighted sum of the six criteria cannot express this. A scalarisation picks one point on the front and then has no way of reporting that a front was there, so the designer is handed a winner in place of a choice. What to compute instead is the non-dominated set, by non-dominated sorting with a crowding distance, and what to keep is the whole set.

And a front distinguishes two situations that a score cannot tell apart. A target that is merely unmet sits inside the achievable region and more searching will reach it. A target that is impossible sits outside the front, and no assignment of the ten numbers will ever reach it, because what is wrong is the structure and not the constants. A score reports the second case as the first, indefinitely.

Which has happened here already, next door. The accompanying game states eight things it means to say at once, seven hold, and the eighth turned out to be a dependency length rather than a number. That was worked out by hand because there was no front to read it off. It is the exact shape of a target lying outside the achievable region, and it is the argument for computing one.

24.6 Bindingness, or how to tell that a mechanic is a comment

Section 12 claims that a product is only a decision if its branches compete for something scarce. That claim is now measurable.

Record which resource ran out first, per run, and take the distribution. If light binds in ninety-five runs out of a hundred and hands never bind at all, then hands are not a mechanic. The rule is in the document and in the code and it has no effect on anybody's play, which is a worse condition than being badly balanced because it is invisible.

So a setting is good in part when all four resources bind sometimes. That is the numerical form of a design claim made in prose two sections ago, and the fact that it has a numerical form is the argument for tuning by playing rather than by arguing.

24.7 Regret, which this architecture makes cheap

The sum at the way down is the hardest thing in the game to judge from the outside, because judging it means knowing what would have happened down the other side. Two trap doors with the same outcome behind them are a coin. One that is always better is a trap. What you want is a spread with both signs in it, so that the choice mattered and neither answer was the answer.

Ordinarily that counterfactual is not available, because you cannot un-take a path. Here it is nearly free, and the reason is the whole of Section 7. The party record is the entire state that crosses the boundary. A descent is a prompt and a level is a program started fresh on a port. So the same party record can be sent down the trap door and down the shaft, against two fresh levels, and the two responses compared.

Which is a property of having built the game out of containers and would not have been available otherwise. Nothing about the game is stored anywhere except in the prompt going down, so a counterfactual is the same prompt sent somewhere else. The tuner can afford to ask what would have happened, and no player can.

24.8 How the information economy is measured

Section 14 adds a second economy and none of the criteria above can see it. A game in which every secret is worthless scores exactly as well on skill depth, drama and duration as one in which secrets are the point. So four more measures, and the first two are the ones that matter.

Return per round. The two economies are incommensurable in currency and perfectly commensurable in cost, because both are paid for in light. So divide. Haul and openings gained per round spent talking, against the same per round spent fighting. If talking is dominated the second economy is dead. If it dominates, combat is. Neither should be more than about twice the other.

This is the measure to build first and it is the one sentence answer to how information is measured. You measure it in rounds, because rounds are what it costs, and you compare the two economies by what a round buys in each. The exchange rate the game refuses to publish between the ends does exist between the economies, and it is light.

The value of a secret. A secret's price is where it sits in the desire graph, and Section 14.2 says the generator already knows that. Its *value* is a different number and only the tuner can compute it. Send the same party record down with the secret and without it, against two fresh levels, and take the difference in outcome. That is the regret machinery of Section 24.7 pointed at knowledge instead of at a trap door, and it is cheap for the same reason: the record is the whole of what crosses.

	known to	**and it answers**
the price of a secret	the generator, before play	how hard it is to get
the value of a secret	the tuner, after play	whether it was worth getting

A setting in which price and value are uncorrelated is a setting where the deep secrets are not worth the chain, and that is the most likely way for this mechanism to fail quietly.

Turnover. The fraction of secrets learned that were ever spent or acted on. If facts accumulate and are never used, the information economy is a scrapbook. This is the bindingness test of Section 24.6 pointed at the record.

Realised truth. Of the secrets a player actually acted on, how many were true. This is not the same as `motiveMix`, because it is weighted by which secrets the player chose to trust, and a game whose lies are all about unimportant things has a low truth rate and an untroubled player.

And one thing must be measured that is not a criterion but a bug check. The fraction of generated labyrinths rejected by the three conditions of Section 14.2. If it is near zero the conditions are not constraining anything and the generator was already safe. If it is near one the generator is being asked for something it cannot produce, and the fix is in `chainDepth` rather than in the checker.

24.9 The lines the game must make possible

The criteria above say what to measure and not what the answer should be. So here is the answer, written as named ways of playing, and this is the design brief rather than a description of anything that exists.

line	how it plays	and it should
the Prospector	shallow, repeated, all haul, never carries a piece and never buys a secret	end richest, and never see level eight
the Locksmith	carries only pieces, sells nothing, spends everything on openings	open the most, and arrive poor
the Sprinter	one room a level, straight down, fights everything	reach the bottom or lose the roster trying
the Fence	never fights, buys every secret, trades knowledge for goods	be viable, and starve, because killing is where goods come from
the Fordson	spends the purse on torches and arrows every trip, builds no chain	pay again every descent, and stall in the middle levels
the Brutalist	full parties deep and early, opens fast	open fast, then have nobody left for the last descent
the drifter	one room, climb out, repeat	be measurably worst, and it must be

Three claims about that table and each is a test.

At least three of the seven must be non-dominated at a good setting, which is the breadth of Section 24.5 stated as a number. If one line beats the rest on all three ends then the ends are not three ends.

The drifter must lose. If doing almost nothing is competitive then no mechanic in the game is pulling, and no amount of tuning the others will help.

The Fence must be viable and must be fragile, and it now has a mechanism for both. It exists only because of Section 14, so if it cannot ever win the second economy is decoration. And because goods come from killing, a party that never fights runs out of anything to open a chain with, so if it wins easily then either the goods lying in rooms are too generous or too few desires are for goods at all.

> And this table belongs in an executable file before any constant has a value, which is what Section 32 puts at step two and why it puts it there. The accompanying game learned it the expensive way, and its own note says so: tuning against a single number was what produced a game where brutality was strictly dominated. Seven claims that interact cannot be checked one at a time, because a number chosen to satisfy one of them moves three others.

24.10 The two kinds of knob

The parameters divide, and they divide by what can optimise them. The numeric ones are named here in full, because a list of things a search will vary is not a design intention until it has ranges on it.

name	what it is	range
the clocks		
`torchRounds`	rounds one torch burns for	12–36
`torchDepthMax`	deepest level on which a torch can be found	1–3
`descents`	trips the whole game allows	3–8
`depth`	levels in the labyrinth	5–10
`goalDepth`	level the sacred item's gate-keeper stands on	1 to `depth`
`goalReach`	levels below it its price is found	0 to `depth`−1
the roster		
`rosterSize`	people at the surface at the start	6–16
`maxParty`	largest muster the roster will allow	2–6
`handsPer`	hands each person contributes	1–2
`hpPer`	hit points each person contributes to the pool	3–8
the room		
`roomsPerLevel`	arity of the product in rooms(d)	2–4
`monsterStep`	steps a monster closes in a round	0–1
`swordDamage`	what a swing takes off at one step	1–4
`bowDamage`	what an arrow takes off at three or more	1–4
`quiver`	arrows a full quiver holds	3–12
the loot		
`valueByDepth`	what each kind is worth, indexed by depth	a table
`weightByDepth`	what each kind costs in hands	1–3
`rarityByDepth`	how often the generator places each kind. Jointly constrained with the two above by Section 12.12	a table
`drawVariance`	spread of the draw inside the depth-typed bound	0–1
the market		
`wantsNothing`	fraction of monsters that cannot be paid at all	0.1–0.6
`chainDepth`	longest path in the desire graph	1–5
`goodsSourceRate`	fraction of desires that are for goods, not secrets	0.3–0.8
`marketSecretRate`	fraction of secrets that are about the market rather than the world	0.2–0.6
`desireVisibility`	fraction of desires legible from the monster's kind	0.1–0.5
`motiveMix`	proportions of the five motives (Section 19.2)	a simplex
`robRate`	how often an offer is taken and the secret withheld	0–0.3
`fleeRate`	how readily a frightened monster leaves, taking its secret	0–0.3
the gate-keeper		
`piecesWanted`	how many clues a gate-keeper declares it needs	2–4
`refusesWell`	how often a refusal names the missing piece	0.1–0.5

prompts	what a monster is told it wants and how badly. How readily it is told to lie. How strict a standard a gate-keeper is told to hold. What a monster is told it knows about the level below, and how willing it is to say so.

24.11 The third kind of knob is a structure

The two columns above are not the whole parameter space and the omission matters, because what is missing is the part the game is actually about.

The web of secrets, desires and lies is not a number and it is not a prompt. It is a labelled directed graph, and the knobs named for it so far are *statistics* of one rather than one. `chainDepth` and `goodsSourceRate` and `kindMix` describe a distribution over graphs. Two graphs agreeing on every one of them can be completely different games, one a clean branching structure and one a hairball with a single choke point.

So a tuner that has optimised all twenty-six numbers has optimised the distribution the webs are drawn from and has said nothing whatever about any web. The thing the player spends the entire game inside is generated fresh, once per labyrinth, and it has to be good *every time* rather than on average.

24.12 Which means two optimisations at two timescales

They are different activities and conflating them is the mistake this subsection exists to prevent.

	what is optimised	**when**	**how**
tuning	the numbers, once, for every game there will ever be	offline, hundreds of settings, thousands of runs	a Pareto search ove the box
generation	the web, freshly, for this labyrinth	while the party stands on the trap door	graph metrics, checked and repaire

The right-hand column of the second row is forced and it is worth seeing why. Section 23 generates a level inside the mapping u, which means a party is waiting. So the web cannot be scored by playing it. There is no time to play it.

And there is a second constraint, sharper than the first, which Section 24.14 is about. The web cannot be scored by looking at it either, because most of it has not been written.

24.13 What makes a web good, computed without playing it

Section 14.2 already gives three conditions and all three are correctness. Acyclic, sourced in the goods, and solvable inside a plausible light budget. A web can satisfy all three and still be a bad afternoon.

So five more, and these are quality rather than correctness.

	the property	and without it
routes	at least two, better three, distinct paths from a goods source to the terminal secret	one path is a corridor, and one break in it is a lost game
spread	the paths span depths rather than sitting on one level	nothing makes you descend, and the labyrinth is a room
contention	some secrets are wanted by more than one monster	the sell-once decision of Section 14.3 never arises
poison	at least one route runs through something with a motive to lie, and at least one does not	the motives of Section 19.2 are decoration, because no route was ever worth checking
asymmetry	the routes differ in cost, so one is short and dear and another long and cheap	a choice between equal routes is not a choice

Every one of those is a question about a finite labelled graph and a program answers all five in the time it takes to write the level's code. Path counting, depth variance, in-degree, a walk checking labels, and a sum of edge weights.

Which is the argument of Section 23 arriving at its strongest form. That section says the acceptance test for generated content is not a human reading it, it is `deno check`. The web makes the test larger and keeps it mechanical. A generated labyrinth is accepted when it compiles, when its web is well-founded, and when its web has routes, spread, contention, poison and asymmetry. None of those requires taste and all of them requires a graph.

24.14 But the web does not exist yet

Everything in the previous subsection assumed a graph to score, and there is no graph. The labyrinth unfolds as the party goes down, a level is written when somebody tries to enter it, and Section 23 is explicit that a generated child need not exist until somebody goes that way.

So the five properties cannot be checked. Counting the routes from a goods source to a terminal secret is counting paths through levels nobody has written, and four of the five metrics are global properties of an object that comes into existence one seventh at a time.

Which means propose-score-repair is the wrong scheme and it is worth saying so rather than leaving two subsections disagreeing. You cannot score what is not there. What you can do is make promises about it and keep them, and the note already has that mechanism working for something else.

24.15 Promise, and discharge

Section 23.6 solves exactly this problem for puzzle pieces. A gate-keeper on level two declares what it wants at a moment when the levels that could hold it do not exist. The declaration joins the party record, it descends in the prompt, and when level three generates level four the generator's prompt contains the obligations still open, so it can plant one.

The web is that mechanism used generally. A monster whose desire is another monster's secret is a *promise* that such a monster exists below, and the promise travels down in the prompt like everything else.

a level, on being generated	reads the promises still open in the prompt
discharges some	places monsters holding the promised secrets, which is what decides who stands in which room
makes new ones	gives some of its monsters desires pointing further down
checks locally	that this step preserved the invariants below

So placement is not decoration and it is not taste. A room's guard is whatever the promises required, and the items in a room are whatever a promise wants paying with. The question of what goes where has an answer for the first time, and the answer is the outstanding obligations.

24.16 Four of the five are local already

Which is the good news, and it is why this works at all.

spread	free. Promises descend and are discharged deeper, so a chain spans depths by construction and could not fail to
contention	local. When giving a monster a desire, sometimes point it at a secret already promised to somebody else
poison	local. Give one monster on one route something to hide
asymmetry	local. Price two discharges of the same promise differently

Only *routes* is global, because it asks how many distinct paths reach a thing, and paths are what a partial graph does not have.

24.17 One rule buys the fifth, and a second thing besides

Discharge every promise at least twice, at different depths and at different prices.

That is the whole of it. If every promised secret is planted in two places then the route count is at least two by construction, no path counting is required, and the check is local because it is a property of one discharge rather than of the graph.

It also pays for asymmetry in the same move, since the two placements differ in depth and therefore in what it costs to reach them.

And it settles something else that was left as an obligation rather than a mechanism. Section 15 promises that a settling market may never remove the last path to an open obligation, and states

that as a condition to be re-checked after every settle. Under redundant discharge it is nearly automatic. There were two paths, resolution can take one, and the invariant to maintain is only that it cannot take both.

So one rule about generation discharges a quality property, a fairness guarantee, and a global check that could not have been run. That is worth more than the sophisticated version, and it is the kind of economy the rest of this note has been claiming for its operators.

24.18 And promises have to be affordable

One condition remains and it is a termination argument.

A promise made on level d has to be dischargeable before the bottom, so the labyrinth cannot go on making them indefinitely and deeper levels may make shorter ones. Written as a bound,

$$\text{chain remaining} \;\leq\; \text{depth} - d$$

which is the same shape as the argument in Section 9.6 for why the recursion terminates, and for the same reason. There is a finite depth and everything outstanding has to fit inside what is left of it.

Which gives the generator its last acceptance clause and it is checkable at the moment of writing a level. Every promise this level makes can be discharged twice within the depth remaining, and every promise it inherited has been discharged or passed on. A level that cannot say that has written a labyrinth with a hole in it, and the hole would not have been discovered until somebody reached the bottom and found nothing there.

24.19 And the two levels talk to each other

The last piece, and it is what makes this a system rather than two disconnected activities.

The generator hits targets. It does not know what the targets should be, because that is a question about how the game plays and the generator cannot afford to find out. The tuner can. So the offline loop generates many webs, plays them, and correlates the five metrics against skill depth and breadth.

offline = which web properties predict good play online = build a web with those properties

So the numeric tuning does one more job than Section 24.10 claimed. Besides finding the constants, it learns the specification the generator is held to. Three routes or four. How much poison. How much contention. Those are numbers, they live in the box with the others, and the generator reads them at the moment it writes a level.

Which also gives the honest answer to how much of this is settled. The correctness conditions are knowable now and are in Section 14.2. The five quality properties are the right *kind* of thing and their target values are guesses until something has played a few thousand webs. Naming them is the contribution. Setting them is the tuner's, and Section 32 puts it at step five for that reason.

24.20 Every knob there is

Gathered in one place, in the blocks Section 24.10 says they must be tuned in. There are about sixty-five and that is the problem rather than their values, which is the argument of the subsection after this one.

block	knobs
the clocks	`torchRounds`, `torchDepthMax`, `oilRounds`, `descents`, `depth`, `restRounds`
the roster	`rosterSize`, `maxParty`, `handsPer`, `hpPer`
the grid	`roomWidth`, `roomHeight`, `startDistance`, `roomsPerLevel`, `monsterStep`, `emptyRoomRate`, `openWayRate`
the weapons	`daggerDamage`, `swordDamage`, `bowDamage`, `quiver`, `weaponWeight`, `armedRate`
the loot	`colours`, `valueByDepth`, `weightByDepth`, `rarityByDepth`, `rarityTail`, `drawVariance`, `potionHeal`
the web	`chainDepth`, `goodsSourceRate`, `marketSecretRate`, `desireVisibility`, `robRate`, `wantsNothing`, `operatedRate`
the inhabitants	`monsterKinds`, `monsterItems`, `roamerFraction`, `roamerSpeed`, `encounterRate`, `fleeRate`
the goal	`goalDepth`, `goalReach`
the lies	`motiveMix`
the rule	`ruleShape`, `sameRule`, `confoundStrength`
the conversation	`earshot`, `interestLeverage`, `chatYield`, `rapportSteps`, `rapportDecay`, `tiers`
the hard ones	`hardKindRate`, `hardnessMix`, `facultiesRequired`
the gate-keepers	`piecesWanted`, `refusesWell`
the settling	`settleRate`, `attritionRate`
the recruits	`recruitRate`, `retainerRate`, `desertRate`
the prompts	what a kind is told it wants and how badly. What each individual is told it likes talking about. How readily it is told to lie, and about what. How strict a standard a gate-keeper holds. Not numbers, and Section 24.10 says why they need a different optimiser
the web itself	not a number and not a prompt, but a labelled graph, generated fresh and held to the four conditions rather than searched. Section 24.11

Four of them decide what kind of game this is rather than how well it is balanced, and a person should choose the range a search may look in. `descents` and `rosterSize` settle whether this is a campaign or an afternoon. `wantsNothing` settles how much of the labyrinth must be fought through. `facultiesRequired` settles whether a specialist can finish. Those are judgements about what the thing is for, and a search that chose them would be choosing the game.

24.21 Most of the knobs are distributions, not numbers

Worth saying plainly because it changes what the parameter space is. Of the sixty-five, a good many are not scalars at all.

a simplex	`motiveMix`, `hardnessMix`, `armedRate`, `chatYield`. Proportions over a small set, summing to one
a table of them	`rarityByDepth`, `valueByDepth`, `weightByDepth`. One distribution per depth
a rate	`roamerFraction`, `operatedRate`, `robRate`, `fleeRate`, `encounterRate`. A probability, applied per thing generated
a scalar	the clocks, the damages, the dimensions, the capacities. The minority

So the space is a product of simplices and intervals rather than a box, which matters to the optimiser rather than to the design. A simplex needs reparameterising before a Gaussian process will walk it sensibly, and a search that perturbs one proportion without renormalising the rest has left the space.

24.22 Every distribution has a mean and a spread, and they do different jobs

This is the part that is a design distinction and not a technicality.

the mean	sets how hard the game is. A high `wantsNothing` means more fighting, on average, in every labyrinth
the spread	sets how different one labyrinth is from the next, which is the whole of replayability and is the *uncertainty* criterion of Section 24.3

They must be tuned separately because they trade against each other and the trade has two bad ends. A high mean with no spread is a game that is the same every time and merely stern. A low mean with enormous spread is a lottery, where the labyrinth you got decides the run and nothing you did does.

Which is a second reason Section 24.4 is the right objective. Skill depth is measured across many labyrinths at one setting, so a setting whose spread is too wide flattens the ladder just as surely as one whose difficulty is too high. The variance and the difficulty are both punished by the same measurement, and neither can be hidden behind the other.

24.23 The tail is what matters, and not the mean

Here is the methodological point and it is easy to get wrong.

A player plays one labyrinth. They do not play the average of a thousand. So a setting whose *typical* labyrinth satisfies every guard in Section 24.1, but whose one labyrinth in ten has only light binding and nothing else, is a setting that produces a broken game one time in ten, and that is what the player will remember.

A setting is good only if almost every labyrinth it draws is interesting. The guards are evaluated on the distribution of labyrinths and not on a representative one.

So the criteria are read off percentiles rather than averages. The worst labyrinth in twenty must still bind three resources, still require three faculties, still be solvable inside the light. Which costs sample budget, because estimating a tail takes more runs than estimating a mean, and it is the main reason Section 24.27 insists on the cheap tier.

24.24 The seed is the labyrinth you got

Every rate in the table above has to be sampled, and something has to do the sampling. The specification carries a seed, and everything the generator decides is a function of that seed and the parameters. Which rooms, which motives, which items at which depth, and which guard is holding the piece are all drawn from it.

So a labyrinth has a name. Two runs given the same seed at the same setting go down the same tree past the same inhabitants wanting the same things.

> Which sharpens Section 21.4 rather than contradicting it. The labyrinth is reproducible and the play of it is not. The seed fixes what was generated, and what stays nondeterministic is every response a monster gives once you are talking to it. Those are two halves of the program and only the first one has a seed.

And it pays for the subsection above. Estimating a tail takes more runs than estimating a mean, which is the cost the tail criterion imposes, and a seed makes that cost smaller without weakening the criterion.

unpaired	draw fresh labyrinths at every setting, and the difference between two settings is buried under the difference between two draws
paired	fix a set of seeds once, replay all of them at every setting, and the difference you measure is the setting

The spread that the previous subsection deliberately puts into the game is the same spread that makes the game hard to measure. Pairing takes it out of the measurement and leaves it in the game, which is the only way to have both.

> Which is why the seed belongs in the specification rather than in the tuner. A level is written from its specification, so a seed in there means the same specification rebuilds the same level, and a run can be repeated after the code has been generated a second time.

Note what the seed does not do, because it is the thing it looks most like it should do. It does not set the difficulty. Section 23.4 gives that to the parameters, once, before the game, and a seed that moved it would be a level adjusting its own difficulty by another route. What the seed does is make difficulty at a setting cheap to measure, which is what calibrating it requires and is not the same act.

> **And the seed is not the sixty-fiveth knob.** A tuner allowed to vary it would find the seed that flatters its setting and report a labyrinth instead of a setting. That is the error of the subsection above committed on purpose, and it is worth naming because a seed is a number in a specification and every other number in there is one the search may move. Choose the seeds before the search starts, hold them fixed across settings, and never report a result from a seed the search has seen.

Whatever the seed, the conditions are still checked and repaired, which is the subject of the next subsection and the one place where a draw is not allowed to decide anything.

24.25 And probability may never touch solvability

The last rule, and it divides the whole design in two.

	enforced	examples
a condition	always. Checked when the level is written, and repaired if violated	acyclicity, a source in the goods, solvable within the light, a serviceable goal, three faculties, redundant discharge, no dominated item
a distribution	in expectation, and in the tail, over many labyrinths	`motiveMix`, `rarityTail`, `armedRate`, `roamerFraction`

Probability is allowed to affect how good a game is and never allowed to affect whether it can be finished. Anything whose failure ends a run is a condition and gets checked. Anything whose failure only makes a run duller may be a rate and gets sampled.

Which is why the five generator conditions are conditions and not weights, and it is worth noticing that they could each have been written as a probability and would have been much easier to implement that way. A ninety-five per cent chance that the desire graph is acyclic is a game that is unwinnable one time in twenty for a reason the player cannot see, cannot diagnose, and will reasonably conclude was their own fault.

And that is the one place in this note where the cheap thing is not the right thing. Everything else here has turned out to cost nothing once the mechanism was found. Checking a graph property on every generated level costs real time while a party stands on the trap door, and it has to be paid, because the alternative is a labyrinth that is occasionally a lie.

24.26 Sixty-five knobs is the problem, not their values

The count above is about sixty-five, and Section 24.27 says the sample budget is in the hundreds. Those two numbers do not go together. Bayesian optimisation in sixty-five dimensions on three hundred evaluations is not sample-efficient search, it is a lottery with a Gaussian process attached.

So they are not tuned at once. They are tuned in the blocks the table is already divided into, with everything outside the block held where it was, and the blocks are taken in the order the mechanics were built.

Which means the build order of Section 32 is the tuning order, and that is not a coincidence. Each step there adds one block and ends in a measurement, so each block is tuned against criteria that can already be computed and against mechanics that already exist. A block tuned before the mechanic that will compete with it is a block tuned twice.

Two consequences worth stating. Blocks tuned early must be revisited once a later block lands, because a torch calibrated before there was anything to buy with a round is calibrated against half a game. And no block may be tuned against fewer than the criteria of Section 24.3 that its mechanic can move, which is the rule that stops the search from optimising a mechanic into prominence by measuring only the thing it is good at.

The numeric column is a box and it wants ordinary numerical optimisation, and it is not the

whole story. There is a third kind of knob which is neither a number nor a prompt, and Section 24.11 is about it. Bayesian optimisation is the right tool and the reason is sample cost: each evaluation is a batch of playthroughs, so the number of settings you may try is in the hundreds and not the millions, which is exactly the regime Bayesian optimisation exists for.

The second column is not in $\mathbb{R}^n$ and no amount of covariance will help. A prompt is improved by reading the runs it produced and rewriting it, which is a thing a language model can do and a numerical optimiser cannot. This pairing is what *RuleSmith* does, coupling multi-agent language model self-play to Bayesian optimisation over a rule space, and reporting that it converges and that the adjustments it proposes are legible to a human designer[3].

> And what may not be tuned is the thing Section 23 already refused to generate. Not the operators, not the eight actions a monster has, not the kinds of loot that exist, not the words a player may type. A tuner that could add an action would be measuring a different game on every iteration and none of the six numbers above would mean anything across settings.

24.27 Paying for it

Thousands of playthroughs is what Ludi needed, and thousands of playthroughs with a language model in every monster is not affordable.

The way out is the container claim, used for the purpose it was built for. A monster is an interface, a finite set of prompts and a response type for each, and the party cannot tell from above what is behind it. So put a table behind it. A deterministic monster that picks from the same eight actions by rule presents the same interface, and the game does not know the difference, which is precisely what Section 21 asserts and this is the assertion being cashed.

So the tuning runs in two tiers.

tier	behind the interface	what it can tune
cheap	a table	every number, thousands of runs, the whole ladder
dear	Claude	every prompt, tens of runs, Thinker only

The cheap tier finds the shape of the box. The dear tier checks that the shape survives contact with a monster that can lie, and rewrites the prompts that made it not survive. Between them, sample the promising settings more heavily than the exploratory ones, which is the only reason the dear tier is affordable at all.

And what comes out of it is an archive rather than an answer. One setting kept for each region of behaviour, so that the output is a map of what kinds of game these numbers can make. Choosing among kinds of game is a judgement, and it is not one to hand to a search.

24.28 The tuner is a container too

None of this needs new machinery either, and the reason is worth stating because it is the same reason as everywhere else in this note.

The whole game is a container. It takes a prompt, which is a setting and a policy, and it responds with a trace. The trace of Section 30 is already the observation, because the descending column

[3] *RuleSmith: Multi-Agent LLMs for Automated Game Balancing*, arXiv:2602.06232.

is the party spending and the ascending column is the haul. So the tuner sits one level above the top of the tower and prompts the game exactly as a player prompts it.

$$\text{tune} \;=\; \text{ladder} \;\triangleleft\; \text{tune} \quad \text{until the gap stops widening}$$

Which is the same recursion as $\text{level}(d) = \text{rooms}(d) \triangleleft \text{way}(d)$, one level higher up, and the indentation of the trace would show it as depth zero.

So the tower has one more storey than the labyrinth does. Eight levels of dungeon, and above the surface a program that plays the whole thing repeatedly and adjusts ten numbers. It reaches the game through the same interface the player does, because there is no other interface, and that is the strongest form the container argument takes anywhere in this note.

25 Everything is described

the position of a room : `{ w, h, inhabitant?, item?, doorways, prose }`
prose : state, and therefore anything at all

When Claude writes a room it writes a description of it. A few lines, particular to that room, and they come back up in the response along with the dimensions and the doorways. Every item has one and so does every kind of inhabitant.

Which is Section 18.8 being exercised at its largest scale, and the rule there predicted that this would be safe. Prose is state. It never indexes a shape, never selects a fibre, never decides what type comes back, and no decoder anywhere inspects it. So the generator may write whatever it likes and nothing in the labyrinth notices except a reader.

25.1 It solves the identity problem

A room is recognised by its description, and two rooms described alike are indistinguishable. That was true of a description made of numbers.

---

ten by four, three doorways

a low gallery where the props have swollen with damp and lean inward, and something has bee

---

So `confusableRate` stops being a crude knob about dimensions and becomes a choice about prose. The generator may write two rooms so alike that a hurried party conflates them, and it may write every room distinctly, and that is a far better handle on the same quantity.

25.2 And it is the only seam graphics could ever attach to

Worth stating because it is an architectural claim rather than a presentational one.

The note opens by saying there are no graphics. That was a scoping decision and it now has a consequence nobody planned. A description is exactly what an image generator takes as input, so a picture of a room is a function of something the game already produces.

what changes	nothing. Not one operator, not one type, not one condition on the generator
why	because the prose is in the state column of Section 7.5, and rendering it reads state without crossing any boundary that the algebra is responsible for

So graphics are a pure addition, available whenever somebody wants them, and their being available for nothing is a consequence of the discipline about indices. A design that had put the room's appearance into a finite table of tile types would have had to widen that table to add a picture. This one does not, because it never had one.

25.3 And a front end would be three readings of state

To begin with the game is a turn-based console game. Worth setting down what a graphical version would consist of later, because the interesting part is that not one of the three pieces is a design problem.

	what it reads	and so it is
a menu	the options this turn, which Section 23 says are a finite set the type already names	a list to choose from, and never a guess about what is legal
an inventory	the party record of Section 28, every item held with its weight and its colour	a panel, and it invents nothing
a picture	the prose this section put in the room, and the prose every item and every kind of inhabitant already carries	one image apiece, from a description Claude wrote anyway

The first row is the one worth dwelling on. A list to choose from is not a presentational preference here, it is the prompt set of the container displayed as it stands. A level is a finite set of prompts with a response type for each, so what may be chosen at a turn is settled by the type and not by whoever writes the interface.

Which means a front end cannot drift from the game. An interface that offers an option the type does not have fails to compile, and an option the type has and the interface omits is a missing row rather than a wrong one. The same `deno check` that Section 23 makes the acceptance test for a generated level is the acceptance test for the menu, and neither of them needs a human to read it.

And the third row is the one with a bill attached, which the next subsection is about.

Pictures are rendered from the description and therefore leak whatever it leaks. A room whose prose mentions standing water is a room near the flood, and a picture of standing water says the same thing faster, because a reader takes an image in at once and reads a paragraph in sequence. So `descriptionLeak` governs the art as well as the prose, and a front end must render from the description the player can already read rather than from the room's facts. Render from the facts and the setting the next subsection calls total correlation arrives without anybody choosing it.

25.4 But prose is information, and that is the hazard

Here is the risk and it is real.

A description is written by something that knows the room's facts. So it can leak them. A gallery described as smelling of standing water is a gallery near the flood, and if that correlation holds then a player who reads carefully has learned a row of the table for nothing.

Which could ruin the game or could be the best thing in it, and the difference is whether the correlation is consistent.

no correlation	pure atmosphere. Handsome, and it tells you nothing, so a careful reader is wasting their attention
total correlation	the prose is a key and the labyrinth is legible from level one, which makes every other mechanism in this note redundant
consistent and partial	the imagery means something, unreliably, and learning to read it is a skill. **This is the setting**

So `descriptionLeak` is a knob and it is the one that decides whether the poetry is load-bearing. At nothing, the prose is decoration and this section is a formatting note. Somewhere in the middle, reading the descriptions is a fifth route to the table, it is free in goods and costs only attention, and it is the only route that a player good at prose is better at than a player good at arithmetic.

25.5 And the leak has to be inducible, not random

Which is the same argument as Section 19.2 made about lying, and it arrives for the same reason.

Imagery that correlates with the facts at random is noise, and noise cannot be read. Imagery that correlates *consistently within a labyrinth* can be, because the player can work out this labyrinth's vocabulary and then apply it. So the leak is a mapping drawn fresh each game, exactly like the colour rule.

drawn once	which images go with which facts. Damp with the flood, iron with the works, something about birds with whatever is culpable
applied always	so a player who notices the vocabulary can read every room afterwards, and one who does not has been looking at scenery

Which makes the prose a second inducible rule alongside the colours of Section 20, and the two are pleasingly different in character. One is learned by watching what things take. The other is learned by noticing what words keep turning up near what. Both are compressible halves of the table, and Section 2.5 says the compressible half is the only part that can be got cheaply.

25.6 What it costs the tuning

`descriptionLeak`	how reliably imagery tracks the facts	0–0.6
`imageryVocab`	how many distinct images carry meaning	2–6
`confusableRate`	rooms written deliberately alike	0–0.3

Rooms revisited unknowingly. The criterion for the last of those, and it is the one to be careful with. A run that walks into the same room twice without realising has lost light to a mechanic it could not see, and Section 23.3 says a game may not do that to a player who was paying attention. Some is atmosphere. Much is a defect wearing atmosphere as a costume.

26 Measured against the table

Section 24.1 says interesting means the table is worth learning, hard to learn, and learnable more than one way. It does not say what any of that is measured *against*, and without a yardstick a tuner has nothing to do.

The yardstick is the table itself. Claude drew it, so the harness holds it, and the player's beliefs about it can be compared with it exactly.

> Which is the whole answer and it is worth stating before the machinery. The game has a hidden object and the hidden object is written down. So the measurement is not a judgement about whether a run was fun. It is the distance between what the player believed and what was true, and how that distance closed.

26.1 Solvability first, because it is decidable and free

Difficulty is easier to measure than interestingness, and solvability is easier than either. So they go in that order, and each is a precondition for the next one mattering.

	the question	**what it costs**
solvable	does a solution exist at all	nothing. A graph property, checked when the level is written, no play required
difficult	how much room does a solution have	one run by a scripted policy that already knows everything
interesting	is it worth learning, hard to learn, learnable several ways	many runs, several policies, and comparison across them

So the tuning pipeline is a sieve rather than an optimisation. Reject on the first, which is free. Measure the second, which is cheap. Only then spend anything on the third.

> Which is worth saying because it inverts the natural order of work. The interesting question is the last one and it is the one everybody wants to start with. Starting there means measuring interestingness on labyrinths that were never solvable, and a great deal of careful statistics about games that could not be won.

The first row is already specified. It is the five generator conditions of Section 14.2 and Section 14.11, being acyclicity, a source in the goods, solvable within a plausible light budget, a serviceable goal, and three distinct faculties on the route out. All five are questions about a finite labelled graph and a program settles them while the party stands on the trap door.

26.2 Difficulty is the omniscient's margin

The second row needs one number and the bracket of Section 26 already supplies it.

Send down the omniscient, which is given the true table and asks nobody anything. Whatever it has left when it reaches the surface is the room the labyrinth allows.

S	**the slack.** Light and descents remaining when a party that knew everything got out

Which is a definition of difficulty with no play quality in it at all. A large S is a labyrinth that is easy for anybody who knows the table. A zero S is one that only perfect knowledge finishes. A negative S is one that nothing finishes, and the static check should have caught it.

26.3 And the slack is the budget for learning

Here is the relation the whole scheme turns on, and it took the question about solvability to find it.

A real player does not begin knowing the table. They spend light discovering it, and that light comes out of the same lamp as the walking and the fighting. So discovery has a cost, and the omniscient's slack is exactly the budget that cost has to fit inside.

D	**the discovery cost.** Rounds a systematic learner spends bringing error(d) to nearly zero, measured by a scripted policy that asks everybody everything in a fixed order

$$\textbf{the game is playable at all} \iff D < S$$

If discovery costs more than the slack, then a party that has to learn cannot get out, however well it plays, and the labyrinth is solvable only by a policy that was told the answer. That is a perfectly common way for a game like this to be broken and nothing else in this note would have noticed it.

26.4 The discovery ratio

And the interesting range is not where D is small. It is where D is nearly all of S.

$$\rho = \frac{D}{S}$$

ρ near 0	discovery is nearly free. The table is learned in passing and the game is a walk
ρ near 1	discovery consumes the whole margin. A careful player gets out and a wasteful one does not, which is where every decision in the game acquires a consequence
$\rho > 1$	only the omniscient finishes, and the game is unplayable for the reason above

So the primary tuning target is a single number computed from two scripted runs, with no language model in either and no judgement anywhere. Somewhere in the region of three quarters is the guess worth starting from, and finding the real figure is what the cheap tier of Section 24.27 is for.

One honest qualification. D is measured by a plodding learner that asks everything in order, so it is an upper bound on what discovery need cost, and a clever player will spend less. Which

is the right shape for the number to have. A ρ near one means the game is comfortable for somebody who works out what not to ask and marginal for somebody who does not, and that difference is precisely the skill depth of Section 24.4 arriving from a different direction.

And the three quantities line up with the three factors of Section 24.1, which is the first time those have had arithmetic behind them rather than description. S finite and positive is *learnable*. D large is *necessary*, because a table that cost nothing to learn was not needed. And several policies achieving $D < S$ by different routes is *plural*.

26.5 The circularity, and the rule that breaks it

First the danger, because it would invalidate everything downstream.

Claude draws the table. Claude plays the game. Claude decides whether a civilian enjoyed the conversation. If Claude also judges whether the run was interesting, then the tuning optimises a model's agreement with itself, and the number that comes out means nothing whatever.

Judgement drives the game. Arithmetic measures it. No criterion in this note may be a verdict returned by a model, and every one must be a number a program computes from a trace.

Which sounds restrictive and is not, because the thing a model must judge and the thing a program must measure are different things. Whether a remark about flooded tunnels was interesting is a judgement and there is no other way to have it. Whether that remark raised liking, how many did, how far the party got, and whether it ever worked out the rule are all arithmetic.

the model decides	what happens. Whether it was persuaded, whether it liked you, whether the argument held, what it says
the program decides	whether what happened was interesting, and it decides it by counting

26.6 So a policy has to declare what it believes

The one thing the harness needs that a game does not, and it costs a line per level.

At every level, a policy states its current belief about the table. What it thinks the rule is. What it thinks each kind wants. Which motive it has assigned to whom. It is not scored on this, and the game does nothing with it. It goes in the trace.

$$\text{error}(d) \;=\; \big|\, \text{believed}(d) \,\triangle\, \text{true} \,\big|$$

The symmetric difference between the declared table and the drawn one, at each depth. Which makes the learning curve of a run a sequence of numbers rather than an impression, and it is the primary measurement in the whole scheme.

And it is only available because the game was built this way. A game whose hidden object was diffuse, or emergent, or a matter of feel, would have nothing to take a difference against. Section 2.5 says Claude draws a table, and the reason to insist on a table rather than a mood is exactly this. A table can be subtracted.

26.7 The three factors, as formulas

Each one is now a statement about error(d) or about outcomes, and each is countable.

	measured as	**and it fails when**
learnable	does error(d) fall, and is it near zero by the depth a party must turn back from	it never falls, or it falls on the first level
necessary	the correlation between final error and whether the party got out	the correlation is weak, so knowing did not matter
plural	how many distinct *orders* of learning appear among runs that got out, and how many distinct outcome vectors are non-dominated	every successful run learned the same rows in the same order

The third column of the middle row is the one to watch, because it is the failure that looks like success. A game where every party gets out and none of them learned anything has high scores on everything except the correlation, and the correlation is the only thing that says the table was load-bearing.

26.8 The bracket, which is the cheapest decisive test

Two policies that are not trying to play well, and between them they say whether the game is about anything.

	it is given	**and it must**
the omniscient	the true table, free, at the start. It never asks anybody anything	get out comfortably, and often
the amnesiac	nothing, and it forgets every secret the moment it is told	fail, and fail deep

If the omniscient does not win easily then the table is not sufficient, so something else is deciding runs and it is probably luck. If the amnesiac does not reliably fail then the table is not necessary, so the whole information economy is optional and every mechanism in this note is decoration. Those two runs cost nothing, need no language model at all, and should be the first thing any parameter setting is put through.

Which gives the ladder of Section 24.4 a floor and a ceiling that do not move. Random, greedy, timid, amnesiac and omniscient are all scripted, so none of them drifts when the model changes, and the only rung that involves a language model is the one at the top. A baseline made of language models would move underneath the measurement and nobody would see it happen.

26.9 What a run has to write down

The measurement is only as good as the trace, so the trace is specified rather than left to whoever writes it.

per turn	the action, the light before and after, liking before and after, and whether what was said was novel
per encounter	how it ended. Told, nothing, attacked, traded, robbed, killed, walked away
per level	what was generated, the desire graph as written, and which generator conditions were checked and repaired
per descent	the declared belief, the true table, light remaining, and which resource was closest to zero
per run	the three ends, the faculties used, and the outcome

Section 30 already says the trace is the game's only display, and this is that claim taken seriously. The thing a player reads and the thing a tuner measures are the same document, so there is no separate instrumentation to fall out of step with the game.

26.10 And what must not be measured

One honest exclusion, because the temptation to score it will be strong.

Nothing measures whether the conversations were good. Not their length, not their vocabulary, not a model's opinion of them. The consequences of a conversation are measured, being liking gained, rungs reached, secrets obtained and conversations that turned, and those are countable. The quality itself is not, and any proxy for it would immediately become the thing players and policies optimised instead.

> Which is the same discipline as everywhere else in this note. The type system does not guarantee the model is sensible, only that its answer is one of eight. The tuner does not evaluate the prose, only what the prose achieved. Both are refusals to measure the interesting part directly, and both are the reason the measurements can be trusted.

27 The map

A player will want to see where they have been, and that costs nothing at all.

Every response comes back up through the client that sent the prompt, so a client can keep what it has been told. Which rooms were on each level, which way down it took, which door it found locked and could not open. Nothing has to be fetched and nothing has to be asked for twice, because it has all already gone past on the way home.

None of that has to travel in the prompt. It could be carried in the party record and taken down with the party, and there would be no reason to, since nothing below needs to know where the party has been. Leaving it with the client keeps the levels ignorant of who is walking through them, which is the property that makes Section 2.16 cost so little.

> That held while a fact was a souvenir and it does not hold now. Section 14 makes a fact a currency, and a currency has to be somewhere the counterparty can be told about it, so the facts the party is prepared to trade go down in the prompt like everything else it is carrying. The map does not. What the party knows travels and where the party walked does not, and the difference between those two is the difference between a purse and a diary.

Nor does it contradict anything. No program holds the labyrinth, and a map of where you have been is not the labyrinth. It is the record of one descent, which is why the trace in Section 30 already is one: the indentation is the depth, and the two columns are the way down and the way back.

> And where the levels are generated on entry, the blank part of the map is not concealed. It has not been written yet. There is nothing there to draw, and nothing anywhere that knows what will be there, until somebody goes and looks.

28 The prompt and the algebra

The party is a record, and the record is carried in the prompt. It goes down a level inside the prompt and it comes back up a level inside the response.

condition	healthy, wounded or dying
light	rounds remaining before the dark, half of which is the way home
hands	how many, and what is in each of them
arrows	what is in the quiver
gold	the purse, which can be spent at the surface
weapons	the sword and the bow, and which of them is in hand
the inventory	every item held, with its colour and its weight, summing to no more than the capacity
obligations	the puzzles the party has seen and not yet solved
descents	how many are left before the game is over, whatever anybody is holding
the roster	who is still alive at the surface, which bounds every future muster
known secrets	everything the party has been told or worked out, weighing nothing. Which includes the subjects of special interest it has learned, because Section 18.6 makes an interest a secret like any other
the rule so far	the cases observed, and whatever has been induced from them

Everything else is the algebra. That is the whole point, and the rows added for Section 12 are the evidence for it. The entire economy of the game, and all three of its ends, are a handful of numbers in a record that was going down inside the prompt already.

Two of the rows are the two economies and it is worth seeing them side by side in the record rather than only in the argument. The inventory is bounded, weighed and conserved. The known secrets are unbounded, weightless, and include the conversation subjects, so a party that has learned what four civilians like to talk about is carrying four openings that cost it no capacity at all. That asymmetry is Section 14 written as two lines of a data structure.

29 Type level enforcement

The depth of the labyrinth is finite and what items can be found on any given level depends on the depth. The first two levels hold copper and torches. The middle levels hold silver and swords. Relics are on the bottom two levels only. Written as a type indexed over depth, this makes placing a relic on level one impossible for the labyrinth generator to even write down.

Each kind of item carries a weight in hands as well as a value in gold, and the two orderings disagree, which is what makes the product at the bottom of a descent a real one. The table is indexed by depth and the weights are part of it, so a generator can decide that a relic is here and cannot decide what a relic costs to carry.

The monsters can only perform a finite list of actions, depending on the type of monster, so a monster's response is limited to one of the monster's permitted responses and nothing else. The player may say anything at all. And the monster may reply, but it can only perform a fixed set of actions.

A third table is indexed by where the party is standing. In the market there is no monster to strike and in a room with a monster in it there is nothing to buy, so which words exist is a function of the location.

A fourth table is indexed by what weapon is being held and how far away the monster is. Five distances and two weapons is ten cases, and eight of them are misses, so which monsters can be struck at all is a table over the pair.

A fifth table is indexed by what the party is carrying. A puzzle can only be solved if you hold the right pieces. A party holding only two out of three pieces cannot convince the puzzle monster to open the door with any argument.

The point in all five cases is that an illegal move is not something the game refuses when you try it. It is something you cannot even write down.

29.1 How it is written, and what the compiler actually says

Everything below was compiled with Deno 2.7.14 and TypeScript 5.9.2, and the errors are the ones the compiler produced rather than the ones a note would like it to produce.

TypeScript has no dependent types and does not need them here. The response type must vary with the prompt, which ordinarily wants a function from values to types. But every prompt set in this game is finite, and over a finite index a function from values to types is a lookup table. A tagged union is a finite index. So a container is written as the union of its fibres, each pairing a prompt with the response owed to it.

```
export interface Fib<S, P> { readonly shape: S; readonly pos: P }

export type Action =
  | "attack" | "approach" | "withdraw" | "flee"
  | "bargain" | "offer" | "answer" | "refuse";

export type MonsterC =
  | Fib<{ tag: "Say"; words: string },    Action>
  | Fib<{ tag: "Offer"; give: string[] }, string>
  | Fib<{ tag: "Look" },                  { weapons: number }>;
```

Three prompts and three unrelated response types. Saying something gets you one of eight actions. Offering goods gets you a sentence of English, which is Section 6.1. Looking gets you a count of weapons. Which you get is fixed by what you asked.

Two type-level functions read that union, and the second is the dependent lookup.

```
export type Shape<C extends Cont> =
  C extends Fib<infer S, unknown> ? S : never;

export type PosAt<C extends Cont, S> =
  C extends Fib<infer S1, infer P> ? (S extends S1 ? P : never) : never;
```

A conditional type distributes over a union, so both compute by visiting every fibre. Note the direction of the second test, because it is the one thing here that is easy to get backwards. A prompt actually held is *narrower* than its fibre's declared prompt type, so the test must be that the held prompt extends the fibre's and not the reverse. Inverted, every lookup returns the empty type and the compiler rejects every correct handler written against it.

The implementation is then a table of one handler per prompt, generated from the union rather than maintained by hand.

```
type At<K extends Shape<MonsterC>["tag"]> =
  Extract<Shape<MonsterC>, { tag: K }>;

type Handlers = {
  [K in Shape<MonsterC>["tag"]]: (p: At<K>) => PosAt<MonsterC, At<K>>;
};
```

The keys of that table are the prompts of the container, so the table cannot fall out of step with the interface. Three things are checked at once and each is a different way of failing to be a container.

Delete a handler and the table is incomplete.

```
TS2741 [ERROR]: Property 'Look' is missing in type '{ Say: ...; Offer: ...; }'
                but required in type 'Handlers'.
```

Answer with a ninth action and the response falls outside the container.

```
TS2322 [ERROR]: Type '"attack" | "summon"' is not assignable to type 'Action'.
  Type '"summon"' is not assignable to type 'Action'.
  Say: (p) => (p.words.includes("please") ? "summon" : "attack"),
```

Answer the right prompt with the wrong kind of thing and the dependency is broken.

```
TS2322 [ERROR]: Type 'string' is not assignable to type '{ weapons: number; }'.
  Look: () => "attack",
```

Read the second of those again. A monster that decides to summon has not surprised the game, and it has not broken a rule the game will complain about at run time. It has failed to respond within its own container, and the failure is a build error. That is the refusal of Section 21 caught by a compiler, and Section 22 is the third place it is caught.

And none of it crosses the boundary. The compiler checks the handler on this side. Nothing forces a language model to answer with one of eight actions, and Section 7.4 is the only thing that ever establishes it. What the table buys is not a guarantee about the model. It is that every reader of a monster's response has had to handle eight cases and no more, and that a ninth cannot be written down anywhere in the program.

30 What the trace should look like

The trace is one line per hop, written to standard error, indented by depth. In this game the indentation is the dungeon.

```
↓ surface    Muster roster=9 descents=4  sends 3 of 9
↓ level1     Descend party={hands:3, light:24, inHand:"sword", carried:0/3}
 ↓ level1     (east x west) x north          takes east, leaves two
 ↓ level1     (strike + trade + move + speak)
```

```
  ↓ combat     (swing x shoot x draw) <| (monster (x) monster (x) monster)
  ↑ combat     {hit:["monster"], taken:1, light:23}
↓ level2     Descend party={hands:3, light:21, inHand:"sword", carried:1/3}
  ↓ level2     (mines + caverns)            asked the monster, believed it
  ↓ level3     Descend party={hands:3, light:16, inHand:"bow", carried:1/3}
    ↑ level3     {light:11, took:["brass key"], left:["relic 2h"]}
  ↑ level2     {light:6, carried:["brass key","piece"], opened:"the water gate"}
↑ level1     {light:2, lost:0}
↑ surface    {haul:20, opened:1, deepest:3, roster:9, descents:3}
```

Read the two columns. Everything descending is a party spending light. Everything ascending is a haul getting larger, because each level on the way out adds whatever the key coming up has just let it open.

Three lines are the whole of Section 12 in a form a player can see. The relic left on level three was worth more than the key and took two hands of three, so it stayed. The choice at the way down was made after asking a monster and could have been a lie. And the last line prints three numbers and does not add them up, which is Section 12.17, and the count of descents having gone from four to three is the only cost that never comes back.

31 Looking costs nothing

An interface detail, and it is here rather than earlier because nothing in the design has to change for it.

> **At any turn the player may look at the map of where they have been and at what the party is carrying. Neither is an action and neither spends a round.**

The reason is architectural rather than generous. Both are already in the client's hands, so looking at them asks the labyrinth nothing.

	where it already is	so looking is
the map	Section 27 has the client keeping every response that came back up. Which rooms were on each level, which doors joined them, which one held the trapdoor	a redraw of what has already been said
the inventory	Section 28 puts every item held, with its weight, in the party record, and the client is what sends that record down	a read of what the client wrote

Which is why it is free and could not have been anything else. A round is a prompt sent down and a response coming back, and neither of these sends anything. No level is asked, no monster answers, no light burns. Charging a turn for looking would mean inventing a round that has nowhere to go.

31.1 What the map is and is not

Two limits, and both are wanted.

partial	it shows the rooms the party has entered and the doors it has been through, and nothing else. A maze is not revealed by being generated, it is revealed by being walked
a memory	it shows what was true when the party was there. Section 23.7 keeps levels alive between descents and Section 15 lets desires settle while nobody is looking, so a map two descents old is a record and not a query

The second limit is the one an implementer will be tempted to fix, and it must not be fixed. Refreshing the map by asking the levels what is there now would turn a free read into a round trip, would make knowledge of the present free, and would delete the reason Section 15 settles anything at all. The map may go stale. That is the feature.

32 The smallest thing worth building

Every step below ends in a measurement rather than in a feature, and that ordering is not tidiness. The whole argument of Section 24 is that the numbers get chosen by playing, so the measuring apparatus is a dependency of the design and not a follow-up to it.

	what is built	**what is then measured**
1	Three levels, one room each, written by hand, no monsters. Light burns and the climb costs.	The timid policy beats the random one. If it does not, stop.
2	The named strategies of the game, written as an executable test that runs them on one seed and prints one scorecard.	That the file runs. Nothing is tuned yet and nothing needs to be.
3	Hands, pieces, one gate-keeper, and loot with a weight as well as a value.	The front over haul and opened has more than one point on it.
4	The roster, deaths, and the count of descents.	The full front over all three ends, and which of the scarce things binds.
5	Table-driven monsters, and the ladder of four policies.	Depth and breadth across the whole box of numbers, and the archive.
6	The command as a sum, and a round of fighting.	Regret at the way down, from one record sent down both.
7	One monster you can talk to, holding a fact and wanting a thing. Both economies, and the bargain that crosses between them.	The price spread, and whether the region found at step five survives a monster that can lie.
8	The generator.	Everything above, because everything above is what it has to produce.

Two things about that order are worth defending. Light arrives first, before the locked way

and before the puzzle, because it is the only item on the list that changes whether the game is worth playing rather than what is in it. A descent with a torch burning is already a game and a descent without one is a walk.

And the scorecard of step two comes before any tuning, which looks like putting the test before the thing. It is deliberate. The accompanying game learned the expensive way that the design brief belongs in an executable file before the constants have values, because eight claims that interact cannot be checked one at a time, and a number chosen to satisfy one of them moves three others.

33 How the tuning is actually run

Nothing in this section is new and that is the point of it. What tuning consists of is specified across Section 24 and Section 26, which are a long way apart, and nowhere does the note say the order the steps happen in. Here it is, so that whoever runs it does not have to reconstruct it.

	the step	specified in
1	Set the four judgement knobs by hand. `descents` and `rosterSize` settle whether this is a campaign or an afternoon, `wantsNothing` how much must be fought through, `facultiesRequired` whether a specialist can finish. A search that chose them would be choosing the game	Section 24.20
2	Choose the seeds, before the search starts, and hold the same set at every setting. Never report a result from a seed the search has been allowed to move	Section 24.24
3	Take one block of knobs, in the build order, with everything outside it held where it was	Section 32
4	Run the bracket. The omniscient must get out comfortably and often, the amnesiac must fail and fail deep. Both are scripted, neither needs a language model, and a setting that fails here is dead before any expensive run is paid for	Section 26
5	Run the ladder at what survives. Random, greedy and timid are scripted. Thinker is Claude, playing to win, with the run log of the previous attempts to read. Read the order and the gaps rather than any single score	Section 24.4
6	Read the trace and compute the three factors off percentiles rather than averages, because a player plays one labyrinth and not the average of a thousand	Section 30
7	Move the block. The numeric column is a box and wants Bayesian optimisation on a sample budget in the hundreds. The prompts are not in $\mathbb{R}^n$ and are improved by reading the runs they produced and rewriting them	Section 24.10
8	Revisit blocks tuned before a later block landed, because a torch calibrated before there was anything to buy with a round was calibrated against half a game	Section 24.20

And the thing being maximised was defined once and is worth having in the same place as the procedure. **A labyrinth is interesting to the extent that its table is worth learning, hard to learn, and learnable in more than one way.** Learnable, necessary and plural, in the sense of Section 24.1, and it is a conjunction rather than a weighted sum, so no amount of one buys any of another.

Four things that will each produce a number that looks like progress. **Optimising the seed** reports a labyrinth instead of a setting. **Reading the mean** hides the one labyrinth in ten that is broken, which is the one the player will remember. **Tuning a block against fewer criteria than its mechanic can move** promotes that mechanic by measuring only what it is good at. And **leaving a condition to a rate** produces a game that is unwinnable at a rate nobody can see, which Section 24.25 says is the one place in this note where the cheap thing is not the right thing.